Agentic Systems Patterns

A practical reference for modern agent architecture: goals, loops, tools, skills, memory, protocols, multi-agent systems, and production runtimes.

Edition: 2026-06-16
Site: https://gturitto.github.io/Agentic-Systems-Patterns/
PDF: https://gturitto.github.io/Agentic-Systems-Patterns/releases/Agentic-Systems-Patterns.pdf
Repository: https://github.com/GTuritto/Agentic-Systems-Patterns

FRONT MATTER

Table of Contents

Table of Contents, Continued

Agentic Systems Patterns

This book is a practical reference for designing, testing, and operating agentic systems.

The catalog favors patterns that help engineers build real systems: control loops, goals, state, tools, skills, protocols, memory, multi-agent coordination, systems architecture, runtime observability, and evaluation.

Runnable examples live in the repository root. Pattern chapters link back to the relevant source folder when code is available. Systems Architecture chapters explain how the patterns fit together into deployable systems.

Publishing

The public site is designed for GitHub Pages at:

```
https://gturitto.github.io/Agentic-Systems-Patterns/
```

Offline release artifacts live in `book/releases`.

The generated PDF is published at:

```
https://gturitto.github.io/Agentic-Systems-Patterns/releases/Agentic-Systems-Patterns.pdf
```

License

How To Read This Book

This book can be used as a reference, a learning path, or an implementation checklist. You do not need to read every chapter in order.

First-Time Path

Start here if you are new to agentic systems:

1. Single Agent
2. Agent Loop
3. Goals and State
4. Tool Use
5. Context Engineering
6. Choosing the Right Pattern
7. Hands-On Labs

This path gives you the core vocabulary before introducing production runtime concerns.

Builder Path

Use this path when you are designing or reviewing a system:

1. Choosing the Right Pattern
2. Resource-Aware Agent Design
3. Agent Development Lifecycle
4. Evaluation-Driven Agent Development
5. Agent Security and Sandboxing
6. Reference Architecture
7. Observability and Evals

This path is best for architecture work, design reviews, and production readiness checks.

Lab Path

Use this path when you want to run code:

1. Lab 01 - Tool-Using Agent
2. Lab 02 - Agent Loop and Planning
3. Lab 03 - Agentic RAG
4. Lab 04 - A2A Communication

5. Lab 05 - Multi-Agent Supervisor
6. Lab 06 - Observability and Evals

Each lab links back to the pattern chapters and downloadable source bundles.

Reference Path

Use the sidebar or search when you need a specific pattern. Each generated pattern chapter follows the same structure:

- when to use it
- when to avoid it
- architecture
- system shape
- core protocol
- implementation notes
- failure modes
- evaluation strategy
- production checklist
- source code and downloads

The repeated structure makes chapters easy to scan during design work.

Publishing and Releases

The book is published as a GitHub Pages site and as a downloadable PDF.

Public URLs

- Book site: https://gturitto.github.io/Agentic-Systems-Patterns/
- PDF: https://gturitto.github.io/Agentic-Systems-Patterns/releases/Agentic-Systems-Patterns.pdf
- Repository: https://github.com/GTuritto/Agentic-Systems-Patterns

Release Artifacts

The checked-in PDF lives at:

```
book/releases/Agentic-Systems-Patterns.pdf
```

The GitHub Pages deployment publishes the same PDF at:

```
/releases/Agentic-Systems-Patterns.pdf
```

Each `Publish Book` workflow run also uploads the PDF as a workflow artifact. Use that artifact when you need to inspect the exact PDF produced by a specific CI run.

Local Publishing Commands

From the repository root:

```
npm run book:content
npm run book:diagrams
npm run book:pdf
npm run book:build
```

Use `npm run book:start` for local preview.

Deployment

Deployment is handled by:

```
.github/workflows/publish-book.yml
```

The workflow runs on every push to `main` and can also be triggered manually with `workflow_dispatch`.

License

This book/reference and its examples are licensed under Creative Commons Attribution-ShareAlike 4.0 International (`CC-BY-SA-4.0`).

When reusing or adapting the material, preserve attribution and share adapted material under the same license.

Single Agent

A single agent receives a goal or message, consults its context, and produces an answer or action. This is the smallest useful unit in the catalog.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

A single agent receives a goal or message, consults its context, and produces an answer or action. This is the smallest useful unit in the catalog.

Use When

- One model-backed worker can complete the task without delegation.
- The interaction has a narrow objective and a clear success condition.
- You want the simplest useful baseline before adding tools, memory, or orchestration.

Avoid When

- The task needs stateful retries, external approvals, multiple specialists, or independent evaluation.
- The agent must recover from long-running failure or resume after interruption.

System Shape

- **Pattern boundary:** a narrow agent function, class, or service boundary accepts input plus context and returns a typed answer, action, or decision.
- **State owner:** the caller or a small application service owns task state until a runtime pattern is introduced.
- **Primary artifact:** `single-agent-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** A single agent receives a goal or message, consults its context, and produces an answer or action. This is the smallest useful unit in the catalog.
- **Runnable path:** start with `npm run single-agent` before adapting the pattern to a larger system.

Core Protocol

1. Accept a bounded input, goal, or task request.
2. Assemble the minimum useful instructions, context, state, and tool descriptions.
3. Run the model or deterministic helper behind a typed boundary.
4. Validate the result before returning it to users, tools, or durable state.
5. Record enough evidence to explain the output later.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Use golden tasks that cover normal requests, ambiguous requests, missing context, and invalid input.
- Check that outputs match the expected shape and that unsafe or unsupported requests are rejected.
- Track accuracy, schema validity, latency, token use, and refusal quality.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Define the input, context, output, and error contract.
- Keep prompts, schemas, and tool descriptions versioned.
- Add deterministic tests for the smallest useful behavior.
- Log model decisions without leaking secrets or private user data.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Run the Example

```
npm run single-agent
```

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

These excerpts show the implementation shape. The complete code is available in the download bundle and repository source.

single-agent-pattern/autogen_typescript_example/single_agent.ts

Open full source

```
// Single Agent Pattern - Autogen TypeScript Example
// To run: npm install && npm run single-agent

import axios from 'axios';
import * as readline from 'readline';
import * as dotenv from 'dotenv';
dotenv.config();

const MISTRAL_API_URL = 'https://api.mistral.ai/v1/chat/completions';
const MISTRAL_API_KEY = process.env.MISTRAL_API_KEY;

async function singleAgent(userInput: string): Promise<string> {
  const response = await axios.post(
    MISTRAL_API_URL,
    {
      model: 'mistral-tiny', // or your preferred Mistral model
      messages: [{ role: 'user', content: userInput }],
    },
    {
      headers: {
        'Authorization': `Bearer ${MISTRAL_API_KEY}`,
        'Content-Type': 'application/json',
```

```
      },
    }
  );
  return response.data.choices[0].message.content;
}

async function main() {
  const idx = process.argv.indexOf('--input');
  const cliInput = idx !== -1 ? process.argv[idx + 1] : undefined;
  const nonInteractive = cliInput || process.env.NON_INTERACTIVE_INPUT;
  if (nonInteractive) {
    try {
      const agentResponse = await singleAgent(String(nonInteractive));
      console.log('Agent:', agentResponse);
    } catch (err) {
      console.error('Error:', err);
      process.exitCode = 1;
    }
    return;
  }

  const rl = readline.createInterface({ input: process.stdin, output: process.stdout
});
  rl.question('User: ', async (userInput: string) => {
    try {
      const agentResponse = await singleAgent(userInput);
      console.log('Agent:', agentResponse);
    } catch (err) {
      console.error('Error:', err);
    }
    rl.close();
  });
}

main();

// duplicate block removed
```

single-agent-pattern/langgraph_python_example/single_agent.py

Open full source

````
# Single Agent Pattern - LangGraph Python Example

This example demonstrates the Single Agent Pattern using LangGraph and Python. The agent receives a user message, sends it to a Mistral LLM, and returns the response.

## Requirements

- Python 3.8+
- `langgraph` library
- `python-dotenv` (for .env support)
- Mistral LLM API access

## Install dependencies

```bash
pip install langgraph python-dotenv requests
```

## Example Code

```python
import os
from langgraph import Agent, Environment, LLM
from dotenv import load_dotenv

load_dotenv()

MISTRAL_API_KEY = os.getenv("MISTRAL_API_KEY")
MISTRAL_API_URL = "https://api.mistral.ai/v1/chat/completions"

class SimpleEnvironment(Environment):
    def get_observation(self):
        return input("User: ")
    def send_action(self, action):
        print(f"Agent: {action}")

class SingleAgent(Agent):
    def __init__(self, llm):
        self.llm = llm
    def act(self, observation):
        return self.llm.complete(observation)
````
````

```
llm = LLM(
    provider="mistral",
    api_key=MISTRAL_API_KEY,
    api_url=MISTRAL_API_URL,
)

env = SimpleEnvironment()
agent = SingleAgent(llm)

observation = env.get_observation()
action = agent.act(observation)
env.send_action(action)
```

- Make sure your `.env` file contains your Mistral API key as `MISTRAL_API_KEY`.
- This is a minimal, functional example.
```

## Download

- Download source bundle
- Open source folder

The download bundle contains the current `single-agent-pattern/` folder from this repository.

## Related Patterns

- Agent Loop
- Goals and State
- Tool Use
- Choosing the Right Pattern
- Resource-Aware Agent Design
```

Agent Loop

The agent loop turns a model call into an agent: observe state, decide the next action, act, evaluate the result, and stop when the goal is complete or a limit is reached.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

The Agent Loop Pattern defines the runtime cycle that turns a model call into an agent: observe state, decide the next action, act through tools or messages, evaluate the result, and stop when the goal is complete or a limit is reached.

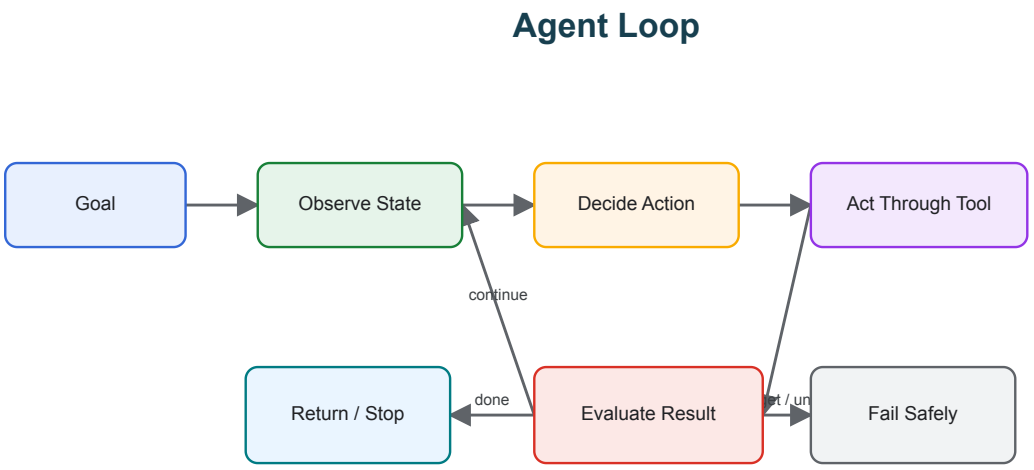
Use When

- The next step is not fully known before execution starts.
- The agent may need multiple tool calls or revisions.
- You need explicit stop conditions, budgets, and recoverable state.

Avoid When

- The task is a fixed workflow with known steps.
- A single prompt or deterministic function is sufficient.
- You cannot safely bound tool use, cost, or runtime.

Architecture



System Shape

- **Pattern boundary:** a narrow agent function, class, or service boundary accepts input plus context and returns a typed answer, action, or decision.
- **State owner:** the caller or a small application service owns task state until a runtime pattern is introduced.
- **Primary artifact:** `agent-loop-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** The agent loop turns a model call into an agent: observe state, decide the next action, act, evaluate the result, and stop when the goal is complete or a limit is reached.

Core Protocol

1. Accept a bounded input, goal, or task request.
2. Assemble the minimum useful instructions, context, state, and tool descriptions.
3. Run the model or deterministic helper behind a typed boundary.
4. Validate the result before returning it to users, tools, or durable state.
5. Record enough evidence to explain the output later.

Implementation Notes

- Persist the loop state: goal, messages, observations, tool calls, outputs, errors, and budget counters.
- Make stop conditions explicit: success predicate, max iterations, max cost, max wall-clock time, and user cancellation.

- Treat model output as a proposal. Validate tool names, tool inputs, structured outputs, and permissions before acting.
- Emit trace events for each loop iteration so failures can be debugged after the run.

Failure Modes

- Infinite loops caused by vague goals or missing stop conditions.
- Repeated tool calls with no new information.
- Hidden state drift when observations are summarized too aggressively.
- Premature success when the evaluator only checks whether an answer exists.

Evaluation Strategy

- Use golden tasks that cover normal requests, ambiguous requests, missing context, and invalid input.
- Check that outputs match the expected shape and that unsafe or unsupported requests are rejected.
- Track accuracy, schema validity, latency, token use, and refusal quality.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Define the input, context, output, and error contract.
- Keep prompts, schemas, and tool descriptions versioned.
- Add deterministic tests for the smallest useful behavior.
- Log model decisions without leaking secrets or private user data.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

This pattern currently has no dedicated code excerpt. Use the source and download links below for the full pattern folder.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `agent-loop-pattern/` folder from this repository.

Related Patterns

- Goals and State
- Tool-Using Agent
- Evaluator-Optimizer

Goals and State

Goals define success; state records progress. Together they make agent work resumable, inspectable, and easier to evaluate.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

The Goals and State Pattern separates what the agent is trying to achieve from the mutable state it accumulates while working. Goals define success; state records progress, constraints, evidence, and pending work.

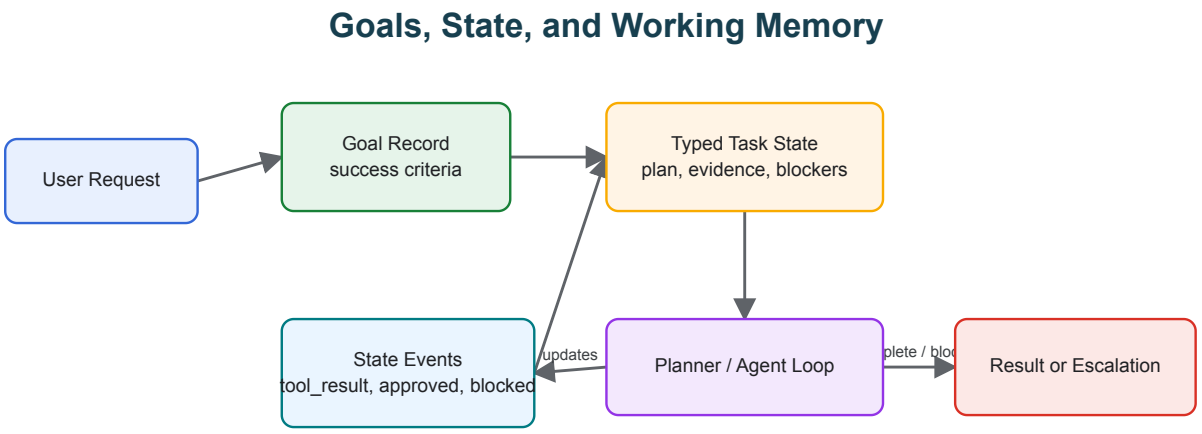
Use When

- A task spans multiple turns, tools, or agents.
- You need resumable execution after failure or interruption.
- The agent must explain progress against an explicit objective.

Avoid When

- The task is stateless and can be answered in one call.
- The goal cannot be expressed as observable success criteria.
- State would contain sensitive data you cannot store safely.

Architecture



System Shape

- **Pattern boundary:** a narrow agent function, class, or service boundary accepts input plus context and returns a typed answer, action, or decision.
- **State owner:** the caller or a small application service owns task state until a runtime pattern is introduced.
- **Primary artifact:** `goals-and-state-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Goals define success; state records progress. Together they make agent work resumable, inspectable, and easier to evaluate.

Core Protocol

1. Accept a bounded input, goal, or task request.
2. Assemble the minimum useful instructions, context, state, and tool descriptions.
3. Run the model or deterministic helper behind a typed boundary.
4. Validate the result before returning it to users, tools, or durable state.
5. Record enough evidence to explain the output later.

Implementation Notes

- Store goals as structured records with `id`, `description`, `success_criteria`, `constraints`, `owner`, and `status`.
- Store state separately from chat history. Chat is evidence; state is the operational model.
- Update state through typed events such as `step_started`, `tool_result`, `blocked`, `approved`, and `completed`.
- Make state transitions auditable and idempotent so a workflow can retry safely.

Failure Modes

- Goals that describe activity rather than success.
- State that becomes a transcript dump instead of a compact task model.
- Agents optimizing for local subgoals that no longer serve the parent goal.
- Lost cancellation or approval state after retries.

Evaluation Strategy

- Use golden tasks that cover normal requests, ambiguous requests, missing context, and invalid input.
- Check that outputs match the expected shape and that unsafe or unsupported requests are rejected.
- Track accuracy, schema validity, latency, token use, and refusal quality.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Define the input, context, output, and error contract.
- Keep prompts, schemas, and tool descriptions versioned.
- Add deterministic tests for the smallest useful behavior.
- Log model decisions without leaking secrets or private user data.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

This pattern currently has no dedicated code excerpt. Use the source and download links below for the full pattern folder.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `goals-and-state-pattern/` folder from this repository.

Related Patterns

- Agent Loop
- Planning Pattern
- Durable Workflow

Tool Use

Tool use gives an agent controlled access to external capability such as calculators, search, databases, files, code execution, APIs, or business systems.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Tool use gives an agent controlled access to external capability such as calculators, search, databases, files, code execution, APIs, or business systems.

Use When

- The model needs facts, computation, side effects, or system access outside its context window.
- Tool inputs and results can be typed, validated, and observed.
- Permissions and policy checks can sit between model intent and execution.

Avoid When

- A normal function or workflow can solve the problem without model selection.
- The tool has broad destructive permissions and no approval boundary.
- Tool results cannot be verified or traced.

System Shape

- **Pattern boundary:** a narrow agent function, class, or service boundary accepts input plus context and returns a typed answer, action, or decision.
- **State owner:** the caller or a small application service owns task state until a runtime pattern is introduced.
- **Primary artifact:** `tool-using-agent-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Tool use gives an agent controlled access to external capability such as calculators, search, databases, files, code execution, APIs, or business systems.
- **Runnable path:** start with `npm run tool-using-agent` before adapting the pattern to a larger system.

Core Protocol

1. Accept a bounded input, goal, or task request.
2. Assemble the minimum useful instructions, context, state, and tool descriptions.
3. Run the model or deterministic helper behind a typed boundary.
4. Validate the result before returning it to users, tools, or durable state.
5. Record enough evidence to explain the output later.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Use golden tasks that cover normal requests, ambiguous requests, missing context, and invalid input.
- Check that outputs match the expected shape and that unsafe or unsupported requests are rejected.
- Track accuracy, schema validity, latency, token use, and refusal quality.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Define the input, context, output, and error contract.
- Keep prompts, schemas, and tool descriptions versioned.
- Add deterministic tests for the smallest useful behavior.
- Log model decisions without leaking secrets or private user data.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Run the Example

```
npm run tool-using-agent
```

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

These excerpts show the implementation shape. The complete code is available in the download bundle and repository source.

tool-using-agent-pattern/autogen_typescript_example/tool_using_agent.ts

Open full source

```
// Tool-Using Agent Pattern - Autogen TypeScript Example
// To run: npm install && npm run tool-using-agent

import axios from 'axios';
import * as readline from 'readline';
import { evaluate } from 'mathjs';
import * as dotenv from 'dotenv';
dotenv.config();

const MISTRAL_API_URL = 'https://api.mistral.ai/v1/chat/completions';
const MISTRAL_API_KEY = process.env.MISTRAL_API_KEY;

function calculatorTool(input: string): string {
  try {
    const val = evaluate(input);
    return val.toString();
  } catch (e) {
    return `Error: ${e}`;
  }
}

async function toolUsingAgent(userInput: string): Promise<string> {
```

```
  if (userInput.toLowerCase().startsWith('calculate ')) {
    const expr = userInput.substring('calculate '.length);
    return calculatorTool(expr);
  } else {
    const response = await axios.post(
      MISTRAL_API_URL,
      {
        model: 'mistral-tiny',
        messages: [{ role: 'user', content: userInput }],
      },
      {
        headers: {
          'Authorization': `Bearer ${MISTRAL_API_KEY}`,
          'Content-Type': 'application/json',
        },
      }
    );
    return response.data.choices[0].message.content;
  }
}

async function main() {
  const idx = process.argv.indexOf('--input');
  const cliInput = idx !== -1 ? process.argv[idx + 1] : undefined;
  const nonInteractive = cliInput || process.env.NON_INTERACTIVE_INPUT;
  if (nonInteractive) {
    try {
      const agentResponse = await toolUsingAgent(String(nonInteractive));
      console.log('Agent:', agentResponse);
    } catch (err) {
      console.error('Error:', err);
      process.exitCode = 1;
    }
    return;
  }
  const rl = readline.createInterface({ input: process.stdin, output: process.stdout
});
  rl.question('User: ', async (userInput: string) => {
    try {
      const agentResponse = await toolUsingAgent(userInput);
      console.log('Agent:', agentResponse);
```

```
    } catch (err) {
      console.error('Error:', err);
    }
    rl.close();
  });
}

main();
```

tool-using-agent-pattern/langgraph_python_example/tool_using_agent.py

Open full source

```
# Tool-Using Agent Pattern - LangGraph Python Example

This example demonstrates the Tool-Using Agent Pattern using LangGraph and Python.
The agent receives a user message, determines if a tool (calculator) is needed, calls
the tool if required, and returns a response. The LLM is Mistral.

## Requirements

- Python 3.8+
- `langgraph` library
- `python-dotenv` (for .env support)
- Mistral LLM API access

## Install dependencies

```bash
pip install langgraph python-dotenv requests
```

Example Code

```python
import os
from langgraph import Agent, Environment, LLM, Tool
from dotenv import load_dotenv

load_dotenv()
```
```

```
MISTRAL_API_KEY = os.getenv("MISTRAL_API_KEY")
MISTRAL_API_URL = "https://api.mistral.ai/v1/chat/completions"

import sympy as _sp

def safe_calc(expr: str) -> str:
    try:
        return str(_sp.sympify(expr, evaluate=True))
    except Exception as e:
        return f"Error: {e}"

class CalculatorTool(Tool):
    def call(self, input_str):
        return safe_calc(input_str)

class SimpleEnvironment(Environment):
    def get_observation(self):
        return input("User: ")
    def send_action(self, action):
        print(f"Agent: {action}")

class ToolUsingAgent(Agent):
    def __init__(self, llm, tools):
        self.llm = llm
        self.tools = {tool.name: tool for tool in tools}
    def act(self, observation):
        if observation.lower().startswith("calculate "):
            expr = observation[len("calculate "):]
            return self.tools["calculator"].call(expr)
        else:
            return self.llm.complete(observation)

llm = LLM(
    provider="mistral",
    api_key=MISTRAL_API_KEY,
    api_url=MISTRAL_API_URL,
)

calc_tool = CalculatorTool(name="calculator")
env = SimpleEnvironment()
agent = ToolUsingAgent(llm, [calc_tool])
```

```
observation = env.get_observation()
action = agent.act(observation)
env.send_action(action)
```

- Type `calculate 2+2` to see the agent use the calculator tool.
- Replace credentials as needed.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `tool-using-agent-pattern/` folder from this repository.

Related Patterns

- Single Agent
- Agent Loop
- Goals and State
- Choosing the Right Pattern
- Resource-Aware Agent Design

Structured Output

Structured output constrains model responses to typed data that software can validate and consume.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

The Structured Output Pattern constrains model responses to typed data that software can validate, route, store, and test. It is the boundary between natural language reasoning and deterministic application logic.

Use When

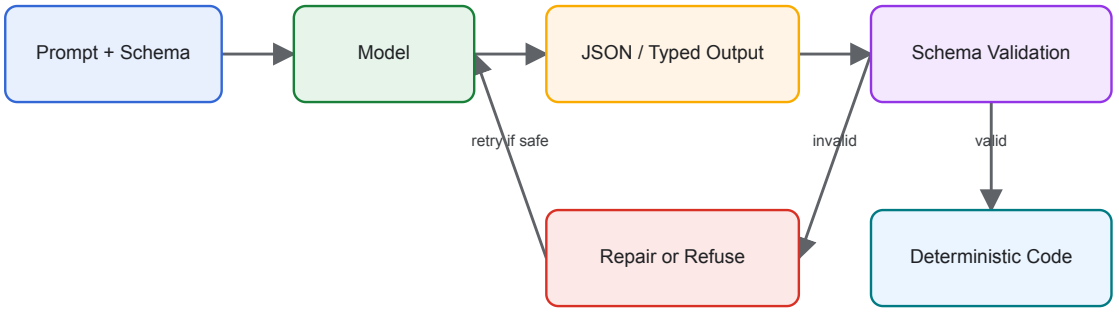
- Model output controls a tool call, workflow branch, policy decision, or database write.
- Downstream code needs stable fields rather than prose.
- You need regression tests for model-assisted behavior.

Avoid When

- The output is purely creative prose for human reading.
- A deterministic parser already handles the input safely.
- The schema is so broad that it no longer constrains behavior.

Architecture

Structured Output Validation



System Shape

- **Pattern boundary:** a narrow agent function, class, or service boundary accepts input plus context and returns a typed answer, action, or decision.
- **State owner:** the caller or a small application service owns task state until a runtime pattern is introduced.
- **Primary artifact:** `structured-output-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Structured output constrains model responses to typed data that software can validate and consume.

Core Protocol

1. Accept a bounded input, goal, or task request.
2. Assemble the minimum useful instructions, context, state, and tool descriptions.
3. Run the model or deterministic helper behind a typed boundary.
4. Validate the result before returning it to users, tools, or durable state.
5. Record enough evidence to explain the output later.

Implementation Notes

- Define schemas close to the code that consumes them.
- Validate every model response before use, even when the provider offers structured output support.
- Prefer enums for routing decisions and discriminated unions for multi-action outputs.
- Log validation failures and repair attempts as first-class evaluation data.

Failure Modes

- Schemas that mirror prose and provide little safety.
- Silent coercion of missing or invalid fields.
- Prompt-only formatting rules with no validator.
- Overly strict schemas that cause brittle failures on harmless variation.

Evaluation Strategy

- Use golden tasks that cover normal requests, ambiguous requests, missing context, and invalid input.
- Check that outputs match the expected shape and that unsafe or unsupported requests are rejected.
- Track accuracy, schema validity, latency, token use, and refusal quality.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Define the input, context, output, and error contract.
- Keep prompts, schemas, and tool descriptions versioned.
- Add deterministic tests for the smallest useful behavior.
- Log model decisions without leaking secrets or private user data.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

This pattern currently has no dedicated code excerpt. Use the source and download links below for the full pattern folder.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `structured-output-pattern/` folder from this repository.

Related Patterns

- Modern Tool Use
- LLM Router
- Compliance/Policy Enforcer

Context Engineering

Context engineering controls what the model sees: instructions, state, retrieval results, tool documentation, memory, examples, and prior messages.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Context engineering controls what the model sees: instructions, state, retrieval results, tool documentation, memory, examples, and prior messages.

Use When

- Answer quality depends on selecting the right information before generation.
- The agent combines instructions, memory, retrieval, tools, and examples.
- You need to manage context budget, freshness, and source trust.

Avoid When

- All required information is already in a short prompt.
- The system cannot distinguish trusted sources from noisy or stale material.

System Shape

- **Pattern boundary:** a narrow agent function, class, or service boundary accepts input plus context and returns a typed answer, action, or decision.
- **State owner:** the caller or a small application service owns task state until a runtime pattern is introduced.
- **Primary artifact:** `context-engineering-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Context engineering controls what the model sees: instructions, state, retrieval results, tool documentation, memory, examples, and prior messages.

Core Protocol

1. Accept a bounded input, goal, or task request.

2. Assemble the minimum useful instructions, context, state, and tool descriptions.
3. Run the model or deterministic helper behind a typed boundary.
4. Validate the result before returning it to users, tools, or durable state.
5. Record enough evidence to explain the output later.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Use golden tasks that cover normal requests, ambiguous requests, missing context, and invalid input.
- Check that outputs match the expected shape and that unsafe or unsupported requests are rejected.
- Track accuracy, schema validity, latency, token use, and refusal quality.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Define the input, context, output, and error contract.
- Keep prompts, schemas, and tool descriptions versioned.
- Add deterministic tests for the smallest useful behavior.
- Log model decisions without leaking secrets or private user data.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

These excerpts show the implementation shape. The complete code is available in the download bundle and repository source.

context-engineering-pattern/langgraph_python_example/rag_example.py

Open full source

```
"""
Context Engineering Example: Retrieval-Augmented Generation (RAG) with Mistral
Requirements: pip install langchain-community sentence-transformers faiss-cpu
requests
Note: This is a minimal example using HuggingFace embeddings by default.
"""
import os
import requests
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings
from typing import List

# Optionally load environment variables from a local .env if present
try:
    from dotenv import load_dotenv, find_dotenv  # type: ignore
    load_dotenv(find_dotenv(usecwd=True), override=True)
except Exception:
    pass

# Dummy documents
docs = [
    {"content": "Agentic systems are autonomous AI systems."},
    {"content": "Prompt engineering improves LLM outputs."}
]

# Build vector store (in-memory for demo)
texts = [d["content"] for d in docs]
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-
```

```
v2")
try:
    # Try FAISS first (fast vector index)
    vectorstore = FAISS.from_texts(texts, embeddings)
    retriever = vectorstore.as_retriever()

    def retrieve(query: str, k: int = 3):
        return retriever.get_relevant_documents(query)
except Exception:
    # Fallback: simple numpy-based cosine similarity without FAISS
    try:
        import numpy as np  # sentence-transformers depends on numpy, so this should
be available
    except Exception as e:  # pragma: no cover
        raise RuntimeError("numpy is required for the FAISS-free fallback but wasn't
found") from e

    doc_vecs = np.array(embeddings.embed_documents(texts), dtype=float)
    doc_norms = np.linalg.norm(doc_vecs, axis=1, keepdims=True) + 1e-12
    doc_vecs = doc_vecs / doc_norms

    class _MiniDoc:
        def __init__(self, text: str):
            self.page_content = text

    def retrieve(query: str, k: int = 3):
        q = np.array(embeddings.embed_query(query), dtype=float)
        q = q / (np.linalg.norm(q) + 1e-12)
        sims = (doc_vecs @ q)
        top_idx = np.argsort(-sims)[:k]
        return [_MiniDoc(texts[i]) for i in top_idx]
MISTRAL_API_KEY = os.getenv("MISTRAL_API_KEY")
if not MISTRAL_API_KEY:
    raise RuntimeError("Set MISTRAL_API_KEY in your environment.")

def chat_mistral(messages):
    resp = requests.post(
        "https://api.mistral.ai/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {MISTRAL_API_KEY}",
            "Content-Type": "application/json",
```

```
        },
        json={
            "model": "mistral-large-latest",
            "messages": messages,
            "temperature": 0.2,
        },
        timeout=60,
    )
    resp.raise_for_status()
    data = resp.json()
    return (data.get("choices") or [{}])[0].get("message", {}).get("content", "")

if __name__ == "__main__":
    query = "What are agentic systems?"
    retrieved = retrieve(query)
    context = "\n\n".join(d.page_content for d in retrieved)
    answer = chat_mistral([
        {"role": "system", "content": "Use the provided context to answer the
question succinctly."},
        {"role": "user", "content": f"Context:\n{context}\n\nQuestion: {query}"},
    ])
    print("Answer:", answer)
```

Download

- Download source bundle
- Open source folder

The download bundle contains the current `context-engineering-pattern/` folder from this repository.

Related Patterns

- Single Agent
- Agent Loop
- Goals and State
- Choosing the Right Pattern
- Resource-Aware Agent Design

Choosing the Right Pattern

Agentic design is a spectrum. The best architecture is usually the least agentic system that can meet the requirement with acceptable quality, latency, cost, and risk.

Use this chapter before choosing a framework, adding agents, or building a multi-agent topology. The decision should start with the workload, not with the most powerful pattern available.

The Autonomy Ladder

Move up the ladder only when the lower rung cannot handle the job.

Level	Pattern Shape	Use When	Main Risk
1	LLM call	A single answer, rewrite, classification, extraction, or summary is enough.	No access to live data or action.
2	Prompt chain	The work has known phases and each phase can be validated before the next one starts.	Brittle gates or unnecessary latency.
3	Deterministic workflow with LLM steps	Code owns the sequence, while LLMs handle bounded judgment inside steps.	Overfitting the workflow to today's process.
4	Routing and handoffs	Different inputs need different models, prompts, tools, agents, or policies.	Bad routing sends work to the wrong authority.
5	Single agent loop	The next step depends on observations discovered during execution.	Loops, tool misuse, and compounding errors.
6	Orchestrator-workers	The system must break an unknown task into subtasks and synthesize results.	The orchestrator becomes a hidden control plane.
7	Multi-agent system	Different specialists need separate context, tools, permissions, or review roles.	Coordination overhead, trace fragmentation, and cost growth.
8	Autonomous long-running agent	The task spans time, failures, external events, and partial progress.	Unbounded autonomy without strong state, policy, and observability.

This ladder is not a maturity model. A production system can stay at level 2 forever if the task is stable and the value is clear.

First Questions

Ask these questions before selecting a pattern:

- Is the workflow known before execution starts?
- Does the model need to choose the next step, or can code do it?
- Does the task need current data, private data, tools, or side effects?
- Can the system tolerate extra latency from multiple model calls?
- What is the maximum acceptable cost per run?
- What requires human approval?
- What evidence proves the answer or action is correct?
- What state must be replayable after a failure?
- What could go wrong if the model is persuasive but wrong?

If the answers are unclear, start with a deterministic workflow and add agentic behavior only where the workflow needs model judgment.

Selection Matrix

Workload Signal	Prefer	Avoid
Known fixed sequence	Prompt Chaining and Gates	Autonomous agents
One of several known task types	Routing and Handoffs	One giant prompt
Independent subtasks	Parallel Agents	Sequential chains that create avoidable latency
Unknown subtasks	Orchestrator-workers	Hard-coded task lists
Quality improves with review	Evaluator-Optimizer	Single-pass generation
High-risk side effects	Human Approval Gates	Direct tool execution from model output
Large tool surface	MCP-first Tool Use with policy	Broad untyped tools
Long-running work	Durable Workflows and Goals and State	Hidden in-memory loops
Sensitive data or compliance	Policy Enforcement and audit logs	Prompt-only controls
Retrieval-heavy answers	Agentic RAG Systems	Blind model responses
Debugging matters	Observability and Evals	Final-answer-only logs

The same product may combine several rows. For example, a support refund workflow may use routing to classify the request, deterministic workflow steps for account lookup, RAG for policy evidence, human approval for exceptions, and an agent loop only for open-ended investigation.

Workflows vs Agents

A workflow uses code to control the path. A model may classify, extract, summarize, critique, or generate inside a step, but software decides what happens next.

An agent uses the model to decide parts of the path. It observes state, decides the next action, calls tools, reads results, and continues until a goal is complete or a limit is reached.

Prefer workflows when:

- the process is stable;
- correctness depends on deterministic rules;
- latency or cost must stay low;
- the system must be easy to audit;
- operators need predictable failure modes.

Prefer agents when:

- the process is open-ended;
- the system must discover missing information;
- tool choice depends on intermediate observations;
- the number of steps is unknown;
- a fixed workflow would branch into an unmaintainable tree.

Complexity Budget

Every additional agent, model call, tool, memory store, and evaluator spends part of the system's complexity budget. Spend it deliberately.

Add complexity when it buys one of these outcomes:

- higher task completion rate;
- lower human effort;
- better evidence grounding;
- safer side effects;
- lower cost through routing or smaller models;
- better debuggability;
- clearer ownership boundaries.

Do not add complexity only because a pattern is popular. A multi-agent system that replaces a reliable four-step workflow usually makes the product slower, harder to test, and harder to explain.

Pattern Evolution Path

A practical evolution path looks like this:

1. Start with one prompt or deterministic workflow.
2. Add structured outputs and validation.
3. Add retrieval or tools where the model needs evidence or action.
4. Add routing when inputs diverge into distinct paths.
5. Add evaluator loops where quality improves with critique.
6. Add durable state when work spans several steps or sessions.
7. Add agents only where dynamic decisions are required.
8. Add multi-agent coordination when separate roles need separate context, tools, or permissions.

Each step should improve a measured outcome. If a pattern does not improve accuracy, reliability, latency, cost, safety, or maintainability, remove it.

Minimum Production Bar

Before a pattern handles users, money, private data, infrastructure, or customer communication, the system should have:

- typed inputs and outputs;
- explicit stop conditions;
- bounded tool permissions;
- traceable model and tool calls;
- replayable state transitions;
- eval datasets for expected behavior;
- fallback behavior for model, retrieval, and tool failure;
- human approval for high-risk actions;
- rollback or remediation for bad actions.

This minimum bar applies to simple systems too. Small systems fail faster because their boundaries are often implicit.

Related Chapters

- Prompt Chaining and Gates
- Routing and Handoffs

- Circuit Breakers, Fallbacks, and Replay
- Agent Loop
- Goals and State
- Agentic System Architecture
- Source Map

Prompt Chaining and Gates

Prompt chaining breaks a task into known LLM steps and places validation gates between them. It is one of the safest ways to add model judgment without giving the model control over the whole workflow.

Use this pattern when the order of work is known but individual steps need language understanding, generation, classification, or extraction.

Intent

Run several bounded model calls in a fixed sequence. Each call receives a narrow input, produces a typed output, and passes through a deterministic gate before the next step runs.

The model performs local judgment. Code owns the workflow.

Use When

- The task has stable phases.
- Each phase can be tested independently.
- Intermediate outputs can be validated.
- You want lower risk than an autonomous agent loop.
- You need clear failure points and retry behavior.

Common examples:

- classify a request, extract fields, verify fields, then draft a response;
- summarize a document, extract claims, check claims, then produce a final answer;
- generate code, run tests, critique failures, then revise;
- retrieve evidence, normalize citations, then synthesize an answer.

Avoid When

- The steps are unknown before execution starts.
- The chain would branch into dozens of special cases.
- No gate can verify intermediate outputs.
- The same input often needs a different sequence of work.
- Latency from multiple calls is unacceptable.

If the chain keeps growing conditional branches, consider Routing and Handoffs or an Agent Loop.

Architecture

```
Input
  -> Step 1: classify or extract
  -> Gate 1: schema, confidence, policy, completeness
  -> Step 2: transform or retrieve
  -> Gate 2: evidence, consistency, permissions
  -> Step 3: generate final output
  -> Gate 3: final eval, formatting, approval
  -> Result
```

Gates should be deterministic whenever possible. A gate can use a model as an evaluator, but the gate still needs explicit criteria and a clear pass/fail output.

Gate Types

Gate	Checks	Failure Behavior
Schema gate	JSON shape, required fields, enums, numeric ranges	Ask model to repair or fail fast.
Evidence gate	Citations, source freshness, retrieval coverage	Retrieve more evidence or escalate.
Policy gate	permissions, user role, restricted actions	Block, redact, or request approval.
Confidence gate	classification confidence, ambiguity, missing inputs	Ask a clarification question.
Consistency gate	claims match previous state and evidence	Re-run the step with narrower context.
Cost gate	model call count, token budget, time budget	Return partial result or defer.
Human gate	subjective or high-impact decision	Pause until approval.

The strongest chains mix several gate types. A support workflow may use schema gates for extracted fields, policy gates for refunds, and human gates for exceptions.

Implementation Notes

- Keep each prompt narrow. A step should have one job.
- Use structured output for every handoff between steps.
- Persist intermediate outputs, not just the final answer.
- Treat every model output as untrusted until a gate accepts it.
- Make repair loops bounded. A failed schema should not trigger infinite retries.

- Prefer deterministic gates for safety and model-based gates for subjective quality.
- Log each step with input hash, model, output, gate result, latency, and cost.

Example Chain

```
type TicketClass = 'billing' | 'technical' | 'account' | 'unknown';

interface Classification {
  type: TicketClass;
  confidence: number;
  reason: string;
}

interface ExtractedFields {
  customerId?: string;
  orderId?: string;
  issue: string;
}

interface ChainState {
  input: string;
  classification?: Classification;
  fields?: ExtractedFields;
  draft?: string;
}

function gateClassification(result: Classification): string | null {
  if (result.confidence < 0.72) return 'classification_low_confidence';
  if (result.type === 'unknown') return 'unknown_ticket_type';
  return null;
}

function gateFields(fields: ExtractedFields): string | null {
  if (!fields.issue.trim()) return 'missing_issue';
  return null;
}

async function runTicketChain(input: string): Promise<ChainState> {
  const state: ChainState = { input };

  state.classification = await classifyTicket(input);
```

```
  const classificationError = gateClassification(state.classification);
  if (classificationError) throw new Error(classificationError);

  state.fields = await extractTicketFields(input, state.classification.type);
  const fieldError = gateFields(state.fields);
  if (fieldError) throw new Error(fieldError);

  state.draft = await draftResponse(state.classification.type, state.fields);
  return state;
}
```

The example keeps control flow in code. The model classifies, extracts, and drafts, but it does not decide which validation rules apply.

Failure Modes

- A chain pretends to be deterministic while hidden prompts make important decisions.
- Gates validate shape but not meaning.
- The chain passes summarized state forward and loses key evidence.
- Repair loops keep asking the model to fix an output without changing the input or prompt.
- A model-based evaluator accepts fluent but unsupported claims.
- The chain becomes a fragile substitute for a real workflow engine.

Production Checklist

- Are all step inputs and outputs typed?
- Can each step be tested with fixture inputs?
- Does every gate have a named failure reason?
- Does every repair loop have a retry limit?
- Are intermediate outputs persisted for replay?
- Are high-risk actions separated from generation steps?
- Can operators see which gate stopped a run?

Related Chapters

- Choosing the Right Pattern
- Structured Output
- Evaluator-Optimizer
- Human Approval Gates
- Durable Workflows

Routing and Handoffs

Routing sends work to the right model, prompt, tool, workflow, or agent. Handoffs transfer responsibility with enough typed state for the next owner to act safely.

This pattern is often the missing middle between a single prompt and a multi-agent system.

Intent

Use a classifier, policy rule, planner, or deterministic condition to choose the next execution path. Then pass a narrow state object to the selected path.

A router should decide where work belongs. It should not complete the work itself.

Use When

- Inputs belong to distinct domains or intents.
- Different paths need different tools, permissions, models, or prompts.
- Simple requests should use cheap, fast paths.
- Risky requests need stricter policy or human approval.
- A large agent is struggling because it has too many tools or responsibilities.

Common examples:

- route support tickets to billing, technical, account, or fraud workflows;
- route easy requests to a small model and unusual requests to a stronger model;
- route code tasks to search, edit, review, or test agents;
- route requests with side effects through approval workflows;
- route RAG questions to the correct index, tenant, or domain corpus.

Avoid When

- There is only one meaningful path.
- The router cannot explain or score its decision.
- The downstream paths have overlapping ownership.
- A wrong route would create dangerous side effects.
- The router depends on hidden conversation context that is not persisted.

If the route cannot be chosen with confidence, ask for clarification or send the task to a safe fallback path.

Architecture

```
Request
  -> Normalize input
  -> Classify intent, risk, domain, and required capability
  -> Apply policy and budget rules
  -> Select path
  -> Handoff typed state
  -> Downstream path completes or escalates
```

The handoff should include only the state the downstream path needs. Avoid dumping the full conversation into every specialist.

Router Outputs

A router output should be structured:

```
type Route =
  | 'answer_from_docs'
  | 'billing_workflow'
  | 'technical_triage'
  | 'human_review'
  | 'reject'
  | 'clarify';

interface RoutingDecision {
  route: Route;
  confidence: number;
  reason: string;
  requiredCapabilities: string[];
  risk: 'low' | 'medium' | 'high';
}
```

Good router decisions are inspectable. The system should log the route, confidence, reason, model version, policy checks, and fallback behavior.

Handoff Contract

A handoff is not just a message. It is a change in responsibility.

Include:

- normalized user intent;
- route decision and reason;
- relevant extracted fields;
- evidence references;
- permission scope;
- budget remaining;
- deadline or timeout;
- previous failed routes or tool failures;
- expected output contract;
- escalation path.

Exclude:

- unrelated conversation history;
- tools the recipient cannot use;
- hidden policy text that should remain system-owned;
- raw sensitive data when a reference or redacted field is enough.

Router Types

Router Type	Best For	Watch Out For
Rule router	Compliance, tenant, role, known metadata	Rule drift as products change.
Model router	Natural language intent and ambiguous requests	Low confidence and prompt injection.
Embedding router	Domain corpus or knowledge base selection	Similar but unsafe corpora.
Cost router	Model tier selection	Cheap model overuse on hard tasks.
Risk router	Approval, sandbox, or policy paths	Missing high-risk edge cases.
Capability router	Tool or agent selection	Overlapping tool descriptions.

Production systems usually combine them. For example, code may enforce tenant and policy rules before a model chooses between specialist workflows.

Implementation Notes

- Keep route labels stable. Treat route names as API contracts.
- Route before loading large context.
- Separate intent routing from policy routing.
- Use confidence thresholds with explicit clarify or fallback paths.
- Validate handoff state before the recipient starts.

- Do not let downstream agents silently reroute forever.
- Trace route decisions across the whole run.
- Test routers with adversarial and ambiguous examples, not only happy paths.

Failure Modes

- A router becomes a hidden general-purpose agent.
- Route labels overlap, so the same request can fit several paths.
- Handoffs lose authority boundaries and give specialists too many tools.
- Low-confidence routes proceed instead of asking for clarification.
- Multi-agent handoffs create loops with no owner.
- Cost routing sends hard tasks to weak models and hides quality loss.

Production Checklist

- Are route labels mutually understandable and stable?
- Does every route have an owner and output contract?
- Does the router have a fallback for low confidence?
- Does policy run before dangerous handoffs?
- Can an operator explain why a request took a path?
- Are route datasets part of regression tests?
- Are route loops impossible or bounded?

Related Chapters

- Choosing the Right Pattern
- MCP-first Tool Use
- A2A Agent Interoperability
- Supervisor / Worker
- Policy Enforcement
- Context Engineering

Resource-Aware Agent Design

Agentic systems spend resources with every model call, retrieval, tool call, handoff, memory operation, and retry. Resource-aware design makes cost, latency, context, and compute part of the architecture.

Use this pattern when an agent must run repeatedly, serve users interactively, or operate under budget constraints.

Intent

Optimize for completed tasks, not raw model intelligence.

A resource-aware agent chooses the cheapest safe path that can complete the job with acceptable quality.

Resource Types

Resource	What To Track
Tokens	prompt tokens, output tokens, hidden context, compression cost.
Model calls	count, model tier, retries, parallel calls.
Latency	end-to-end time, step time, tool wait time, human wait time.
Money	cost per run, cost per completed task, cost per route.
Context window	relevant context, stale context, duplicated context.
Tool quota	API rate limits, browser sessions, database load.
Human attention	approvals, corrections, escalations, review time.
Operational risk	side effects, policy checks, incident cost.

Human attention is a resource. A cheap model path that creates constant review work is not cheap.

Budget Objects

Give each run a budget object:

```
interface AgentBudget {
  maxCostUsd: number;
  maxModelCalls: number;
  maxToolCalls: number;
```

```
  maxIterations: number;
  maxWallClockMs: number;
  maxContextTokens: number;
  approvalRequiredAboveUsd?: number;
}
```

Budgets should travel with state so routers, tools, evaluators, and fallback logic can make consistent decisions.

Optimization Levers

Lever	Use When	Trade-off
Model routing	Some tasks are easy and some are hard.	Route mistakes can reduce quality.
Prompt chaining	Steps are known and can be validated.	More calls, but less context per call.
Context minimization	Context is large or noisy.	May omit useful evidence.
Retrieval filtering	Sources are numerous or mixed quality.	Filters can miss edge cases.
Parallelization	Independent subtasks dominate latency.	More total calls and harder aggregation.
Caching	Queries or tool outputs repeat.	Cache invalidation and freshness risks.
Early exit	Confidence is high enough.	Needs calibrated thresholds.
Human escalation	Automation would be risky or expensive.	Uses scarce human attention.

Optimize only after you can trace costs by route and step.

Context Compression

Compress context when it is too large, repeated, or stale.

Prefer:

- structured state over long chat history;
- file or record references over pasted content;
- retrieved snippets over entire documents;
- task-specific memory over global memory;
- summaries with source pointers;

- explicit deletion of irrelevant context.

Do not compress away evidence, constraints, tool errors, approvals, or unresolved questions.

Cost-Aware Routing

Cost-aware routing sends work to the cheapest safe path:

- deterministic code for known operations;
- small model for classification and extraction;
- stronger model for planning or synthesis;
- specialized agent for domain work;
- human review for high-risk ambiguity.

The router needs evals. Otherwise cost savings can silently become quality loss.

Latency Design

Interactive agents need visible progress and bounded wait.

Use:

- streaming status at workflow boundaries;
- parallel retrieval where safe;
- background jobs for long work;
- partial results with clear limits;
- cancellation;
- resumable state.

For long-running tasks, optimize for resumability before raw speed.

Failure Modes

- Optimizing token count while increasing human correction work.
- Using a cheap model for routes it cannot classify reliably.
- Parallelizing dependent work and creating inconsistent results.
- Compressing context so aggressively that the agent forgets constraints.
- Caching stale evidence in regulated or fast-changing domains.
- Measuring cost per call instead of cost per completed task.

Related Chapters

- Choosing the Right Pattern

- Routing and Handoffs
- Context Engineering
- Working Memory
- Observability and Evals
- Circuit Breakers, Fallbacks, and Replay

Circuit Breakers, Fallbacks, and Replay

Agentic systems need controls that stop waste, contain damage, and make failures explainable. Circuit breakers stop unsafe or unproductive execution. Fallbacks give the system a safer next move. Replay lets engineers reconstruct what happened.

This is a reliability pattern for agent loops, tool use, RAG, workflow orchestration, and multi-agent systems.

Intent

Protect the system from repeated failure and make every run recoverable enough to debug.

An agent should not keep calling the same failing tool, searching the same empty corpus, revising the same bad draft, or handing the same task between agents without progress.

Use When

- Agents can call tools, APIs, browsers, shells, or workflows.
- Work can loop, retry, delegate, or wait.
- Costs can grow with each model or tool call.
- Partial state matters after failure.
- Operators need to explain why a run stopped.

This pattern is mandatory for production systems with side effects.

Avoid When

- The task is a throwaway prototype with no external effect.
- The system has no loop, retry, or tool use.
- Failures do not need investigation.

Even then, keep a simple run log. The prototype that works often becomes the production seed.

Circuit Breakers

A circuit breaker turns repeated or high-risk failure into a stop, fallback, or escalation.

Breaker	Trigger	Action
Tool failure breaker	Same tool fails N times or returns malformed output.	Disable tool for run and choose fallback.

Breaker	Trigger	Action
No-progress breaker	State does not change across iterations.	Stop loop and return blocked status.
Cost breaker	Token, model-call, or money budget is reached.	Stop, summarize progress, and ask for approval.
Latency breaker	Step or run exceeds time budget.	Defer, enqueue, or return partial result.
Retrieval breaker	Search returns low-coverage or conflicting evidence.	Ask clarification or escalate.
Policy breaker	Tool intent violates permissions or risk rules.	Block action and log policy reason.
Handoff breaker	Agents bounce a task between roles.	Assign owner or escalate to human.
Output breaker	Final answer fails schema, citation, or eval checks.	Repair once, then fail safely.

Breakers need names and thresholds. “The agent got stuck” is not operational enough. “No-progress breaker fired after 4 iterations with unchanged evidence set” is debuggable.

Fallbacks

A fallback should be safer than the failed path.

Useful fallback types:

- ask the user for missing information;
- return a partial answer with explicit limits;
- switch from autonomous tool use to deterministic workflow;
- switch from a weak model to a stronger model;
- switch from write action to read-only analysis;
- use cached or previously verified data;
- route to human review;
- schedule background processing;
- fail closed for restricted actions.

Do not use fallback as a way to hide failure. The user or operator should know what changed.

Replay

Replay is the ability to reconstruct a run from durable state.

Store:

- run ID and parent run ID;
- goal and normalized intent;
- model calls, model versions, parameters, and prompt references;
- tool calls, inputs, outputs, errors, and side-effect identifiers;
- retrieval queries, source IDs, and citation metadata;
- state transitions;
- route decisions and handoffs;
- policy checks;
- breaker and fallback events;
- final output and eval results.

Redact sensitive content when required, but keep enough references to investigate the run.

Replay Levels

Level	Capability	Use
Trace replay	Reconstruct what happened from logs.	Incident review and debugging.
Deterministic replay	Re-run deterministic code with recorded model/tool outputs.	Regression tests and workflow validation.
Full replay	Re-run model and tool calls in a sandbox.	Reproduction when external systems are stable.
Counterfactual replay	Re-run with changed prompt, policy, model, or tool.	Evaluate fixes before rollout.

Most teams should start with trace replay. Full replay is useful but harder because models, tools, and external data change.

Implementation Notes

- Add breakers at the loop, tool, retrieval, route, and workflow levels.
- Record breaker events as first-class trace events.
- Tie fallback decisions to explicit failure reasons.
- Separate safe retries from repeated guessing.
- Make every side effect idempotent or traceable.
- Store enough state to resume or fail cleanly.
- Turn incidents into eval cases.

Example Breaker Logic

```
interface ToolFailure {
  tool: string;
```

```
  count: number;
  lastError: string;
}

interface RunBudget {
  maxIterations: number;
  maxToolFailuresPerTool: number;
  maxCostUsd: number;
}

function shouldOpenToolBreaker(
  failure: ToolFailure,
  budget: RunBudget
): boolean {
  return failure.count >= budget.maxToolFailuresPerTool;
}

function shouldStopLoop(iteration: number, costUsd: number, budget: RunBudget):
string | null {
  if (iteration >= budget.maxIterations) return 'max_iterations_reached';
  if (costUsd >= budget.maxCostUsd) return 'max_cost_reached';
  return null;
}
```

The important part is not the code size. The important part is that the stop reason becomes part of the run state and trace.

Failure Modes

- Breakers exist in logs but do not affect execution.
- Fallbacks silently lower quality without telling the user.
- Retries repeat the same inputs and expect different outcomes.
- Traces omit tool inputs, making replay impossible.
- Side effects cannot be matched to the agent run that created them.
- Multi-agent systems have no single owner when a breaker fires.

Production Checklist

- Does every loop have max iterations, max cost, and max wall-clock time?
- Does every tool have timeout, retry, and failure thresholds?
- Does every fallback tell the operator what changed?

- Can the system stop safely after partial progress?
- Can a failed run become an eval fixture?
- Can side effects be audited and reversed when possible?
- Can operators replay a run without reading raw prompts from scattered logs?

Related Chapters

- Agent Loop
- Goals and State
- Self-Healing Workflows
- Durable Workflows
- Observability and Evals
- Policy Enforcement

Source Map

This page maps external references to this book's chapters. Use it as a reading guide, not as a replacement for the book's pattern language.

The sources repeat several core ideas: start simple, separate workflows from agents, use tools through typed contracts, add memory deliberately, evaluate behavior, and reserve multi-agent systems for cases where specialization justifies orchestration cost.

Primary References

Source	Useful Ideas	Book Mapping
Anthropic: Building Effective Agents	Workflows vs agents, prompt chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer, autonomous agents.	Choosing the Right Pattern, Prompt Chaining and Gates, Routing and Handoffs, Evaluator-Optimizer, Agent Loop.
Google Cloud: Choose a design pattern for your agentic AI system	Requirements-first selection, single-agent and multi-agent patterns, sequential, parallel, iterative refinement, human-in-the-loop, custom logic.	Choosing the Right Pattern, Parallel Agents, Human Approval Gates, Agentic System Architecture.
Databricks: Agent system design patterns	Complexity continuum from prompt to deterministic chain to single-agent and multi-agent systems, plus production guidance for testing, tracing, failure handling, model updates, and cost.	Choosing the Right Pattern, Circuit Breakers, Fallbacks, and Replay, Observability and Evals.
MongoDB: 7 Practical Design Patterns for Agentic Systems	Controlled flows, LLM routing, parallelization, reflection, human-in-the-loop, agents, and multi-agent architectures.	Prompt Chaining and Gates, Routing and Handoffs, Reflection, Supervisor / Worker.
Agentic Patterns Graph	Large pattern catalog with categories for context, orchestration, reliability, security, tool use, learning, and UX.	Circuit Breakers, Fallbacks, and Replay, Context Engineering, Working Memory, Policy Enforcement.
GitHub: awesome-agentic-patterns	Curated repository of production and emerging patterns, useful as a discovery index.	This source informs the extended catalog and future additions, especially reliability, context, coding-agent, and tool-use patterns.

Secondary References

Source	Useful Ideas	How To Use
ByteByteGo: Top AI Agentic Workflow Patterns	Accessible explanations of reflection, tool use, ReAct, planning, and multi-agent patterns.	Good introductory reading for readers who want a lighter explanation before the deeper chapters.
Phil Schmid: Zero to One - Learning Agentic Patterns	Practical examples for routing, parallelization, tool use, orchestrator-workers, and multi-agent systems.	Useful for implementation intuition and example shapes.
ProjectPro: Agentic AI Design Patterns	Planning, tool use, reflection, multi-agent examples, and a practical MCP discussion.	Useful for explaining common enterprise examples.
Tungsten Automation: Tool-Use Pattern	Enterprise framing for tool use, ground truth, and workflow APIs.	Maps to MCP-first Tool Use and Policy Enforcement.
Towards AI: 5 Design Patterns in Agentic AI Workflow	Introductory framing for prompt chaining and workflow decomposition.	Useful as a light overview; the article is partially gated.
Medium: Agentic Design Patterns	Common pattern explanations for reflection, tool use, planning, and multi-agent design.	Duplicates core concepts already covered here.
Medium: Agentic Patterns - Architectures for Coordinated AI Systems	Hierarchical, peer-to-peer, market-based, and swarm coordination.	Useful for future expansion of multi-agent coordination patterns.
YouTube: AI agent design patterns	Video walkthrough of agent design patterns.	Use as companion media, not as a primary textual source unless transcript review is added.
YouTube: Master ALL 20 Agentic AI Design Patterns	Broad video catalog of pattern names and examples.	Use as discovery material; validate individual claims against primary sources.

The LinkedIn checkpoint link was not useful as a public source because it requires a login challenge.

Local PDF References Reviewed

The following PDFs were reviewed locally as source-of-knowledge inputs. They are not included in this repository, linked from the book, or reproduced. They informed coverage checks, chapter structure, and topic prioritization only.

Local Reference	Useful Themes	Book Mapping
30 Agents Every AI Engineer Must Build, Imran Ahmad	Agent engineering foundations, toolkit decisions, domain agents, retrieval agents, tool orchestration, software agents, explainability, and domain-specific agents.	Agent Engineer Toolkit, Framework Selection, Domain Agent Architectures, Coding Agents, Agentic RAG Systems.
Agentic Architectural Patterns for Building Multi-Agent Systems, Ali Arsanjani, Juan Pablo Bustos, Thomas Kurian	Enterprise maturity, agent-ready model selection, RAG-to-fine-tuning spectrum, coordination patterns, compliance, robustness, human-agent interaction, production readiness.	Choosing the Right Pattern, Agent Development Lifecycle, Agent Security and Sandboxing, Domain Agent Architectures, Agentic System Architecture.
Agentic Design Patterns, Antonio Gulli	Prompt chaining, routing, parallelization, reflection, tool use, planning, memory, MCP, A2A, monitoring, guardrails, resource optimization, CLI and coding agents.	Prompt Chaining and Gates, Routing and Handoffs, Resource-Aware Agent Design, MCP-first Tool Use, A2A Agent Interoperability.
Designing Multi-Agent Systems, Victor Dibia	Multi-agent taxonomy, UX principles, execution loops, cancellation, memory, middleware, computer-use agents, workflow graphs, observability, evaluation, distributed protocols, ethics.	Agent UX and Human Trust, Evaluation-Driven Agent Development, Computer-Use Agents, Supervisor / Worker, Secure Agent Communication.
Patterns for Building AI Agents, Sam Bhagwat and Michelle Gienow	Agent capability design, context engineering, context compression, eval suites, production data evaluation, sandboxing, granular access control, guardrails.	Context Engineering, Resource-Aware Agent Design, Evaluation-Driven Agent Development, Agent Security and Sandboxing, Circuit Breakers, Fallbacks, and Replay.

Pattern Coverage Map

External Pattern Name	Current Book Chapter
Augmented LLM	Single Agent, Tool Use, Structured Output
Prompt chaining	Prompt Chaining and Gates
Routing	Routing and Handoffs
Parallelization	Parallel Agents
Orchestrator-workers	Supervisor / Worker, Planning and Execution
Evaluator-optimizer	Evaluator-Optimizer

External Pattern Name	Current Book Chapter
Reflection	Reflection
ReAct	ReAct
Agent loop	Agent Loop
Human-in-the-loop	Human Approval Gates
Tool use	Tool Use, MCP-first Tool Use
Memory and context	Context Engineering, Working Memory, Long-Term Episodic Memory
Resource-aware optimization	Resource-Aware Agent Design
Agentic RAG	Semantic Recall and RAG, Agentic RAG Systems
Multi-agent supervisor	Supervisor / Worker
Peer or network agents	A2A Agent Interoperability, Debate and Consensus
Circuit breaker	Circuit Breakers, Fallbacks, and Replay
Action replay	Circuit Breakers, Fallbacks, and Replay, Observability and Evals
Deterministic chain	Choosing the Right Pattern, Durable Workflows
Coding agent	Coding Agents
Computer-use agent	Computer-Use Agents
Domain-specific agent	Domain Agent Architectures

Editorial Rule

External sources are used to validate coverage and expose missing patterns. The book keeps its own taxonomy:

- foundations first;
- control loops second;
- memory and knowledge as a separate concern;
- tools, skills, and protocols as integration boundaries;
- multi-agent systems only when specialization has value;
- production runtime patterns for safety, evaluation, and operations;
- source-informed pattern selection as the entry point for choosing the right design.

This keeps the book useful as a reference instead of turning it into a link dump.

Agent Development Lifecycle

Agent development is not prompt writing with deployment at the end. It is a product engineering lifecycle with requirements, architecture, implementation, evaluation, operations, and governance.

Use this lifecycle when an agent will serve real users, call tools, touch private data, or run for more than a demo session.

Lifecycle Stages

Stage	Main Question	Output
1. Capability framing	What should the agent do, and what should it never do?	Capability map, exclusions, risk class.
2. Pattern selection	What is the least agentic architecture that works?	Pattern choice and complexity budget.
3. Boundary design	Where do model decisions stop and software controls begin?	Tool contracts, policies, state model.
4. Implementation	How does the agent perceive, decide, act, and stop?	Runnable agent, workflow, or multi-agent system.
5. Evaluation	How do we know it works?	Eval suite, failure modes, quality gates.
6. Deployment	How does it run safely in production?	Observability, rollback, rate limits, approvals.
7. Governance	How does it improve without losing control?	Review process, audit logs, versioning, incident loop.

The lifecycle is iterative. Production data should update evals, policies, prompts, and tool design.

Capability Framing

Start with a capability map, not a framework choice.

Capture:

- user jobs and business outcomes;
- allowed actions;
- forbidden actions;
- required evidence;

- required tools and data;
- approval points;
- privacy and compliance constraints;
- acceptable latency and cost;
- expected failure behavior.

An agent with a clear “will not do” list is easier to ship than an agent defined only by what it might do.

Pattern Selection

Select the simplest pattern that handles the task:

- Use a prompt for a single bounded task.
- Use a prompt chain for known phases.
- Use routing when inputs need different paths.
- Use an agent loop when the next step depends on observations.
- Use multi-agent systems when separate roles need separate context, tools, or permissions.

Do not start with a multi-agent architecture because the domain has many tasks. Start by grouping capabilities into stable workflows, then split agents only where role boundaries are real.

Boundary Design

Before implementation, define the system boundaries:

- What owns state?
- What owns policy?
- What owns tool permissions?
- What owns memory writes?
- What owns final approval?
- What can be replayed after failure?
- What can be changed without redeploying code?

The model may propose actions. Software should validate, execute, log, and enforce.

Implementation

Implementation should make the lifecycle visible in code.

Minimum implementation units:

- a goal or request object;
- typed state;
- tool schemas;
- policy checks;
- stop conditions;
- trace events;
- eval fixtures;
- deployment configuration.

Avoid implementations where the only durable artifact is a conversation transcript. Transcripts are useful, but they are not a state model.

Evaluation

Evaluate before launch and after launch.

Pre-launch evaluation should include:

- happy-path tasks;
- ambiguous user requests;
- missing data;
- tool failure;
- retrieval failure;
- prompt injection attempts;
- approval-required actions;
- cost and latency budgets.

Post-launch evaluation should add production failures, user corrections, human override cases, and edge cases found in traces.

Deployment

Production deployment needs more than a hosted endpoint.

Include:

- model and prompt versioning;
- rate limits and cost budgets;
- tool-level timeouts;
- audit logs;
- trace IDs across tools and agents;
- alerting for breaker events;
- human escalation paths;

- rollback for prompts, policies, and tools.

If a deployment cannot explain what the agent did, it is not ready for high-impact work.

Governance

Governance is the control loop around the agent.

Review:

- new tools;
- prompt and policy changes;
- memory schemas;
- eval changes;
- approval thresholds;
- incident reports;
- model upgrades.

Treat agent changes like product and infrastructure changes. Small prompt edits can change behavior as much as code changes.

Related Chapters

- Choosing the Right Pattern
- Goals and State
- MCP-first Tool Use
- Observability and Evals
- Policy Enforcement
- Agentic System Architecture

Agent Engineer Toolkit

An agent engineer chooses models, tools, memory, orchestration, evals, and deployment infrastructure as one system. The toolkit should support the architecture, not define it.

Use this chapter to decide what to build directly and what to take from a framework.

Toolkit Layers

Layer	Responsibility	Examples of Decisions
Model layer	Reasoning, generation, tool calling, structured output.	Model quality, latency, cost, context window, deployment mode.
Orchestration layer	Workflow, routing, loops, state, retries.	Code workflow, LangGraph-style graph, CrewAI-style crew, Mastra-style runtime.
Tool layer	External actions and data access.	MCP servers, internal APIs, browser tools, code execution, database functions.
Memory layer	Working, episodic, semantic, and user memory.	Vector index, relational store, filesystem state, memory write policy.
Evaluation layer	Quality and regression checks.	Golden datasets, judges, human labels, production traces.
Observability layer	Debugging and operations.	Traces, metrics, costs, logs, run replay.
Security layer	Permissions and containment.	Sandboxes, secrets, policy gates, audit, egress controls.
UX layer	Human interaction and trust.	Streaming, approvals, explanations, handoffs, recovery.

The layers can come from one framework, several libraries, or direct code. The important part is that each layer has an owner.

Build vs Framework

Use a framework when it removes real operational complexity:

- graph execution;
- durable state;
- tracing;
- tool registration;
- eval integration;

- human approval flows;
- deployment management;
- multi-agent orchestration.

Build directly when:

- the workflow is small;
- the team needs full control over state and policy;
- framework abstractions hide important failures;
- the system must integrate with existing infrastructure;
- the agent is part of a larger product workflow.

Frameworks should make boundaries clearer. If a framework makes it hard to see state, tool calls, permissions, or failure modes, use a smaller abstraction.

Framework Evaluation Checklist

Before adopting a framework, test it against the agent's production requirements.

- Can it persist and resume state?
- Can it represent approval gates?
- Can it restrict tools per role or route?
- Can it trace model calls, tool calls, and handoffs?
- Can it run evals against realistic fixtures?
- Can it support model fallback and routing?
- Can it expose cost and latency by step?
- Can it run in the team's deployment environment?
- Can engineers debug failures without reading framework internals?
- Can it be removed later if the product outgrows it?

Do a thin vertical slice before committing the whole architecture.

Model Selection

Choose models by workload, not benchmark rank alone.

Evaluate:

- structured output reliability;
- tool-use reliability;
- instruction following;
- latency;
- cost per completed task;
- context window behavior;

- refusal and safety behavior;
- multimodal needs;
- support for deployment constraints;
- provider stability and versioning.

Use multiple models when it reduces cost or improves quality. A small model may route, classify, or extract. A stronger model may plan, synthesize, or resolve edge cases.

Tooling Decisions

Tools need engineering discipline.

Good tools have:

- narrow names;
- explicit descriptions;
- typed inputs;
- typed outputs;
- permission scopes;
- timeouts;
- idempotency where possible;
- clear side-effect labels;
- test fixtures.

Poor tools expose broad surfaces such as unrestricted shell, generic browser control, or “database query” without policy checks. Broad tools can be useful for trusted coding agents, but they need sandboxing and audit.

Memory Decisions

Memory is not one feature. It is several stores with different rules.

- Working memory holds the current task state.
- Episodic memory stores previous runs and user interactions.
- Semantic memory stores facts and documents.
- Procedural memory stores skills, playbooks, and preferences.

Decide who can write memory, how writes are validated, how old memory expires, and how retrieval is cited.

Toolkit Anti-Patterns

- Choosing a framework before choosing a pattern.

- Giving every agent every tool.
- Treating vector search as memory without write policy.
- Logging final answers but not tool decisions.
- Using model routing without evals for route quality.
- Shipping a demo framework path without production state, policy, and observability.

Related Chapters

- Choosing the Right Pattern
- MCP-first Tool Use
- Working Memory
- Mastra Runtime
- CrewAI Flows and Crews
- Agentic System Architecture

Framework Selection

Agent frameworks can accelerate development, but they also shape the system's boundaries. Select a framework after you know the pattern, state model, tools, eval needs, and deployment constraints.

Use this chapter to compare frameworks without turning the architecture into a framework demo.

Intent

Choose whether to build directly, use a lightweight library, or adopt a full agent framework.

The right framework should make the important parts more visible: state, tools, policy, traces, handoffs, and failure modes.

Selection Criteria

Criterion	Questions
Control flow	Does it support chains, graphs, loops, routing, and human gates?
State	Can it persist, resume, inspect, and migrate state?
Tooling	Can tools be typed, scoped, tested, and audited?
Multi-agent	Can it represent roles, handoffs, and ownership clearly?
Memory	Does it separate working, episodic, semantic, and procedural memory?
Evaluation	Can evals run against routes, tools, traces, and final outputs?
Observability	Does it expose model calls, tool calls, costs, latency, and errors?
Security	Can tools and agents run with least privilege?
Deployment	Can it run where your system must run?
Escape hatch	Can you drop to code when the abstraction is wrong?

Do not adopt a framework that hides a concern your product must control.

Common Framework Shapes

Shape	Strength	Risk
Prompt and tool library	Fast for simple agents.	You must build state, evals, and operations yourself.
Graph/workflow framework	Clear execution paths, durable state, conditional edges.	Graphs can become hard to read if every branch becomes a node.

Shape	Strength	Risk
Multi-agent framework	Role and delegation primitives.	Easy to overuse agents where a workflow is enough.
RAG framework	Retrieval, indexes, and data connectors.	Retrieval can become disconnected from policy and evals.
Runtime framework	Deployment, tracing, memory, workflows, and operations.	Runtime choices can become platform lock-in.

The best choice may combine a small number of layers instead of one large framework.

Build Directly When

- The workflow is short and stable.
- You need exact control over state and policy.
- The team already has workflow infrastructure.
- The agent is embedded inside an existing product.
- Debuggability matters more than rapid prototyping.
- The framework would introduce more concepts than the domain requires.

Direct code is often the best first production version for a narrow workflow.

Use a Framework When

- The system needs durable graph execution.
- Agents need tool registration and discovery.
- Multi-agent handoffs are central to the product.
- You need built-in tracing, evals, or deployment support.
- You will build many related agents.
- The team benefits from shared conventions.

Use a framework to make repeated patterns consistent, not to avoid understanding the architecture.

Evaluation Process

Run a framework through a vertical slice:

1. one real user request;
2. one route or workflow;
3. one read tool;
4. one write tool behind approval;
5. one memory read or write;

6. one failure path;
7. one eval case;
8. one trace inspection;
9. one deployment path.

If the slice is hard to inspect, hard to test, or hard to secure, stop before the framework spreads.

Framework Anti-Patterns

- Selecting a framework because it supports every pattern.
- Replacing architecture decisions with framework defaults.
- Giving all framework agents the same context and tools.
- Accepting opaque traces.
- Skipping evals because the framework demo worked.
- Treating framework memory as trustworthy by default.

Related Chapters

- Agent Engineer Toolkit
- Choosing the Right Pattern
- Mastra Runtime
- CrewAI Flows and Crews
- Agentic System Architecture

Evaluation-Driven Agent Development

Agent quality improves when evaluation drives development. A useful eval suite tells the team what fails, why it matters, and whether a change made the system better.

Use this chapter when an agent is moving from prototype to production.

Intent

Build agents from failure modes and success metrics, not from demos.

The eval loop is:

1. list the tasks the agent must handle;
2. list the ways those tasks can fail;
3. define business and quality metrics;
4. create fixtures and datasets;
5. run the agent against those datasets;
6. inspect failures;
7. improve prompts, tools, state, policy, or architecture;
8. repeat on every meaningful change.

What To Evaluate

Evaluate the full trajectory, not only the final answer.

Layer	What To Check
Routing	Correct path, confidence, fallback behavior.
Planning	Valid steps, dependency order, missing constraints.
Retrieval	Source relevance, freshness, coverage, citations.
Tool use	Correct tool, valid input, safe side effects, error handling.
Memory	Correct recall, safe write, no stale or private leakage.
Policy	Permissions, approval requirements, refusals.
Final output	Correctness, completeness, tone, format, evidence.
Operations	Latency, cost, retries, breaker events, escalation.

If only the final response is evaluated, the team will miss the failure that caused it.

Failure Mode Inventory

Start each agent project with a failure mode inventory.

Examples:

- selects the wrong route;
- asks for information it already has;
- misses a required approval;
- calls a write tool when read-only analysis is enough;
- retrieves irrelevant evidence;
- trusts stale memory;
- loops without progress;
- gives a confident answer with weak evidence;
- exposes sensitive data;
- exceeds cost or latency budget;
- cannot resume after interruption.

Each failure mode should map to at least one test, trace query, or production alert.

Metrics

Use a small set of metrics that match the product.

Common metrics:

- task completion rate;
- correctness;
- evidence coverage;
- tool-call accuracy;
- route accuracy;
- approval precision and recall;
- hallucination rate;
- user correction rate;
- human escalation rate;
- cost per completed task;
- latency by step;
- incident rate.

Do not optimize one metric in isolation. A lower escalation rate is bad if the agent is making unsafe decisions instead.

Eval Dataset Types

Dataset	Source	Use
Golden tasks	Handwritten examples	Basic regression coverage.
Adversarial tasks	Designed edge cases	Safety, routing, policy, prompt injection.
Production traces	Real runs with redaction	Realistic behavior and long-tail failures.
Human-labeled data	Subject matter experts	Ground truth for judgment-heavy tasks.
Synthetic variants	Generated from known cases	Coverage expansion after human review.
Incident fixtures	Past failures	Prevent regressions.

Production traces are valuable, but they need privacy controls and careful labeling.

Judges

Model judges can help, but they need constraints.

Use model judges for:

- subjective quality;
- rubric-based review;
- comparing two outputs;
- checking whether a claim is supported by evidence.

Use deterministic checks for:

- schema validity;
- required fields;
- forbidden tools;
- budget limits;
- citation presence;
- route labels;
- permission gates.

Combine both. A model judge should not be the only control before a high-risk action.

Evaluation-Driven Development Loop

For every meaningful agent change:

1. Add or update eval cases before changing prompts or tools.
2. Run the current system to establish a baseline.
3. Make the smallest change that targets the failure.
4. Run the eval suite again.
5. Inspect regressions, not only the average score.

6. Promote the change only if it improves the targeted behavior without harming critical cases.

This loop works for prompts, tools, policies, memory, routing, models, and orchestration.

Production Monitoring

Runtime monitoring should feed the eval suite.

Capture:

- failed runs;
- human overrides;
- user corrections;
- low-confidence routes;
- breaker events;
- tool failures;
- high-cost runs;
- slow runs;
- policy denials;
- unusual memory writes.

Every serious incident should become at least one eval case.

Related Chapters

- Observability and Evals
- Circuit Breakers, Fallbacks, and Replay
- Policy Enforcement
- Evaluator-Optimizer
- Agent Development Lifecycle

Agent Security and Sandboxing

Agent security is different from chatbot safety because agents can act. They can read private data, call APIs, write files, run code, trigger workflows, and communicate with other systems.

Use this chapter when an agent has tools, memory, external data, or side effects.

Security Model

Secure agents by separating four concerns:

- what the user asks for;
- what the model proposes;
- what policy allows;
- what the tool actually executes.

The model should not be the policy engine. It can classify intent or explain a decision, but deterministic software should enforce permissions.

The High-Risk Combination

The most dangerous agent shape combines:

- access to private or trusted data;
- exposure to untrusted content;
- ability to perform external actions.

When these three meet, a malicious document, webpage, email, or tool result can try to steer the agent into leaking data or taking an unsafe action.

Mitigate this with least privilege, content isolation, approval gates, egress controls, and explicit policy checks.

Sandboxing

Sandbox any agent that can execute code, operate a browser, manipulate files, or call write APIs.

Sandbox controls:

- isolated process, container, VM, or browser profile;
- read-only filesystem by default;
- scoped workspace directory;
- no ambient credentials;

- explicit secret injection only for approved tools;
- outbound network restrictions;
- time and CPU limits;
- file size limits;
- audit logs for every side effect;
- cleanup after run completion.

Coding and computer-use agents need stronger sandboxes than read-only research agents.

Access Control

Grant access by role, task, and route.

Examples:

- a support answer agent can read public docs but cannot issue refunds;
- a billing workflow can read invoice state but needs approval to apply credit;
- a coding agent can edit files in a branch but cannot deploy production;
- a research agent can browse the web but cannot access customer data.

Avoid global tool lists. Each route or agent should receive only the tools needed for the task.

Guardrails

Guardrails should run before and after model calls and before tools.

Guardrail Point	Checks
Input	prompt injection, sensitive data, unsupported request, user authorization.
Retrieval	untrusted source, stale source, tenant boundary, citation quality.
Tool intent	permission, side effect level, policy, approval requirement.
Tool input	schema, allowed fields, data minimization, idempotency key.
Output	data leakage, unsupported claims, unsafe instructions, missing caveats.
Memory write	privacy, source, expiry, correction path.

No single guardrail is enough. Use layers.

Secrets

Agents should not see raw secrets unless the tool contract requires them.

Prefer:

- server-side tool execution;

- scoped tokens;
- short-lived credentials;
- per-tool secret access;
- no secrets in prompts;
- no secrets in memory;
- redacted traces.

If a model can read a secret, assume it can accidentally expose it.

Approval Gates

Require approval for:

- money movement;
- account changes;
- customer communication at scale;
- production infrastructure changes;
- legal, medical, or compliance decisions;
- deletion or irreversible writes;
- broad data export;
- privilege escalation.

Approval records should include the proposed action, evidence, policy result, approver, timestamp, and final tool call.

Incident Response

Plan for agent incidents before launch.

The team should know how to:

- disable a tool;
- disable a route;
- roll back a prompt or policy;
- revoke credentials;
- quarantine memory;
- inspect traces;
- notify affected users;
- create evals from the incident.

If the response requires code archaeology during an incident, the system is not operationally ready.

Related Chapters

- Policy Enforcement
- Human Approval Gates
- Secure Agent Communication
- MCP-first Tool Use
- Circuit Breakers, Fallbacks, and Replay
- Coding Agents

Agent UX and Human Trust

Agent UX is the interface between human judgment and machine autonomy. A good agent interface shows progress, asks for help at the right time, and makes its actions inspectable.

Use this chapter when humans assign goals, review outputs, approve actions, or recover failed runs.

UX Goals

An agent experience should help users answer:

- What is the agent trying to do?
- What has it already done?
- What evidence is it using?
- What will it do next?
- What needs my approval?
- What failed?
- How do I correct it?
- How do I stop it?

Trust comes from control and visibility, not from confident language.

Interaction Modes

Mode	Use When	UX Requirement
Interactive	The user is present and can clarify or approve.	Streaming progress, questions, cancel, approval UI.
Background	Work takes minutes or hours.	Status page, notifications, resume, audit trail.
Human review	The agent prepares work for approval.	Diff, evidence, rationale, accept/edit/reject.
Autonomous	The agent runs within bounded authority.	Policy scope, traceability, alerts, rollback.
Multi-agent	Several agents collaborate.	Role visibility, handoff status, final owner.

The UX should match the risk. A read-only research agent can be lightweight. A production-deploying coding agent needs stronger review and rollback.

Progress Design

Show progress at meaningful boundaries:

- understanding the goal;
- selecting a route;
- retrieving evidence;
- calling tools;
- waiting for external systems;
- evaluating output;
- requesting approval;
- completing or stopping.

Do not stream hidden reasoning as the primary progress signal. Show actions, evidence, and state transitions.

Explainability

Users need actionable explanations.

Good explanations include:

- source evidence;
- tool results;
- policy checks;
- uncertainty;
- alternative paths considered when relevant;
- reason for escalation or refusal.

Avoid explanations that expose private prompts, irrelevant internal chains of thought, or vague claims such as “the model decided.”

Corrections

Design correction paths before users need them.

Correction types:

- edit final answer;
- correct extracted fields;
- change route;
- add missing context;
- reject memory write;
- retry with a different tool;
- escalate to human;
- cancel and roll back.

Corrections should update evals and, where appropriate, memory. They should not disappear into chat history.

Multi-Agent UX

Multi-agent systems need role clarity.

Show:

- which agent owns the task;
- which agents contributed;
- what each role produced;
- where handoffs occurred;
- which agent or workflow made the final decision;
- where a human entered the loop.

If users cannot tell who is responsible, the system will feel unreliable even when the output is correct.

Approval UX

Approval requests should be specific.

An approval should show:

- proposed action;
- target system;
- affected data or user;
- evidence;
- risk level;
- policy result;
- reversible or irreversible status;
- expected result;
- approval and rejection options.

Never ask for broad approval such as “continue with all actions” when the agent can perform high-risk side effects.

Failure UX

A failed run should be useful.

Return:

- what completed;
- what failed;
- why it stopped;
- whether anything changed externally;
- what the user can do next;
- link or ID for trace review.

Failure UX is part of reliability. A system that fails clearly can still be trusted.

Related Chapters

- Human Approval Gates
- Goals and State
- Routing and Handoffs
- Observability and Evals
- Agent Development Lifecycle

Hands-On Labs

The labs turn the reference chapters into a build path. Each lab uses code that already lives in this repository, so you can read the pattern, run the example, change one thing, and connect the result back to production design.

Run these commands from the repository root before starting:

```
npm install
npm test
npm run typecheck
```

Some examples can run with deterministic fallbacks. Examples that call live models require a `.env` file with `MISTRAL_API_KEY`.

Lab Sequence

1. Build a Tool-Using Agent
2. Build an Agent Loop with Planning
3. Build Agentic RAG
4. Build A2A Agent Communication
5. Build a Multi-Agent Supervisor
6. Add Observability and Evals

How To Use These Labs

Read the objective first, then run the command exactly as shown. After that, inspect the named source files and make the small change in the lab. The goal is not to memorize a framework API. The goal is to see where the pattern becomes code: input contracts, state, tool boundaries, stop conditions, evaluation, and failure handling.

Each lab ends with a production extension. Treat that section as the bridge between a working demo and an architecture decision.

Recommended Order

Do the labs in order if you are new to agent systems. The sequence starts with one agent and one tool, then adds planning, retrieval, remote agent communication, multi-agent coordination, and production-quality evaluation.

If you already know the basics, start with the lab closest to your current system and use the related chapters as reference material.

Lab 01 - Build a Tool-Using Agent

Objective

Build the smallest useful agent boundary: a user message enters, the agent decides whether a tool is needed, the tool runs behind code, and the result returns through a controlled interface.

What You Will Use

- Pattern chapter: Tool Use
- Source folder: `tool-using-agent-pattern/`
- Download: tool-use.zip
- Main file: `tool-using-agent-pattern/autogen_typescript_example/tool_using_agent.ts`

Setup

From the repository root:

```
npm install
```

The calculator path works without a model key. The fallback chat path calls Mistral and requires `MISTRAL_API_KEY`.

Run It

```
NON_INTERACTIVE_INPUT="calculate 2+2" npm run tool-using-agent
```

Expected result:

```
Agent: 4
```

Inspect The Code

Open `tool-using-agent-pattern/autogen_typescript_example/tool_using_agent.ts` and find:

- `calculatorTool(input: string)`: the real capability.
- `toolUsingAgent(userInput: string)`: the routing boundary.
- The `calculate` prefix check: the smallest possible tool-selection policy.

The important design point is that the model does not execute the calculation. Code owns the tool.

Change One Thing

Change the input:

```
NON_INTERACTIVE_INPUT="calculate (12+8)/5" npm run tool-using-agent
```

Then try invalid input:

```
NON_INTERACTIVE_INPUT="calculate not-a-number" npm run tool-using-agent
```

Expected Result

The valid expression should return a number. The invalid expression should return an error string instead of crashing the process.

Production Extension

Replace prefix routing with a typed tool request:

- tool name
- JSON schema for arguments
- authorization check
- timeout
- structured success or error result
- trace ID for the run

Do not give the model broad access to arbitrary functions. Expose narrow tools with explicit permissions.

Related Chapters

- MCP-first Tool Use
- Human Approval Gates
- Policy Enforcement

Lab 02 - Build an Agent Loop with Planning

Objective

Separate planning from execution. The planner decides the steps; the executor runs bounded operations and records results. This gives you the control structure behind many agent loops without making every step autonomous.

What You Will Use

- Pattern chapters: Agent Loop, Planning and Execution
- Source folder: `planning-pattern/`
- Download: planning-and-execution.zip
- Main files:
  - `planning-pattern/typescript/src/planner.ts`
  - `planning-pattern/typescript/src/executor.ts`
  - `planning-pattern/typescript/src/run.ts`

Setup

From the repository root:

```
npm install
```

Run It

Run the deterministic test:

```
npm run plan:test
```

Run the CLI path:

```
npm run plan:run -- "Compute average of [1,2,3,4]"
```

Run the Python mirror:

```
npm run plan:py
```

Inspect The Code

Open `planning-pattern/typescript/src/planner.ts` and inspect how a task becomes a list of steps. Then open `planning-pattern/typescript/src/executor.ts` and inspect how execution turns those steps into named results.

Look for these boundaries:

- the plan object
- step IDs
- deterministic executor functions
- result map
- failure surface

Change One Thing

Change the task text:

```
npm run plan:run -- "Compute average of [10,20,30]"
```

Then inspect whether the deterministic fallback still produces the expected plan shape.

Expected Result

The test should print:

```
Planning test OK
```

The CLI should print a plan and computed result. If you extend the planner, keep the executor deterministic and testable.

Production Extension

Add loop controls before using this pattern in production:

- maximum steps
- maximum retries
- stop reason
- checkpoint after each step
- structured error result
- human review for high-risk steps

Planning is useful only when execution is bounded and inspectable.

Related Chapters

- Goals and State
- Self-Healing Workflows
- Durable Workflows

Lab 03 - Build Agentic RAG

Objective

Build the retrieval boundary behind an Agentic RAG system: retrieve scoped evidence, inject it into context, answer from that evidence, and refuse or escalate when the evidence is not enough.

What You Will Use

- Pattern chapters: Semantic Recall and RAG, Agentic RAG Systems
- Source folder: `context-engineering-pattern/`
- Download: semantic-recall-rag.zip
- Main file: `context-engineering-pattern/langgraph_python_example/rag_example.py`

Setup

The RAG example is Python-based and calls Mistral for answer generation. From the repository root:

```
python3 -m venv .venv-rag
source .venv-rag/bin/activate
pip install -r context-engineering-pattern/langgraph_python_example/requirements.txt
export MISTRAL_API_KEY="<your key>"
```

Run It

```
python3 context-engineering-pattern/langgraph_python_example/rag_example.py
```

Inspect The Code

Open `context-engineering-pattern/langgraph_python_example/rag_example.py` and find:

- `docs`: the tiny demo corpus.
- `HuggingFaceEmbeddings`: the embedding model.
- `FAISS.from_texts`: the vector index.
- `retrieve(query)`: the retrieval boundary.
- `chat_mistral(messages)`: the generation boundary.

The key design move is to keep retrieved evidence separate from the instruction. The model should answer from the context, not from vague memory.

Change One Thing

Add a new document to the `docs` list:

```
{"content": "Agentic RAG uses retrieval, planning, tool use, and verification around a knowledge base."}
```

Then change the query:

```
query = "What makes RAG agentic?"
```

Expected Result

The answer should reflect the new document. If the answer does not cite or use retrieved evidence, the retrieval boundary is not doing enough work.

Production Extension

Add production controls:

- document IDs
- source URLs
- freshness timestamps
- access-control filters
- citation requirements
- prompt-injection checks on retrieved text
- refusal when evidence is missing

Agentic RAG is not only vector search. It is a controlled loop around retrieval, evidence, tool use, and verification.

Related Chapters

- Context Engineering
- Knowledge-Bound Agents
- Working Memory

Lab 04 - Build A2A Agent Communication

Objective

Build a typed communication boundary between two agents. One agent requests work, another agent accepts, refuses, errors, or receives cancellation through explicit messages.

What You Will Use

- Pattern chapter: A2A Agent Interoperability
- Source folder: `agent-to-agent-communication-pattern/`
- Download: a2a-agent-interoperability.zip
- Main files:
  - `agent-to-agent-communication-pattern/src/agent_a.ts`
  - `agent-to-agent-communication-pattern/src/agent_b.ts`
  - `agent-to-agent-communication-pattern/src/bus_memory.ts`
  - `agent-to-agent-communication-pattern/protocol/a2a.schema.json`

Setup

From the repository root:

```
npm install
```

Run It

Run the protocol test:

```
npm run a2a:test
```

Run the demo:

```
npm run a2a:run
```

Inspect The Code

Open `agent-to-agent-communication-pattern/src/agent_a.ts` and `agent-to-agent-communication-pattern/src/agent_b.ts`.

Look for:

- handshake
- task request
- task response
- refusal
- invalid input error
- cancellation
- schema validation with Ajv

Change One Thing

In the test, add a second valid task request with a different ID:

```
a.requestTask('t5', 'sum', { a: 10, b: 15 });
```

Run:

```
npm run a2a:test
```

Expected Result

The receiver should process the new task without confusing it with the existing task IDs. If correlation IDs are missing, progress and results become hard to match.

Production Extension

Before using A2A across services, add:

- authentication
- authorization
- idempotency keys
- task leases
- retry policy
- durable task state
- audit logs
- transport-level encryption

A2A is a protocol boundary, not just one agent calling another function.

Related Chapters

- Secure Agent Communication
- MCP-first Tool Use
- Open Personal Agent Architectures

Lab 05 - Build a Multi-Agent Supervisor

Objective

Build the supervision shape behind multi-agent systems: one coordinator owns the goal, delegates bounded work, gathers worker outputs, and produces the final answer.

What You Will Use

- Pattern chapters: Supervisor / Worker, Task Delegation
- Source folder: `hierarchical-agent-pattern/`
- Download: supervisor-worker.zip
- Main file: `hierarchical-agent-pattern/autogen_typescript_example/hierarchical_agent.ts`

Setup

This example calls Mistral. From the repository root:

```
npm install
cp .env.example .env
```

Set `MISTRAL_API_KEY` in `.env`.

Run It

```
npm run hierarchical-agent
```

When prompted, enter a goal such as:

```
Draft a short plan for evaluating an agentic RAG prototype.
```

Inspect The Code

Open `hierarchical-agent-pattern/autogen_typescript_example/hierarchical_agent.ts` and find:

- the manager prompt
- worker prompts

- subtask extraction
- aggregation prompt
- final answer path

The supervisor owns decomposition and final acceptance. Workers should not silently redefine the goal.

Change One Thing

Change the manager instruction so it asks for three subtasks instead of two. Then inspect the parsing logic:

```
const subTasks = managerPlan.match(/Sub-task [12]: (.*)/g) || [];
```

Update the regex so the code can collect the third subtask.

Expected Result

The manager should produce a plan, workers should produce bounded results, and the final aggregation should combine the worker outputs. If the parsing is brittle, the supervisor loses work.

Production Extension

Replace natural-language subtask parsing with structured output:

- `subtasks: Array<{ id, role, objective, constraints, expected_output }>`
- worker-specific permissions
- per-worker timeout
- merge policy
- judge or evaluator
- human escalation for disagreement

Multi-agent systems need strong contracts. More agents without a merge policy usually means more failure modes.

Related Chapters

- Parallel Agents
- Debate and Consensus
- CrewAI Flows and Crews

Lab 06 - Add Observability and Evals

Objective

Turn the examples into something you can evaluate. A production agent needs trace data, regression tasks, expected outcomes, and failure review before it needs more autonomy.

What You Will Use

- Pattern chapters: Observability and Evals, Evaluator-Optimizer
- Source folder: `observability-and-evals-pattern/`
- Download: observability-and-evals.zip
- Existing test commands:
  - `npm run plan:test`
  - `npm run a2a:test`
  - `npm run mcp:test`
  - `npm test`

Setup

From the repository root:

```
npm install
```

Run It

Run the deterministic checks:

```
npm run plan:test
npm run a2a:test
npm run mcp:test
```

Then run the full suite:

```
npm test
```

Inspect The Code

Use the test files as the first eval dataset:

- `planning-pattern/typescript/test/planning.spec.ts`
- `agent-to-agent-communication-pattern/test/a2a.spec.ts`
- `modern-tool-use-pattern/typescript/test/modern-tools.spec.ts`

Each test checks a contract:

- planning produces executable steps
- A2A handles success, refusal, error, and cancel
- MCP tool use discovers tools and returns useful data

Change One Thing

Add one negative case to the A2A test by sending malformed input with a new task ID:

```
a.requestTask('t6', 'sum', { a: null, b: 10 } as any);
```

Run:

```
npm run a2a:test
```

Expected Result

The system should report an error outcome, not a successful result. In an eval dataset, negative cases are as important as happy paths.

Production Extension

Create a trace and eval record for every run:

```
{
  "run_id": "run_001",
  "pattern": "a2a-agent-interoperability",
  "input": "sum task",
  "expected": "success with sum",
  "actual": "success with sum",
  "score": 1,
  "stop_reason": "completed",
  "latency_ms": 25
}
```

Then add release gates:

- no schema failures
- no missing trace IDs
- no unexpected tool calls
- no regression in golden tasks
- no unresolved safety or policy failures

Observability records what happened. Evals decide whether it is good enough to ship.

Related Chapters

- Agent Development Lifecycle
- Evaluation-Driven Agent Development
- Policy Enforcement

Planning and Execution

Planning separates deciding what to do from doing it. The planner creates steps; the executor runs them, reports progress, and handles errors.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Planning separates deciding what to do from doing it. The planner creates steps; the executor runs them, reports progress, and handles errors.

Use When

- The task has meaningful sequencing, dependencies, or recoverable failure points.
- You want to inspect or revise the plan before execution.
- Execution can be deterministic even if planning uses a model.

Avoid When

- The plan would always be a single step.
- The executor cannot report structured progress or failure.
- The model is allowed to execute unvalidated plans directly.

System Shape

- **Pattern boundary:** a controller repeatedly chooses the next step, executes it, observes the result, and decides whether to continue.
- **State owner:** the loop controller owns progress, budgets, stop conditions, and recovery state.
- **Primary artifact:** `planning-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Planning separates deciding what to do from doing it. The planner creates steps; the executor runs them, reports progress, and handles errors.
- **Runnable path:** start with `npm run plan:test` before adapting the pattern to a larger system.

Core Protocol

1. Initialize goal state, constraints, budgets, and stop conditions.
2. Choose the next action from the current state instead of assuming the whole path upfront.
3. Execute the action through a validated tool, worker, or local function.
4. Observe the result and update state with evidence, errors, and remaining work.
5. Stop, retry, re-plan, or escalate according to explicit policy.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Test success cases, partial failure, repeated failure, budget exhaustion, and bad intermediate observations.
- Assert that the loop stops for the right reason and does not hide failed steps.
- Measure completion rate, number of iterations, recovery quality, cost, and latency.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Set hard iteration, cost, and time limits.
- Persist state after meaningful steps if the run can be interrupted.
- Make retries idempotent or add compensation.
- Expose trace events for each decision, action, observation, and stop reason.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Run the Example

```
npm run plan:test
npm run plan:run -- "Compute average of [1,2,3,4]"
npm run plan:py
```

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

These excerpts show the implementation shape. The complete code is available in the download bundle and repository source.

planning-pattern/typescript/src/planner.ts

Open full source

```
import axios from 'axios';

const MISTRAL_API = 'https://api.mistral.ai/v1/chat/completions';

export interface PlanStep { id: string; description: string }
export interface Plan { steps: PlanStep[]; rationale: string }

export async function planTask(goal: string, apiKey?: string): Promise<Plan> {
  if (!apiKey) {
    // deterministic fallback plan (no network) for tests
    return { steps: [
      { id: 's1', description: 'Load numbers [1,2,3,4]' },
      { id: 's2', description: 'Compute average' }
    ], rationale: 'synthetic' };
  }
  const resp = await axios.post(MISTRAL_API, {
    model: 'mistral-small-latest',
    messages: [
      { role: 'system', content: 'Return JSON {steps: [{id, description}], rationale}
for the goal.' },
```

```
      { role: 'user', content: goal }
    ],
    temperature: 0
  }, { headers: { Authorization: `Bearer ${apiKey}` } });
  const content = resp.data?.choices?.[0]?.message?.content || '';
  try { return JSON.parse(content); } catch { return { steps: [], rationale: content
}; }
}
```

planning-pattern/typescript/src/executor.ts

Open full source

```
export async function executePlan(steps: { id: string; description: string }[],
onProgress?: (pct: number, stage: string) => void) {
  const results: Record<string, any> = {};
  for (let i = 0; i < steps.length; i++) {
    const s = steps[i];
    onProgress?.(Math.round((i / steps.length) * 100), s.id);
    // trivial synthetic execution
    if (s.description.includes('Load numbers')) results[s.id] = [1,2,3,4];
    else if (s.description.includes('Compute average')) {
      const arr = results['s1'] || [];
      results[s.id] = arr.reduce((a:number,b:number)=>a+b,0)/arr.length;
    } else results[s.id] = null;
  }
  onProgress?.(100, 'done');
  return results;
}
```

planning-pattern/typescript/src/run.ts

Open full source

```
import { planTask } from './planner.js';
import { executePlan } from './executor.js';

async function main() {
  const goal = process.argv.slice(2).join(' ') || 'Compute average of [1,2,3,4]';
  const plan = await planTask(goal, process.env.MISTRAL_API_KEY);
  console.log('Plan:', plan);
```

```
  const results = await executePlan(plan.steps, (pct, stage) =>
console.log('Progress', pct, stage));
  console.log('Results:', results);
}

main();
```

Download

- Download source bundle
- Open source folder

The download bundle contains the current `planning-pattern/` folder from this repository.

Related Patterns

- ReAct
- Reflection
- Evaluator-Optimizer
- Choosing the Right Pattern
- Resource-Aware Agent Design

ReAct

ReAct alternates reasoning and acting. The agent reasons about current state, takes an action, observes the result, and repeats.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

ReAct alternates reasoning and acting. The agent reasons about current state, takes an action, observes the result, and repeats.

Use When

- The task requires tool use and the agent cannot know all required information upfront.
- Observations should change the next step.
- A bounded loop can stop on success, failure, or budget.

Avoid When

- The task is a deterministic workflow with known steps.
- You cannot validate actions before they run.
- The reasoning trace would expose sensitive information to users.

System Shape

- **Pattern boundary:** a controller repeatedly chooses the next step, executes it, observes the result, and decides whether to continue.
- **State owner:** the loop controller owns progress, budgets, stop conditions, and recovery state.
- **Primary artifact:** `react-pattern-reason-act/` contains the runnable reference implementation and examples.
- **Operational promise:** ReAct alternates reasoning and acting. The agent reasons about current state, takes an action, observes the result, and repeats.
- **Runnable path:** start with `npm run react-agent` before adapting the pattern to a larger system.

Core Protocol

1. Initialize goal state, constraints, budgets, and stop conditions.
2. Choose the next action from the current state instead of assuming the whole path upfront.
3. Execute the action through a validated tool, worker, or local function.
4. Observe the result and update state with evidence, errors, and remaining work.
5. Stop, retry, re-plan, or escalate according to explicit policy.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Test success cases, partial failure, repeated failure, budget exhaustion, and bad intermediate observations.
- Assert that the loop stops for the right reason and does not hide failed steps.
- Measure completion rate, number of iterations, recovery quality, cost, and latency.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Set hard iteration, cost, and time limits.
- Persist state after meaningful steps if the run can be interrupted.
- Make retries idempotent or add compensation.
- Expose trace events for each decision, action, observation, and stop reason.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Run the Example

```
npm run react-agent
```

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

These excerpts show the implementation shape. The complete code is available in the download bundle and repository source.

react-pattern-reason-act/autogen_typescript_example/react_agent.ts

Open full source

```
// ReAct Pattern (Reason + Act) - Autogen TypeScript Example
// To run: npm install && npm run react-agent

import axios from 'axios';
import * as readline from 'readline';
import { evaluate } from 'mathjs';
import * as dotenv from 'dotenv';
dotenv.config();

const MISTRAL_API_URL = 'https://api.mistral.ai/v1/chat/completions';
const MISTRAL_API_KEY = process.env.MISTRAL_API_KEY;

function calculatorTool(input: string): string {
  try {
    const val = evaluate(input);
    return val.toString();
  } catch (e) {
    return `Error: ${e}`;
  }
}

async function reactAgent(userInput: string): Promise<string> {
```

```
  let context = userInput;
  let done = false;
  let result = '';
  while (!done) {
    // Step 1: Reason
    const reasoningPrompt = `You are an agent. Think step by step about what to do next. If you need to use a tool, say TOOL: <expression>. If you are done, say FINAL: <answer>.\nContext: ${context}`;
    const response = await axios.post(
      MISTRAL_API_URL,
      {
        model: 'mistral-tiny',
        messages: [{ role: 'user', content: reasoningPrompt }],
      },
      {
        headers: {
          'Authorization': `Bearer ${MISTRAL_API_KEY}`,
          'Content-Type': 'application/json',
        },
      }
    );
    const agentOutput = response.data.choices[0].message.content.trim();
    console.log('Agent:', agentOutput);
    if (agentOutput.startsWith('TOOL:')) {
      const expr = agentOutput.replace('TOOL:', '').trim();
      const toolResult = calculatorTool(expr);
      context += `\nTool result: ${toolResult}`;
    } else if (agentOutput.startsWith('FINAL:')) {
      result = agentOutput.replace('FINAL:', '').trim();
      done = true;
    } else {
      // If the agent doesn't follow the protocol, end loop
      result = agentOutput;
      done = true;
    }
  }
  return result;
}

async function main() {
  const idx = process.argv.indexOf('--input');
```

```
  const cliInput = idx !== -1 ? process.argv[idx + 1] : undefined;
  const nonInteractive = cliInput || process.env.NON_INTERACTIVE_INPUT;
  if (nonInteractive) {
    try {
      const agentResponse = await reactAgent(String(nonInteractive));
      console.log('Final Answer:', agentResponse);
    } catch (err) {
      console.error('Error:', err);
      process.exitCode = 1;
    }
    return;
  }
  const rl = readline.createInterface({ input: process.stdin, output: process.stdout
});
  rl.question('Task: ', async (userInput: string) => {
    try {
      const agentResponse = await reactAgent(userInput);
      console.log('Final Answer:', agentResponse);
    } catch (err) {
      console.error('Error:', err);
    }
    rl.close();
  });
}

main();
```

react-pattern-reason-act/langgraph_python_example/react_agent.py

Open full source

```
# ReAct Pattern (Reason + Act) - LangGraph Python Example

This example demonstrates the ReAct Pattern using LangGraph and Python. The agent
alternates between reasoning and acting (using a calculator tool), and iteratively
solves the task. The LLM is Mistral.

## Requirements

- Python 3.8+
- `langgraph` library
```

```
- `python-dotenv` (for .env support)
- Mistral LLM API access

## Install dependencies

```bash
pip install langgraph python-dotenv requests
```

Example Code

```python
import os
from langgraph import Agent, Environment, LLM, Tool
from dotenv import load_dotenv

load_dotenv()

MISTRAL_API_KEY = os.getenv("MISTRAL_API_KEY")
MISTRAL_API_URL = "https://api.mistral.ai/v1/chat/completions"

import sympy as _sp

def safe_calc(expr: str) -> str:
    try:
        return str(_sp.sympify(expr, evaluate=True))
    except Exception as e:
        return f"Error: {e}"

class CalculatorTool(Tool):
    def call(self, input_str):
        return safe_calc(input_str)

class SimpleEnvironment(Environment):
    def get_observation(self):
        return input("Task: ")
    def send_action(self, action):
        print(f"Agent: {action}")

class ReActAgent(Agent):
    def __init__(self, llm, tools):
```
```

```
        self.llm = llm
        self.tools = {tool.name: tool for tool in tools}
    def act(self, observation):
        context = observation
        while True:
            prompt = (
                "You are an agent. Think step by step about what to do next. "
                "If you need to use a tool, say TOOL: <expression>. "
                "If you are done, say FINAL: <answer>.\nContext: " + context
            )
            response = self.llm.complete(prompt)
            print(f"Agent: {response}")
            if response.startswith("TOOL:"):
                expr = response.replace("TOOL:", "").strip()
                tool_result = self.tools["calculator"].call(expr)
                context += f"\nTool result: {tool_result}"
            elif response.startswith("FINAL:"):
                return response.replace("FINAL:", "").strip()
            else:
                return response

llm = LLM(
    provider="mistral",
    api_key=MISTRAL_API_KEY,
    api_url=MISTRAL_API_URL,
)

calc_tool = CalculatorTool(name="calculator")
env = SimpleEnvironment()
agent = ReActAgent(llm, [calc_tool])

observation = env.get_observation()
action = agent.act(observation)
env.send_action(action)
```

- Try a multi-step math or logic task to see reasoning and tool use.
- Make sure your `.env` file contains your Mistral API key.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `react-pattern-reason-act/` folder from this repository.

Related Patterns

- Planning and Execution
- Reflection
- Evaluator-Optimizer
- Choosing the Right Pattern
- Resource-Aware Agent Design

Reflection

Reflection asks an agent or evaluator to inspect prior output and identify concrete improvements.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Reflection asks an agent or evaluator to inspect prior output and identify concrete improvements.

Use When

- The system has explicit quality criteria.
- A critique can change decisions, tests, state, or final artifacts.
- You need a lightweight improvement pass without a full evaluator-optimizer loop.

Avoid When

- The critique only produces longer output.
- There is no acceptance criterion for the revised result.
- The model is asked to approve its own unsafe actions.

System Shape

- **Pattern boundary:** a controller repeatedly chooses the next step, executes it, observes the result, and decides whether to continue.
- **State owner:** the loop controller owns progress, budgets, stop conditions, and recovery state.
- **Primary artifact:** `reflection-and-self-improvement-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Reflection asks an agent or evaluator to inspect prior output and identify concrete improvements.
- **Runnable path:** start with `npm run reflection-self-improvement-agent` before adapting the pattern to a larger system.

Core Protocol

1. Initialize goal state, constraints, budgets, and stop conditions.
2. Choose the next action from the current state instead of assuming the whole path upfront.
3. Execute the action through a validated tool, worker, or local function.
4. Observe the result and update state with evidence, errors, and remaining work.
5. Stop, retry, re-plan, or escalate according to explicit policy.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Test success cases, partial failure, repeated failure, budget exhaustion, and bad intermediate observations.
- Assert that the loop stops for the right reason and does not hide failed steps.
- Measure completion rate, number of iterations, recovery quality, cost, and latency.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Set hard iteration, cost, and time limits.
- Persist state after meaningful steps if the run can be interrupted.
- Make retries idempotent or add compensation.
- Expose trace events for each decision, action, observation, and stop reason.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Run the Example

```
npm run reflection-self-improvement-agent
```

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

These excerpts show the implementation shape. The complete code is available in the download bundle and repository source.

`reflection-and-self-improvement-pattern/autogen_typescript_example/reflection_agent.ts`

Open full source

```
import dotenv from 'dotenv';
dotenv.config();
import axios from 'axios';
import readline from 'readline';

const MISTRAL_API_KEY = process.env.MISTRAL_API_KEY;
const MISTRAL_API_URL = 'https://api.mistral.ai/v1/chat/completions';

if (!MISTRAL_API_KEY) {
  console.error('Please set MISTRAL_API_KEY in your .env file');
  process.exit(1);
}

async function askMistral(messages: any[]) {
  const response = await axios.post(
    MISTRAL_API_URL,
    {
      model: 'mistral-tiny',
      messages,
    },
    {
      headers: {
```

```
        'Authorization': `Bearer ${MISTRAL_API_KEY}`,
        'Content-Type': 'application/json',
      },
    }
  );
  return response.data.choices[0].message.content.trim();
}

async function main() {
  const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout,
  });

  rl.question('Ask the agent a question: ', async (userInput) => {
    let messages = [
      { role: 'system', content: 'You are a helpful assistant that reflects on your answers and tries to improve them if possible.' },
      { role: 'user', content: userInput },
    ];

    // First response
    let answer = await askMistral(messages);
    console.log('\nInitial Answer:\n', answer);

    // Reflection step
    messages.push({ role: 'assistant', content: answer });
    messages.push({ role: 'system', content: 'Reflect on your previous answer. Was it correct, clear, and complete? If not, revise and improve it.' });
    let reflection = await askMistral(messages);
    console.log('\nReflected/Improved Answer:\n', reflection);

    rl.close();
  });
}

main();
```

reflection-and-self-improvement-pattern/langgraph_python_example/reflection_agent.py

Open full source

```
import os
import requests

def ask_mistral(messages):
    api_key = os.getenv('MISTRAL_API_KEY')
    if not api_key:
        raise ValueError('Please set MISTRAL_API_KEY in your .env file')
    url = 'https://api.mistral.ai/v1/chat/completions'
    headers = {
        'Authorization': f'Bearer {api_key}',
        'Content-Type': 'application/json',
    }
    data = {
        'model': 'mistral-tiny',
        'messages': messages,
    }
    response = requests.post(url, headers=headers, json=data)
    response.raise_for_status()
    return response.json()['choices'][0]['message']['content'].strip()

def main():
    user_input = input('Ask the agent a question: ')
    messages = [
        {'role': 'system', 'content': 'You are a helpful assistant that reflects on
your answers and tries to improve them if possible.'},
        {'role': 'user', 'content': user_input},
    ]
    # First response
    answer = ask_mistral(messages)
    print('\nInitial Answer:\n', answer)
    # Reflection step
    messages.append({'role': 'assistant', 'content': answer})
    messages.append({'role': 'system', 'content': 'Reflect on your previous answer.
Was it correct, clear, and complete? If not, revise and improve it.'})
    reflection = ask_mistral(messages)
    print('\nReflected/Improved Answer:\n', reflection)

if __name__ == '__main__':
    main()
```

Download

- Download source bundle
- Open source folder

The download bundle contains the current `reflection-and-self-improvement-pattern/` folder from this repository.

Related Patterns

- Planning and Execution
- ReAct
- Evaluator-Optimizer
- Choosing the Right Pattern
- Resource-Aware Agent Design

Evaluator-Optimizer

Evaluator-Optimizer pairs a generator with an evaluator. The generator proposes; the evaluator scores; the optimizer revises or stops.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

The Evaluator-Optimizer Pattern pairs a generator with an evaluator. The generator proposes work; the evaluator scores it against explicit criteria; the optimizer revises until the output passes or the budget ends.

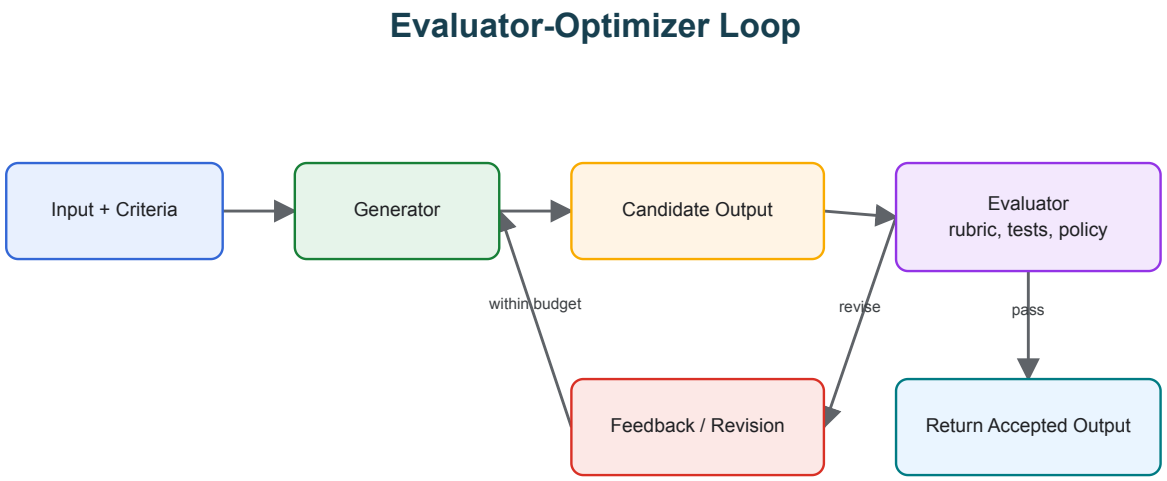
Use When

- Quality can be judged more reliably than it can be produced in one pass.
- You have explicit rubrics, tests, policies, or examples.
- Iterative improvement is worth the extra cost and latency.

Avoid When

- The evaluator is just another vague opinion prompt.
- The task must respond with very low latency.
- You cannot define pass/fail or ranking criteria.

Architecture



System Shape

- **Pattern boundary:** a controller repeatedly chooses the next step, executes it, observes the result, and decides whether to continue.
- **State owner:** the loop controller owns progress, budgets, stop conditions, and recovery state.
- **Primary artifact:** `evaluator-optimizer-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Evaluator-Optimizer pairs a generator with an evaluator. The generator proposes; the evaluator scores; the optimizer revises or stops.

Core Protocol

1. Initialize goal state, constraints, budgets, and stop conditions.
2. Choose the next action from the current state instead of assuming the whole path upfront.
3. Execute the action through a validated tool, worker, or local function.
4. Observe the result and update state with evidence, errors, and remaining work.
5. Stop, retry, re-plan, or escalate according to explicit policy.

Implementation Notes

- Separate generation prompts from evaluation prompts.
- Use deterministic tests when possible, then model-based critique for subjective gaps.
- Persist evaluator feedback so regressions can be analyzed.
- Define max revision count and a fallback behavior before running the loop.

Failure Modes

- Evaluator drift: the evaluator rewards style over correctness.
- Generator overfits to the evaluator and hides flaws.
- Revision loops that make output longer but not better.
- No retained evidence for why a candidate passed.

Evaluation Strategy

- Test success cases, partial failure, repeated failure, budget exhaustion, and bad intermediate observations.
- Assert that the loop stops for the right reason and does not hide failed steps.
- Measure completion rate, number of iterations, recovery quality, cost, and latency.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Set hard iteration, cost, and time limits.
- Persist state after meaningful steps if the run can be interrupted.
- Make retries idempotent or add compensation.
- Expose trace events for each decision, action, observation, and stop reason.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

This pattern currently has no dedicated code excerpt. Use the source and download links below for the full pattern folder.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `evaluator-optimizer-pattern/` folder from this repository.

Related Patterns

- Reflection and Self-Improvement
- Observability and Evals
- Agent Loop

Self-Improvement

Self-improvement uses feedback from prior runs to improve future runs through reviewed changes to prompts, tools, retrieval, policies, tests, or skills.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Self-improvement uses feedback from prior runs to improve future runs through reviewed changes to prompts, tools, retrieval, policies, tests, or skills.

Use When

- You have run logs, eval failures, and review processes.
- Improvements are applied through versioned artifacts.
- Humans or automated gates can approve behavior changes.

Avoid When

- The agent silently rewrites its own instructions in production.
- No eval suite exists to catch regressions.
- The feedback signal is noisy or easy to game.

System Shape

- **Pattern boundary:** a controller repeatedly chooses the next step, executes it, observes the result, and decides whether to continue.
- **State owner:** the loop controller owns progress, budgets, stop conditions, and recovery state.
- **Primary artifact:** `reflection-and-self-improvement-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Self-improvement uses feedback from prior runs to improve future runs through reviewed changes to prompts, tools, retrieval, policies, tests, or skills.

Core Protocol

1. Initialize goal state, constraints, budgets, and stop conditions.
2. Choose the next action from the current state instead of assuming the whole path upfront.
3. Execute the action through a validated tool, worker, or local function.
4. Observe the result and update state with evidence, errors, and remaining work.
5. Stop, retry, re-plan, or escalate according to explicit policy.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Test success cases, partial failure, repeated failure, budget exhaustion, and bad intermediate observations.
- Assert that the loop stops for the right reason and does not hide failed steps.
- Measure completion rate, number of iterations, recovery quality, cost, and latency.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Set hard iteration, cost, and time limits.
- Persist state after meaningful steps if the run can be interrupted.
- Make retries idempotent or add compensation.
- Expose trace events for each decision, action, observation, and stop reason.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

These excerpts show the implementation shape. The complete code is available in the download bundle and repository source.

`reflection-and-self-improvement-pattern/autogen_typescript_example/reflection_agent.ts`

Open full source

```
import dotenv from 'dotenv';
dotenv.config();
import axios from 'axios';
import readline from 'readline';

const MISTRAL_API_KEY = process.env.MISTRAL_API_KEY;
const MISTRAL_API_URL = 'https://api.mistral.ai/v1/chat/completions';

if (!MISTRAL_API_KEY) {
  console.error('Please set MISTRAL_API_KEY in your .env file');
  process.exit(1);
}

async function askMistral(messages: any[]) {
  const response = await axios.post(
    MISTRAL_API_URL,
    {
      model: 'mistral-tiny',
      messages,
    },
    {
      headers: {
        'Authorization': `Bearer ${MISTRAL_API_KEY}`,
        'Content-Type': 'application/json',
      },
    }
  );
  return response.data.choices[0].message.content.trim();
```

```
}

async function main() {
  const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout,
  });

  rl.question('Ask the agent a question: ', async (userInput) => {
    let messages = [
      { role: 'system', content: 'You are a helpful assistant that reflects on your answers and tries to improve them if possible.' },
      { role: 'user', content: userInput },
    ];

    // First response
    let answer = await askMistral(messages);
    console.log('\nInitial Answer:\n', answer);

    // Reflection step
    messages.push({ role: 'assistant', content: answer });
    messages.push({ role: 'system', content: 'Reflect on your previous answer. Was it correct, clear, and complete? If not, revise and improve it.' });
    let reflection = await askMistral(messages);
    console.log('\nReflected/Improved Answer:\n', reflection);

    rl.close();
  });
}

main();
```

reflection-and-self-improvement-pattern/langgraph_python_example/reflection_agent.py

Open full source

```
import os
import requests

def ask_mistral(messages):
    api_key = os.getenv('MISTRAL_API_KEY')
```

```
    if not api_key:
        raise ValueError('Please set MISTRAL_API_KEY in your .env file')
    url = 'https://api.mistral.ai/v1/chat/completions'
    headers = {
        'Authorization': f'Bearer {api_key}',
        'Content-Type': 'application/json',
    }
    data = {
        'model': 'mistral-tiny',
        'messages': messages,
    }
    response = requests.post(url, headers=headers, json=data)
    response.raise_for_status()
    return response.json()['choices'][0]['message']['content'].strip()

def main():
    user_input = input('Ask the agent a question: ')
    messages = [
        {'role': 'system', 'content': 'You are a helpful assistant that reflects on
your answers and tries to improve them if possible.'},
        {'role': 'user', 'content': user_input},
    ]
    # First response
    answer = ask_mistral(messages)
    print('\nInitial Answer:\n', answer)
    # Reflection step
    messages.append({'role': 'assistant', 'content': answer})
    messages.append({'role': 'system', 'content': 'Reflect on your previous answer.
Was it correct, clear, and complete? If not, revise and improve it.'})
    reflection = ask_mistral(messages)
    print('\nReflected/Improved Answer:\n', reflection)

if __name__ == '__main__':
    main()
```

Download

- Download source bundle
- Open source folder

The download bundle contains the current `reflection-and-self-improvement-pattern/` folder from this repository.

Related Patterns

- Planning and Execution
- ReAct
- Reflection
- Choosing the Right Pattern
- Resource-Aware Agent Design

Self-Healing Workflows

Self-healing workflows detect failed steps and recover through retry, fallback, re-planning, or escalation.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Self-healing workflows detect failed steps and recover through retry, fallback, re-planning, or escalation.

Use When

- Failures are expected and can be classified.
- The workflow can retry idempotently or compensate safely.
- Recovery policies are explicit and observable.

Avoid When

- Every failure requires human judgment.
- Retries can duplicate side effects.
- The system would call a model again without new information.

System Shape

- **Pattern boundary:** a controller repeatedly chooses the next step, executes it, observes the result, and decides whether to continue.
- **State owner:** the loop controller owns progress, budgets, stop conditions, and recovery state.
- **Primary artifact:** `self-healing-workflow-agent-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Self-healing workflows detect failed steps and recover through retry, fallback, re-planning, or escalation.

Core Protocol

1. Initialize goal state, constraints, budgets, and stop conditions.
2. Choose the next action from the current state instead of assuming the whole path upfront.
3. Execute the action through a validated tool, worker, or local function.
4. Observe the result and update state with evidence, errors, and remaining work.
5. Stop, retry, re-plan, or escalate according to explicit policy.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Test success cases, partial failure, repeated failure, budget exhaustion, and bad intermediate observations.
- Assert that the loop stops for the right reason and does not hide failed steps.
- Measure completion rate, number of iterations, recovery quality, cost, and latency.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Set hard iteration, cost, and time limits.
- Persist state after meaningful steps if the run can be interrupted.
- Make retries idempotent or add compensation.
- Expose trace events for each decision, action, observation, and stop reason.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

This pattern currently has no dedicated code excerpt. Use the source and download links below for the full pattern folder.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `self-healing-workflow-agent-pattern/` folder from this repository.

Related Patterns

- Planning and Execution
- ReAct
- Reflection
- Choosing the Right Pattern
- Resource-Aware Agent Design

Memory-Augmented Agent

Memory-augmented agents store and retrieve information across turns or sessions.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Memory-augmented agents store and retrieve information across turns or sessions.

Use When

- The agent needs continuity beyond one interaction.
- Stored facts can be scoped, updated, and deleted.
- Retrieval results can be cited or inspected.

Avoid When

- The system would store sensitive data without consent or retention rules.
- Retrieved memories cannot be distinguished from current instructions.
- The memory store is used as an uncurated transcript dump.

System Shape

- **Pattern boundary:** a retrieval or memory boundary decides what information enters context and what new information can be stored.
- **State owner:** the memory or retrieval layer owns long-lived knowledge, while the agent owns task-local working state.
- **Primary artifact:** `memory-augmented-agent-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Memory-augmented agents store and retrieve information across turns or sessions.
- **Runnable path:** start with `npm run memory-augmented-agent` before adapting the pattern to a larger system.

Core Protocol

1. Classify the information need: working state, episodic memory, semantic knowledge, policy, or source evidence.
2. Retrieve only scoped, relevant, and permitted material.
3. Inject retrieved material with source labels, freshness, and trust level.
4. Generate or act while keeping retrieved evidence separate from instructions.
5. Write back memory only after validation, consent, retention, and correction rules pass.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Use questions with known source answers, stale sources, conflicting sources, and missing evidence.
- Measure recall, precision, citation faithfulness, freshness, and refusal when evidence is absent.
- Test deletion, correction, and privacy boundaries separately from answer quality.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Define retention, deletion, correction, and consent rules.
- Separate instructions from retrieved facts and user memories.
- Record source IDs and retrieval scores for audit and debugging.
- Add guards against prompt injection from retrieved documents.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Run the Example

```
npm run memory-augmented-agent
```

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

These excerpts show the implementation shape. The complete code is available in the download bundle and repository source.

memory-augmented-agent-pattern/autogen_typescript_example/memory_agent.ts

Open full source

```
import dotenv from 'dotenv';
dotenv.config();
import axios from 'axios';
import readline from 'readline';
import fs from 'fs';

type MemoryMessage = { role: string; content: string };

const MISTRAL_API_KEY = process.env.MISTRAL_API_KEY;
const MISTRAL_API_URL = 'https://api.mistral.ai/v1/chat/completions';
const MEMORY_FILE = './memory-augmented-agent-
pattern/autogen_typescript_example/memory.json';

if (!MISTRAL_API_KEY) {
  console.error('Please set MISTRAL_API_KEY in your .env file');
  process.exit(1);
}

function loadMemory(): MemoryMessage[] {
  if (fs.existsSync(MEMORY_FILE)) {
  return JSON.parse(fs.readFileSync(MEMORY_FILE, 'utf-8')) as MemoryMessage[];
  }
```

```
  return [];
}

function saveMemory(memory: MemoryMessage[]) {
  fs.writeFileSync(MEMORY_FILE, JSON.stringify(memory, null, 2));
}

async function askMistral(messages: any[]) {
  const response = await axios.post(
    MISTRAL_API_URL,
    {
      model: 'mistral-tiny',
      messages,
    },
    {
      headers: {
        'Authorization': `Bearer ${MISTRAL_API_KEY}`,
        'Content-Type': 'application/json',
      },
    }
  );
  return response.data.choices[0].message.content.trim();
}

async function main() {
  const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout,
  });

  let memory: MemoryMessage[] = loadMemory();

  rl.question('Ask the agent a question: ', async (userInput) => {
    // Retrieve relevant memory (for demo, just concatenate all previous
user/assistant messages)
  let context = memory.map((m: MemoryMessage) => `${m.role}:
${m.content}`).join('\n');
    let messages = [
      { role: 'system', content: 'You are a helpful assistant with memory. Use the
following context from previous interactions if relevant:\n' + context },
      { role: 'user', content: userInput },
```

```
    ];

    let answer = await askMistral(messages);
    console.log('\nAgent Answer (with memory):\n', answer);

    // Update memory
  memory.push({ role: 'user', content: userInput });
  memory.push({ role: 'assistant', content: answer });
    saveMemory(memory);

    rl.close();
  });
}

main();
```

memory-augmented-agent-pattern/langgraph_python_example/memory_agent.py

Open full source

```
import os
import json
import requests

MEMORY_FILE = os.path.join(os.path.dirname(__file__), 'memory.json')

def load_memory():
    if os.path.exists(MEMORY_FILE):
        with open(MEMORY_FILE, 'r') as f:
            return json.load(f)
    return []

def save_memory(memory):
    with open(MEMORY_FILE, 'w') as f:
        json.dump(memory, f, indent=2)

def ask_mistral(messages):
    api_key = os.getenv('MISTRAL_API_KEY')
    if not api_key:
        raise ValueError('Please set MISTRAL_API_KEY in your .env file')
    url = 'https://api.mistral.ai/v1/chat/completions'
```

```
    headers = {
        'Authorization': f'Bearer {api_key}',
        'Content-Type': 'application/json',
    }
    data = {
        'model': 'mistral-tiny',
        'messages': messages,
    }
    response = requests.post(url, headers=headers, json=data)
    response.raise_for_status()
    return response.json()['choices'][0]['message']['content'].strip()

def main():
    memory = load_memory()
    user_input = input('Ask the agent a question: ')
    # Retrieve relevant memory (for demo, just concatenate all previous
user/assistant messages)
    context = '\n'.join([f"{m['role']}: {m['content']}" for m in memory])
    messages = [
        {'role': 'system', 'content': 'You are a helpful assistant with memory. Use
the following context from previous interactions if relevant:\n' + context},
        {'role': 'user', 'content': user_input},
    ]
    answer = ask_mistral(messages)
    print('\nAgent Answer (with memory):\n', answer)
    # Update memory
    memory.append({'role': 'user', 'content': user_input})
    memory.append({'role': 'assistant', 'content': answer})
    save_memory(memory)

if __name__ == '__main__':
    main()
```

Download

- Download source bundle
- Open source folder

The download bundle contains the current `memory-augmented-agent-pattern/` folder from this repository.

Related Patterns

- Long-Term Episodic Memory
- Semantic Recall and RAG
- Working Memory
- Choosing the Right Pattern
- Resource-Aware Agent Design

Long-Term Episodic Memory

Long-term episodic memory stores events: what happened, when, who was involved, and why it mattered.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Long-term episodic memory stores events: what happened, when, who was involved, and why it mattered.

Use When

- The assistant needs continuity across sessions.
- Events can be retrieved by relevance, recency, user, and project scope.
- You can enforce retention, privacy, and correction policies.

Avoid When

- The task only needs semantic facts, not event history.
- The system cannot explain or delete remembered events.

System Shape

- **Pattern boundary:** a retrieval or memory boundary decides what information enters context and what new information can be stored.
- **State owner:** the memory or retrieval layer owns long-lived knowledge, while the agent owns task-local working state.
- **Primary artifact:** `long-term-episodic-memory-agent-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Long-term episodic memory stores events: what happened, when, who was involved, and why it mattered.

Core Protocol

1. Classify the information need: working state, episodic memory, semantic knowledge, policy, or source evidence.
2. Retrieve only scoped, relevant, and permitted material.
3. Inject retrieved material with source labels, freshness, and trust level.
4. Generate or act while keeping retrieved evidence separate from instructions.
5. Write back memory only after validation, consent, retention, and correction rules pass.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Use questions with known source answers, stale sources, conflicting sources, and missing evidence.
- Measure recall, precision, citation faithfulness, freshness, and refusal when evidence is absent.
- Test deletion, correction, and privacy boundaries separately from answer quality.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Define retention, deletion, correction, and consent rules.
- Separate instructions from retrieved facts and user memories.
- Record source IDs and retrieval scores for audit and debugging.
- Add guards against prompt injection from retrieved documents.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

This pattern currently has no dedicated code excerpt. Use the source and download links below for the full pattern folder.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `long-term-episodic-memory-agent-pattern/` folder from this repository.

Related Patterns

- Memory-Augmented Agent
- Semantic Recall and RAG
- Working Memory
- Choosing the Right Pattern
- Resource-Aware Agent Design

Semantic Recall and RAG

Semantic recall retrieves relevant material by meaning rather than exact keywords. RAG injects retrieved material into context before generation.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Semantic recall retrieves relevant material by meaning rather than exact keywords. RAG injects retrieved material into context before generation.

Use When

- The agent must answer from a changing or large knowledge base.
- Relevant documents can be chunked, embedded, filtered, and cited.
- The retrieval layer can enforce source trust and freshness.

Avoid When

- The answer should come from model knowledge only.
- The corpus is too noisy to retrieve safely.
- The system cannot cite or inspect retrieved context.

System Shape

- **Pattern boundary:** a retrieval or memory boundary decides what information enters context and what new information can be stored.
- **State owner:** the memory or retrieval layer owns long-lived knowledge, while the agent owns task-local working state.
- **Primary artifact:** `context-engineering-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Semantic recall retrieves relevant material by meaning rather than exact keywords. RAG injects retrieved material into context before generation.

Core Protocol

1. Classify the information need: working state, episodic memory, semantic knowledge, policy, or source evidence.
2. Retrieve only scoped, relevant, and permitted material.
3. Inject retrieved material with source labels, freshness, and trust level.
4. Generate or act while keeping retrieved evidence separate from instructions.
5. Write back memory only after validation, consent, retention, and correction rules pass.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Use questions with known source answers, stale sources, conflicting sources, and missing evidence.
- Measure recall, precision, citation faithfulness, freshness, and refusal when evidence is absent.
- Test deletion, correction, and privacy boundaries separately from answer quality.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Define retention, deletion, correction, and consent rules.
- Separate instructions from retrieved facts and user memories.
- Record source IDs and retrieval scores for audit and debugging.
- Add guards against prompt injection from retrieved documents.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

These excerpts show the implementation shape. The complete code is available in the download bundle and repository source.

context-engineering-pattern/langgraph_python_example/rag_example.py

Open full source

```
"""
Context Engineering Example: Retrieval-Augmented Generation (RAG) with Mistral
Requirements: pip install langchain-community sentence-transformers faiss-cpu
requests
Note: This is a minimal example using HuggingFace embeddings by default.
"""
import os
import requests
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings
from typing import List

# Optionally load environment variables from a local .env if present
try:
    from dotenv import load_dotenv, find_dotenv  # type: ignore
    load_dotenv(find_dotenv(usecwd=True), override=True)
except Exception:
    pass

# Dummy documents
docs = [
    {"content": "Agentic systems are autonomous AI systems."},
    {"content": "Prompt engineering improves LLM outputs."}
]

# Build vector store (in-memory for demo)
texts = [d["content"] for d in docs]
```

```
embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
try:
    # Try FAISS first (fast vector index)
    vectorstore = FAISS.from_texts(texts, embeddings)
    retriever = vectorstore.as_retriever()

    def retrieve(query: str, k: int = 3):
        return retriever.get_relevant_documents(query)
except Exception:
    # Fallback: simple numpy-based cosine similarity without FAISS
    try:
        import numpy as np  # sentence-transformers depends on numpy, so this should be available
    except Exception as e:  # pragma: no cover
        raise RuntimeError("numpy is required for the FAISS-free fallback but wasn't found") from e

    doc_vecs = np.array(embeddings.embed_documents(texts), dtype=float)
    doc_norms = np.linalg.norm(doc_vecs, axis=1, keepdims=True) + 1e-12
    doc_vecs = doc_vecs / doc_norms

    class _MiniDoc:
        def __init__(self, text: str):
            self.page_content = text

    def retrieve(query: str, k: int = 3):
        q = np.array(embeddings.embed_query(query), dtype=float)
        q = q / (np.linalg.norm(q) + 1e-12)
        sims = (doc_vecs @ q)
        top_idx = np.argsort(-sims)[:k]
        return [_MiniDoc(texts[i]) for i in top_idx]
MISTRAL_API_KEY = os.getenv("MISTRAL_API_KEY")
if not MISTRAL_API_KEY:
    raise RuntimeError("Set MISTRAL_API_KEY in your environment.")

def chat_mistral(messages):
    resp = requests.post(
        "https://api.mistral.ai/v1/chat/completions",
        headers={
            "Authorization": f"Bearer {MISTRAL_API_KEY}",
```

```
            "Content-Type": "application/json",
        },
        json={
            "model": "mistral-large-latest",
            "messages": messages,
            "temperature": 0.2,
        },
        timeout=60,
    )
    resp.raise_for_status()
    data = resp.json()
    return (data.get("choices") or [{}])[0].get("message", {}).get("content", "")

if __name__ == "__main__":
    query = "What are agentic systems?"
    retrieved = retrieve(query)
    context = "\n\n".join(d.page_content for d in retrieved)
    answer = chat_mistral([
        {"role": "system", "content": "Use the provided context to answer the
question succinctly."},
        {"role": "user", "content": f"Context:\n{context}\n\nQuestion: {query}"},
    ])
    print("Answer:", answer)
```

Download

- Download source bundle
- Open source folder

The download bundle contains the current `context-engineering-pattern/` folder from this repository.

Related Patterns

- Memory-Augmented Agent
- Long-Term Episodic Memory
- Working Memory
- Choosing the Right Pattern
- Resource-Aware Agent Design

Working Memory

Working memory is compact, typed task state the agent can update and consult during a run.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

The Goals and State Pattern separates what the agent is trying to achieve from the mutable state it accumulates while working. Goals define success; state records progress, constraints, evidence, and pending work.

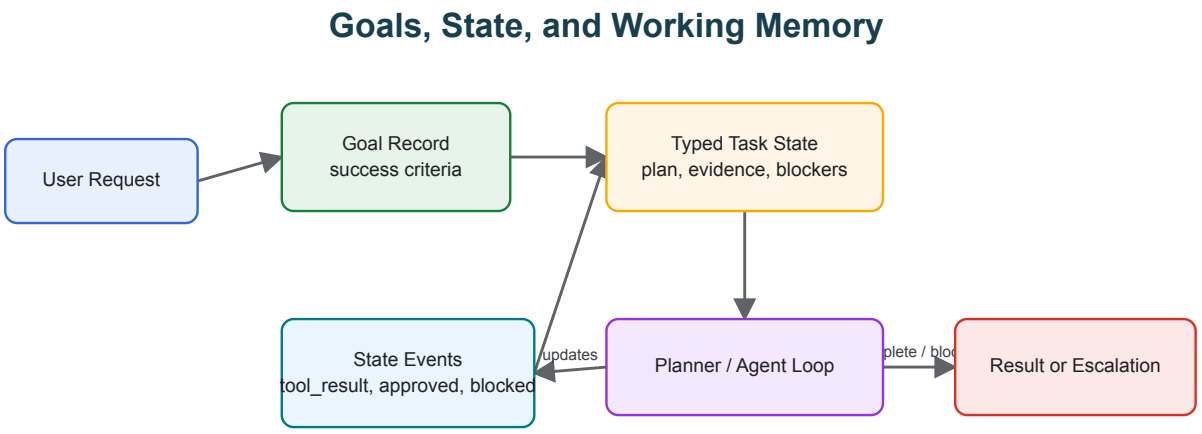
Use When

- A task spans multiple turns, tools, or agents.
- You need resumable execution after failure or interruption.
- The agent must explain progress against an explicit objective.

Avoid When

- The task is stateless and can be answered in one call.
- The goal cannot be expressed as observable success criteria.
- State would contain sensitive data you cannot store safely.

Architecture



System Shape

- **Pattern boundary:** a retrieval or memory boundary decides what information enters context and what new information can be stored.
- **State owner:** the memory or retrieval layer owns long-lived knowledge, while the agent owns task-local working state.
- **Primary artifact:** `goals-and-state-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Working memory is compact, typed task state the agent can update and consult during a run.

Core Protocol

1. Classify the information need: working state, episodic memory, semantic knowledge, policy, or source evidence.
2. Retrieve only scoped, relevant, and permitted material.
3. Inject retrieved material with source labels, freshness, and trust level.
4. Generate or act while keeping retrieved evidence separate from instructions.
5. Write back memory only after validation, consent, retention, and correction rules pass.

Implementation Notes

- Store goals as structured records with `id`, `description`, `success_criteria`, `constraints`, `owner`, and `status`.
- Store state separately from chat history. Chat is evidence; state is the operational model.
- Update state through typed events such as `step_started`, `tool_result`, `blocked`, `approved`, and `completed`.

- Make state transitions auditable and idempotent so a workflow can retry safely.

Failure Modes

- Goals that describe activity rather than success.
- State that becomes a transcript dump instead of a compact task model.
- Agents optimizing for local subgoals that no longer serve the parent goal.
- Lost cancellation or approval state after retries.

Evaluation Strategy

- Use questions with known source answers, stale sources, conflicting sources, and missing evidence.
- Measure recall, precision, citation faithfulness, freshness, and refusal when evidence is absent.
- Test deletion, correction, and privacy boundaries separately from answer quality.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Define retention, deletion, correction, and consent rules.
- Separate instructions from retrieved facts and user memories.
- Record source IDs and retrieval scores for audit and debugging.
- Add guards against prompt injection from retrieved documents.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

This pattern currently has no dedicated code excerpt. Use the source and download links below for the full pattern folder.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `goals-and-state-pattern/` folder from this repository.

Related Patterns

- Agent Loop
- Planning Pattern
- Durable Workflow

Knowledge-Bound Agents

Knowledge-bound agents ground answers and actions in approved sources, policies, and citation rules.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Knowledge-bound agents ground answers and actions in approved sources, policies, and citation rules.

Use When

- The domain requires approved sources, citations, or compliance constraints.
- The agent should refuse or escalate when evidence is missing.
- Freshness and source trust matter.

Avoid When

- The agent is allowed to speculate freely.
- Approved sources cannot be identified or updated.
- Policy checks happen only after irreversible actions.

System Shape

- **Pattern boundary:** a retrieval or memory boundary decides what information enters context and what new information can be stored.
- **State owner:** the memory or retrieval layer owns long-lived knowledge, while the agent owns task-local working state.
- **Primary artifact:** `compliance-policy-enforcer-agent/` contains the runnable reference implementation and examples.
- **Operational promise:** Knowledge-bound agents ground answers and actions in approved sources, policies, and citation rules.

Core Protocol

1. Classify the information need: working state, episodic memory, semantic knowledge, policy, or source evidence.
2. Retrieve only scoped, relevant, and permitted material.
3. Inject retrieved material with source labels, freshness, and trust level.
4. Generate or act while keeping retrieved evidence separate from instructions.
5. Write back memory only after validation, consent, retention, and correction rules pass.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Use questions with known source answers, stale sources, conflicting sources, and missing evidence.
- Measure recall, precision, citation faithfulness, freshness, and refusal when evidence is absent.
- Test deletion, correction, and privacy boundaries separately from answer quality.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Define retention, deletion, correction, and consent rules.
- Separate instructions from retrieved facts and user memories.
- Record source IDs and retrieval scores for audit and debugging.
- Add guards against prompt injection from retrieved documents.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

This pattern currently has no dedicated code excerpt. Use the source and download links below for the full pattern folder.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `compliance-policy-enforcer-agent/` folder from this repository.

Related Patterns

- Memory-Augmented Agent
- Long-Term Episodic Memory
- Semantic Recall and RAG
- Choosing the Right Pattern
- Resource-Aware Agent Design

Skills

Skills package procedural knowledge as discoverable folders of instructions, references, scripts, templates, and assets.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

The Skills Pattern packages procedural knowledge as discoverable folders: concise instructions, references, scripts, templates, and tests that an agent loads only when relevant.

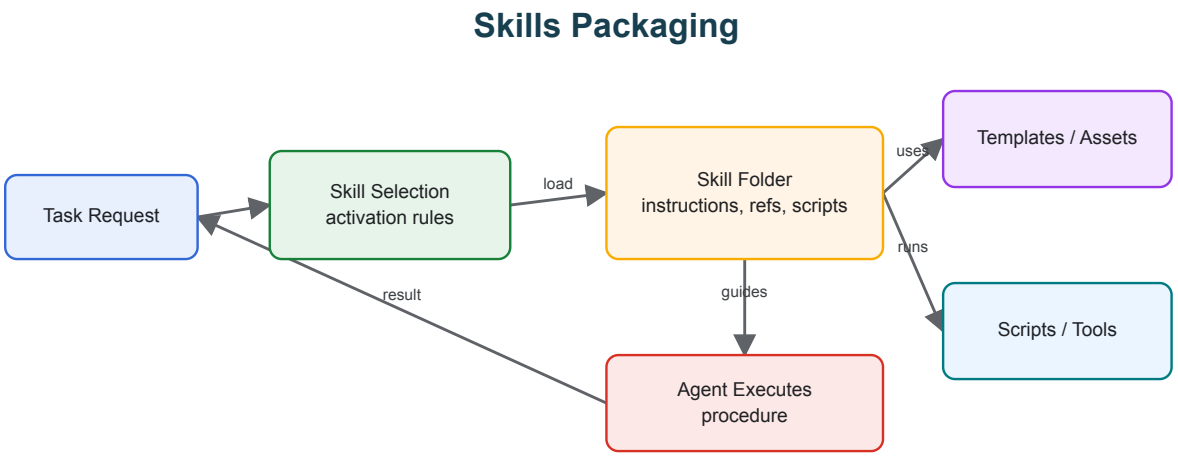
Use When

- A capability requires repeatable domain procedure rather than only a tool API.
- You want reusable know-how across agents, teams, or projects.
- The agent benefits from progressive disclosure: short instructions first, deeper references only when needed.

Avoid When

- A simple tool schema fully describes the capability.
- The skill would embed secrets, credentials, or unsafe scripts.
- The instructions are too vague to test with real tasks.

Architecture



System Shape

- **Pattern boundary:** the agent discovers or selects a capability, submits a typed request, and receives a typed result across a policy boundary.
- **State owner:** the protocol or capability boundary owns schemas, permissions, invocation records, and response validation.
- **Primary artifact:** `skills-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Skills package procedural knowledge as discoverable folders of instructions, references, scripts, templates, and assets.

Core Protocol

1. Discover the capability, schema, permissions, and operating constraints.
2. Prepare a typed request from the current goal and state.
3. Authorize the request before invocation.
4. Invoke the tool, skill, or remote agent and validate the result.
5. Return structured output, refusal, progress, or error without losing correlation IDs.

Implementation Notes

- Keep `SKILL.md` short and route to deeper files only when needed.
- Bundle scripts and templates instead of asking the model to recreate fragile artifacts.
- Treat skills as supply-chain inputs: review, version, test, and restrict execution.
- Include examples of successful and unsuccessful use.

Failure Modes

- Skill descriptions that are too broad, causing irrelevant activation.
- Long instruction files that consume context before the task is understood.
- Hidden dependencies that only work on one machine.
- Malicious or outdated bundled scripts.

Evaluation Strategy

- Test valid calls, invalid arguments, unauthorized calls, timeouts, refusals, and malformed responses.
- Assert that dangerous actions require approval or are blocked before execution.
- Measure tool-selection accuracy, schema validity, authorization failures, and recovery behavior.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Use typed schemas for inputs and outputs.
- Separate model intent from actual execution permissions.
- Add timeouts, retries, idempotency keys, and audit records.
- Treat refusal and cancellation as first-class outcomes.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

This pattern currently has no dedicated code excerpt. Use the source and download links below for the full pattern folder.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `skills-pattern/` folder from this repository.

Related Patterns

- MCP-first Tool Use
- Context Engineering
- Human-in-the-Loop Approval

MCP-first Tool Use

MCP-first tool use separates tool capability from agent logic through manifests, validation, invocation, and structured results.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Use this pattern when the set of tools may evolve independently from the agent. MCP gives tools a manifest boundary so the agent can inspect capabilities instead of relying on hard-coded assumptions.

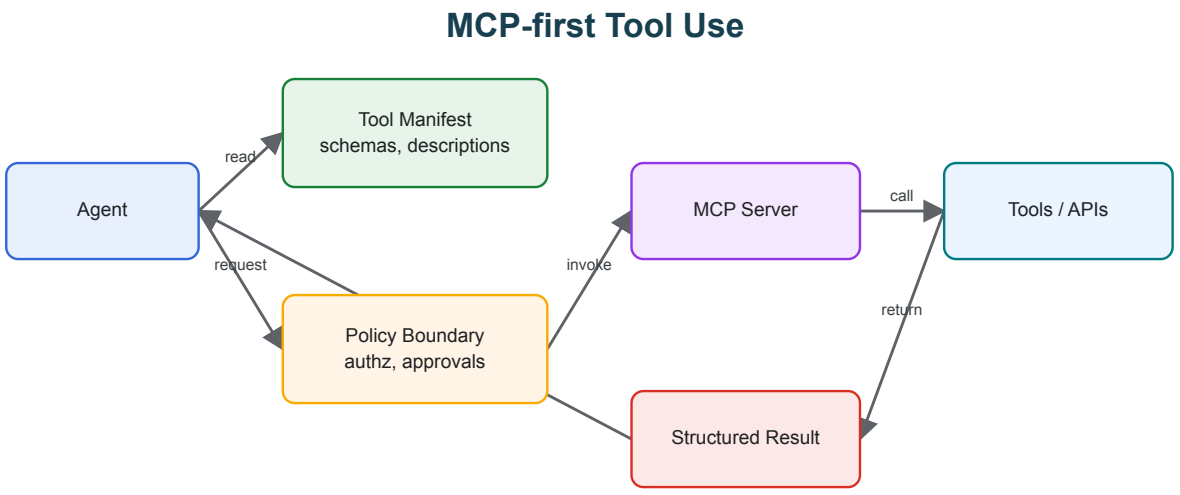
Use When

- Tools are shared across agents or applications.
- Tool schemas, context, and permissions need to be discoverable.
- You want to test tool invocation separately from model reasoning.

Avoid When

- A local function call is enough and no discovery boundary is needed.
- Tool inputs cannot be validated.
- The agent has permission to call too many tools without policy checks.

Architecture



System Shape

- **Pattern boundary:** the agent discovers or selects a capability, submits a typed request, and receives a typed result across a policy boundary.
- **State owner:** the protocol or capability boundary owns schemas, permissions, invocation records, and response validation.
- **Primary artifact:** `modern-tool-use-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** MCP-first tool use separates tool capability from agent logic through manifests, validation, invocation, and structured results.
- **Runnable path:** start with `npm run mcp:search` before adapting the pattern to a larger system.

Core Protocol

1. Discover the capability, schema, permissions, and operating constraints.
2. Prepare a typed request from the current goal and state.
3. Authorize the request before invocation.
4. Invoke the tool, skill, or remote agent and validate the result.
5. Return structured output, refusal, progress, or error without losing correlation IDs.

Implementation Notes

- Validate tool input before every invocation.
- Keep tool results structured and observable.
- Put policy checks between model intent and tool execution.
- Prefer small tools with clear contracts over broad tools with vague descriptions.

Failure Modes

- Tool manifests that describe capabilities too vaguely.
- Model-generated tool inputs used without schema validation.
- Hidden side effects that are not visible in the tool contract.
- No trace linking model decision, tool input, and tool result.

Evaluation Strategy

- Test valid calls, invalid arguments, unauthorized calls, timeouts, refusals, and malformed responses.
- Assert that dangerous actions require approval or are blocked before execution.
- Measure tool-selection accuracy, schema validity, authorization failures, and recovery behavior.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Use typed schemas for inputs and outputs.
- Separate model intent from actual execution permissions.
- Add timeouts, retries, idempotency keys, and audit records.
- Treat refusal and cancellation as first-class outcomes.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Run the Example

```
npm run mcp:search
npm run mcp:cloud
npm run mcp:agent
npm run mcp:test
```

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

These excerpts show the implementation shape. The complete code is available in the download bundle and repository source.

modern-tool-use-pattern/typescript/src/agent.ts

Open full source

```
import axios from 'axios';
import Ajv from 'ajv';
import { pathToFileURL } from 'node:url';

const ajv = new Ajv({ allErrors: true, strict: true });

async function getManifest(base: string) {
  const { data } = await axios.get(`${base}/manifest`);
  return data;
}

async function invoke(base: string, tool: string, input: any) {
  const { data } = await axios.post(`${base}/invoke`, { tool, input });
  if (!data?.ok) throw new Error(data?.error || 'invoke_failed');
  return data.output;
}

export async function runScenario() {
  // Discover tools via MCP manifests
  const search = await getManifest('http://localhost:3031');
  const cloud = await getManifest('http://localhost:3032');
  // Validate inputs before invoking
  const searchSchema = search.tools.find((t: any) => t.name ===
'search.query')?.input_schema;
  const cloudStoreSchema = cloud.tools.find((t: any) => t.name ===
'cloud.store')?.input_schema;
  const vSearch = ajv.compile(searchSchema);
  const vCloudStore = ajv.compile(cloudStoreSchema);
  // Plan (deterministic): search for a topic, then store results in cloud
  const q = 'agents';
  const inputSearch = { q, limit: 2 };
  if (!vSearch(inputSearch)) throw new Error('bad search input');
  const { items } = await invoke('http://localhost:3031', 'search.query',
```

```
inputSearch);
  const doc = { topic: q, titles: items.map((i: any) => i.title) };
  const inputStore = { key: `topic:${q}`, value: doc };
  if (!vCloudStore(inputStore)) throw new Error('bad cloud.store input');
  await invoke('http://localhost:3032', 'cloud.store', inputStore);
  return doc;
}

async function main() {
  const out = await runScenario();
  console.log('Stored doc:', out);
}

if (process.argv[1] && import.meta.url === pathToFileURL(process.argv[1]).href) {
  main().catch(err => { console.error(err); process.exit(1); });
}
```

modern-tool-use-pattern/typescript/src/mcp_search_server.ts

Open full source

```
import express from 'express';
import type { Request, Response } from 'express-serve-static-core';

const app = express();
app.use(express.json());

const manifest = {
  name: 'search-tools',
  version: '1.0.0',
  tools: [
    {
      name: 'search.query',
      description: 'Mock web search returning titles for a query',
      input_schema: {
        type: 'object',
        required: ['q'],
        properties: {
          q: { type: 'string' },
          limit: { type: 'number', minimum: 1, maximum: 5 }
        },
```

```
        additionalProperties: false
      }
    }
  ]
};

app.get('/manifest', (_req: Request, res: Response) => res.json(manifest));

app.post('/invoke', (req: Request, res: Response) => {
  const { tool, input } = req.body || {};
  if (tool === 'search.query') {
    const q = String(input?.q || '').trim();
    const limit = Math.max(1, Math.min(5, Number(input?.limit ?? 3)));
    const items = Array.from({ length: limit }, (_, i) => ({ title: `${q} result ${i
+ 1}` }));
    return res.json({ ok: true, output: { items } });
  }
  return res.status(400).json({ ok: false, error: 'unknown_tool' });
});

app.listen(3031, () => console.log('MCP search server on 3031'));
```

modern-tool-use-pattern/typescript/src/mcp_cloud_server.ts

Open full source

```
import express from 'express';
import type { Request, Response } from 'express-serve-static-core';

const app = express();
app.use(express.json());

const store = new Map<string, any>();

const manifest = {
  name: 'cloud-store',
  version: '1.0.0',
  tools: [
    {
      name: 'cloud.store',
      description: 'Store a JSON document under a key',
```

```
      input_schema: {
        type: 'object',
        required: ['key', 'value'],
        properties: { key: { type: 'string' }, value: { type: 'object' } },
        additionalProperties: false
      }
    },
    {
      name: 'cloud.fetch',
      description: 'Fetch a document by key',
      input_schema: {
        type: 'object',
        required: ['key'],
        properties: { key: { type: 'string' } },
        additionalProperties: false
      }
    }
  ]
};

app.get('/manifest', (_req: Request, res: Response) => res.json(manifest));

app.post('/invoke', (req: Request, res: Response) => {
  const { tool, input } = req.body || {};
  if (tool === 'cloud.store') {
    store.set(String(input.key), input.value);
    return res.json({ ok: true });
  }
  if (tool === 'cloud.fetch') {
    return res.json({ ok: true, output: { value: store.get(String(input.key)) ?? null
} });
  }
  return res.status(400).json({ ok: false, error: 'unknown_tool' });
});

app.listen(3032, () => console.log('MCP cloud server on 3032'));
```

Download

- Download source bundle
- Open source folder

The download bundle contains the current `modern-tool-use-pattern/` folder from this repository.

Related Patterns

- Skills
- A2A Agent Interoperability
- Secure Agent Communication
- Choosing the Right Pattern
- Resource-Aware Agent Design

A2A Agent Interoperability

A2A makes agents discoverable and callable across process, team, runtime, and vendor boundaries.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Use this pattern when one agent needs to call another agent as a remote collaborator. The protocol boundary should make capability discovery, task submission, progress, refusal, cancellation, and results explicit.

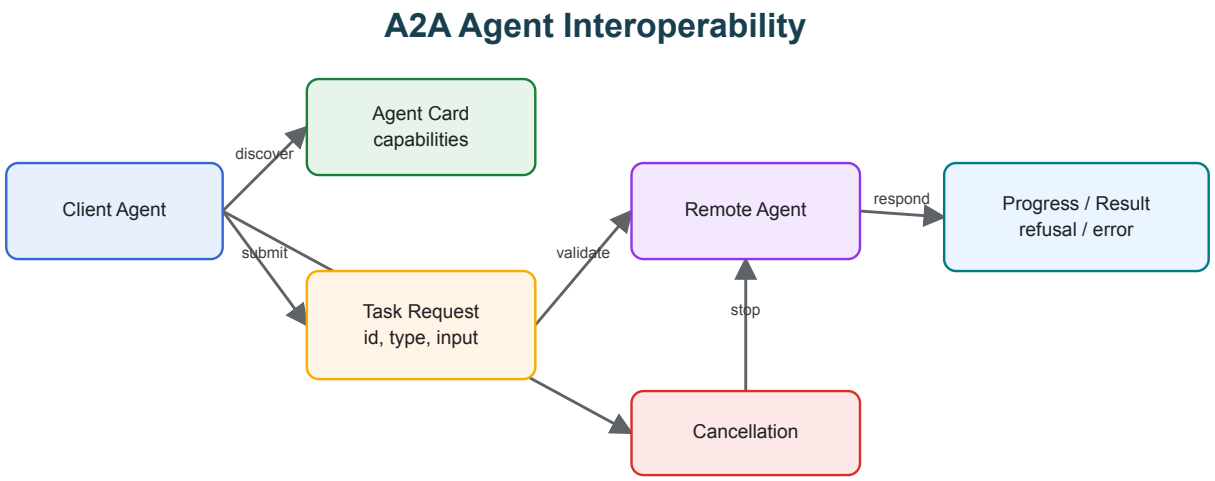
Use When

- Agents are owned by different services, teams, runtimes, or vendors.
- A caller must discover what a remote agent can do before sending work.
- Task state must survive asynchronous progress, refusal, error, or cancellation.

Avoid When

- Both agents are simple functions inside one process.
- The interaction is only a local tool call with a typed input and output.
- You cannot authenticate callers or validate messages.

Architecture



System Shape

- **Pattern boundary:** the agent discovers or selects a capability, submits a typed request, and receives a typed result across a policy boundary.
- **State owner:** the protocol or capability boundary owns schemas, permissions, invocation records, and response validation.
- **Primary artifact:** `agent-to-agent-communication-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** A2A makes agents discoverable and callable across process, team, runtime, and vendor boundaries.
- **Runnable path:** start with `npm run a2a:test` before adapting the pattern to a larger system.

Core Protocol

1. Discover the capability, schema, permissions, and operating constraints.
2. Prepare a typed request from the current goal and state.
3. Authorize the request before invocation.
4. Invoke the tool, skill, or remote agent and validate the result.
5. Return structured output, refusal, progress, or error without losing correlation IDs.

Implementation Notes

- Messages validated against schemas before delivery.
- Bus is in-memory; swap to a real transport without changing messages.
- Treat refusals as valid protocol outcomes, not exceptions.
- Include idempotency keys or task IDs so retries do not duplicate work.

- Add authentication and authorization before crossing a trust boundary.

Failure Modes

- Treating a remote agent like a local tool and ignoring latency, refusal, or cancellation.
- Sending unvalidated natural language blobs instead of typed task messages.
- Missing capability discovery, causing callers to rely on stale assumptions.
- No correlation ID across progress, result, and error messages.

Evaluation Strategy

- Test valid calls, invalid arguments, unauthorized calls, timeouts, refusals, and malformed responses.
- Assert that dangerous actions require approval or are blocked before execution.
- Measure tool-selection accuracy, schema validity, authorization failures, and recovery behavior.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Use typed schemas for inputs and outputs.
- Separate model intent from actual execution permissions.
- Add timeouts, retries, idempotency keys, and audit records.
- Treat refusal and cancellation as first-class outcomes.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Run the Example

```
npm run a2a:test
npm run a2a:run
```

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

These excerpts show the implementation shape. The complete code is available in the download bundle and repository source.

agent-to-agent-communication-pattern/src/run_demo.ts

Open full source

```
import { BusMemory } from './bus_memory.ts';
import { AgentA } from './agent_a.ts';
import { AgentB } from './agent_b.ts';

async function run() {
  const bus = new BusMemory();
  const a = new AgentA(bus);
  const b = new AgentB(bus);
  a.start();
  b.start();
  a.handshake();
  a.requestTask('t1', 'sum', { a: 2, b: 5 });
}

run();
```

agent-to-agent-communication-pattern/src/agent_a.ts

Open full source

```
import { BusMemory, A2A_SCHEMA } from './bus_memory.ts';
import type { Msg } from './bus_memory.ts';
import Ajv from 'ajv';
const ajv = new Ajv({ allErrors: true, strict: true });
const validateResponse = ajv.compile((A2A_SCHEMA as any).properties.TaskResponse);

export class AgentA {
  private bus: BusMemory;
  constructor(bus: BusMemory) { this.bus = bus; }
  start() {
    // listen for responses
    this.bus.subscribe('TaskResponse', (m: Msg) => {
```

```
      if (!validateResponse(m.payload)) {
        console.error('Invalid TaskResponse', validateResponse.errors);
        return;
      }
      console.log('AgentA received response:', m.payload);
    });
  }
  handshake() {
    this.bus.publish({ type: 'Handshake', payload: { version: '1.0', capabilities:
['tasks', 'cancel'] } });
  }
  requestTask(id: string, task_type: string, input: any) {
    this.bus.publish({ type: 'TaskRequest', payload: { id, task_type, input, meta: {
ts: Date.now() } } });
  }
  cancel(id: string, reason: string) {
    this.bus.publish({ type: 'Cancel', payload: { id, reason } });
  }
}
```

agent-to-agent-communication-pattern/src/agent_b.ts

Open full source

```
import { BusMemory, A2A_SCHEMA } from './bus_memory.ts';
import type { Msg } from './bus_memory.ts';
import Ajv from 'ajv';
const ajv = new Ajv({ allErrors: true, strict: true });
const validateReq = ajv.compile((A2A_SCHEMA as any).properties.TaskRequest);

export class AgentB {
  private bus: BusMemory;
  private handshakeAckSent = false;
  constructor(bus: BusMemory) { this.bus = bus; }
  start() {
    this.bus.subscribe('Handshake', () => {
      if (this.handshakeAckSent) return;
      this.handshakeAckSent = true;
      this.bus.publish({ type: 'Handshake', payload: { version: '1.0', capabilities:
['tasks'] } });
    });
```

```
    this.bus.subscribe('TaskRequest', (m: Msg) => {
      if (!validateReq(m.payload)) return;
      const payload = m.payload as any;
      const { id, task_type, input } = payload;
      if (task_type !== 'sum') {
        this.bus.publish({ type: 'TaskResponse', payload: { id, status: 'refused',
error: 'unsupported_task' } });
        return;
      }
      // progress
      this.bus.publish({ type: 'Progress', payload: { id, stage: 'start', pct: 10,
message: 'starting' } });
      const a = input?.a;
      const b = input?.b;
      if (typeof a !== 'number' || typeof b !== 'number') {
        this.bus.publish({ type: 'TaskResponse', payload: { id, status: 'error',
error: 'invalid_input' } });
        return;
      }
      // compute safely
      const sum = a + b;
      this.bus.publish({ type: 'Progress', payload: { id, stage: 'compute', pct: 60 }
});
      this.bus.publish({ type: 'TaskResponse', payload: { id, status: 'success',
output: { sum } } });
    });
    this.bus.subscribe('Cancel', (m: Msg) => {
      console.log('AgentB cancel received:', m.payload);
    });
  }
}
```

Download

- Download source bundle
- Open source folder

The download bundle contains the current `agent-to-agent-communication-pattern/` folder from this repository.

Related Patterns

- Skills
- MCP-first Tool Use
- Secure Agent Communication
- Choosing the Right Pattern
- Resource-Aware Agent Design

Secure Agent Communication

Secure communication protects messages between agents with authentication, integrity, confidentiality, and policy checks.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Secure communication protects messages between agents with authentication, integrity, confidentiality, and policy checks.

Use When

- Agents cross trust boundaries.
- Messages contain private data or trigger external side effects.
- You need identity, authorization, replay protection, and audit logs.

Avoid When

- All communication is local and already protected by process boundaries.
- Security checks cannot be enforced before action.
- The protocol lacks correlation IDs and auditability.

System Shape

- **Pattern boundary:** the agent discovers or selects a capability, submits a typed request, and receives a typed result across a policy boundary.
- **State owner:** the protocol or capability boundary owns schemas, permissions, invocation records, and response validation.
- **Primary artifact:** `secure-agent-communication-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Secure communication protects messages between agents with authentication, integrity, confidentiality, and policy checks.
- **Runnable path:** start with `npm run secure-agent` before adapting the pattern to a larger system.

Core Protocol

1. Discover the capability, schema, permissions, and operating constraints.
2. Prepare a typed request from the current goal and state.
3. Authorize the request before invocation.
4. Invoke the tool, skill, or remote agent and validate the result.
5. Return structured output, refusal, progress, or error without losing correlation IDs.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Test valid calls, invalid arguments, unauthorized calls, timeouts, refusals, and malformed responses.
- Assert that dangerous actions require approval or are blocked before execution.
- Measure tool-selection accuracy, schema validity, authorization failures, and recovery behavior.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Use typed schemas for inputs and outputs.
- Separate model intent from actual execution permissions.
- Add timeouts, retries, idempotency keys, and audit records.
- Treat refusal and cancellation as first-class outcomes.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Run the Example

```
npm run secure-agent
```

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

These excerpts show the implementation shape. The complete code is available in the download bundle and repository source.

secure-agent-communication-pattern/autogen_typescript_example/secure_agent.ts

Open full source

```
import crypto from 'crypto';
import readline from 'readline';

// Simulated symmetric key for demo (never hardcode in production)
const SECRET_KEY = 'my_demo_secret_key_123';

function encrypt(text: string): string {
  const cipher = crypto.createCipher('aes-256-ctr', SECRET_KEY);
  let encrypted = cipher.update(text, 'utf8', 'hex');
  encrypted += cipher.final('hex');
  return encrypted;
}

function decrypt(encrypted: string): string {
  const decipher = crypto.createDecipher('aes-256-ctr', SECRET_KEY);
  let decrypted = decipher.update(encrypted, 'hex', 'utf8');
  decrypted += decipher.final('utf8');
  return decrypted;
}

async function main() {
  const rl = readline.createInterface({
```

```
    input: process.stdin,
    output: process.stdout,
  });

  rl.question('Agent 1: Enter a message to send securely: ', (message) => {
    const encrypted = encrypt(message);
    console.log('\n[Agent 1] Encrypted message sent:', encrypted);

    // Simulate Agent 2 receiving and decrypting
    const decrypted = decrypt(encrypted);
    console.log('[Agent 2] Decrypted message received:', decrypted);
    rl.close();
  });
}

main();
```

secure-agent-communication-pattern/langgraph_python_example/secure_agent.py

Open full source

```
import os
from cryptography.fernet import Fernet

def get_key():
    # For demo, use a static key (never do this in production)
    return Fernet.generate_key()

SECRET_KEY = get_key()
fernet = Fernet(SECRET_KEY)

def encrypt(text):
    return fernet.encrypt(text.encode()).decode()

def decrypt(token):
    return fernet.decrypt(token.encode()).decode()

def main():
    message = input('Agent 1: Enter a message to send securely: ')
    encrypted = encrypt(message)
    print(f'\n[Agent 1] Encrypted message sent: {encrypted}')
```

```
    # Simulate Agent 2 receiving and decrypting
    decrypted = decrypt(encrypted)
    print(f'[Agent 2] Decrypted message received: {decrypted}')

if __name__ == '__main__':
    main()
```

Download

- Download source bundle
- Open source folder

The download bundle contains the current `secure-agent-communication-pattern/` folder from this repository.

Related Patterns

- Skills
- MCP-first Tool Use
- A2A Agent Interoperability
- Choosing the Right Pattern
- Resource-Aware Agent Design

Human Approval Gates

Human approval gates pause execution before sensitive, expensive, destructive, or externally visible actions.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Human approval gates pause execution before sensitive, expensive, destructive, or externally visible actions.

Use When

- The agent may trigger irreversible or high-risk actions.
- A human must review context before continuation.
- The workflow can preserve state while waiting.

Avoid When

- Every step requires approval and the agent adds no value.
- Approval records cannot capture who approved what and why.
- The workflow cannot safely resume after waiting.

System Shape

- **Pattern boundary:** the agent discovers or selects a capability, submits a typed request, and receives a typed result across a policy boundary.
- **State owner:** the protocol or capability boundary owns schemas, permissions, invocation records, and response validation.
- **Primary artifact:** `human-in-the-loop-approval-agent/` contains the runnable reference implementation and examples.
- **Operational promise:** Human approval gates pause execution before sensitive, expensive, destructive, or externally visible actions.

Core Protocol

1. Discover the capability, schema, permissions, and operating constraints.
2. Prepare a typed request from the current goal and state.
3. Authorize the request before invocation.
4. Invoke the tool, skill, or remote agent and validate the result.
5. Return structured output, refusal, progress, or error without losing correlation IDs.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Test valid calls, invalid arguments, unauthorized calls, timeouts, refusals, and malformed responses.
- Assert that dangerous actions require approval or are blocked before execution.
- Measure tool-selection accuracy, schema validity, authorization failures, and recovery behavior.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Use typed schemas for inputs and outputs.
- Separate model intent from actual execution permissions.
- Add timeouts, retries, idempotency keys, and audit records.
- Treat refusal and cancellation as first-class outcomes.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

This pattern currently has no dedicated code excerpt. Use the source and download links below for the full pattern folder.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `human-in-the-loop-approval-agent/` folder from this repository.

Related Patterns

- Skills
- MCP-first Tool Use
- A2A Agent Interoperability
- Choosing the Right Pattern
- Resource-Aware Agent Design

Task Delegation

Task delegation assigns bounded subtasks to specialized workers and combines their outputs.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Task delegation assigns bounded subtasks to specialized workers and combines their outputs.

Use When

- Specialized workers can complete independent subtasks better than one general agent.
- The manager can define expected outputs, constraints, and acceptance criteria.
- Subtask results can be merged and checked.

Avoid When

- The task has no useful decomposition.
- Workers share hidden mutable state.
- The manager cannot evaluate returned work.

System Shape

- **Pattern boundary:** a coordinator delegates bounded work to agents with narrow roles, then evaluates and merges their outputs.
- **State owner:** the coordinator owns the shared goal, decomposition, assignments, merge policy, and final acceptance.
- **Primary artifact:** `task-delegation-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Task delegation assigns bounded subtasks to specialized workers and combines their outputs.
- **Runnable path:** start with `npm run task-delegation` before adapting the pattern to a larger system.

Core Protocol

1. Define the shared goal, worker roles, expected outputs, and acceptance criteria.
2. Split work only where independent or specialist execution adds value.
3. Dispatch tasks with scoped context and permissions.
4. Collect outputs, errors, refusals, and evidence from each worker.
5. Merge results through an explicit judge, reducer, supervisor, or human review gate.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Compare multi-agent output against a single-agent baseline on the same tasks.
- Test worker disagreement, worker failure, duplicated work, and bad merge decisions.
- Measure quality lift, latency cost, token cost, merge accuracy, and accountability.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Give every worker a narrow contract and permission set.
- Make the merge policy explicit before workers run.
- Log per-worker inputs, outputs, and decision evidence.
- Keep one owner for final acceptance and escalation.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Run the Example

```
npm run task-delegation
```

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

These excerpts show the implementation shape. The complete code is available in the download bundle and repository source.

task-delegation-pattern/autogen_typescript_example/task_delegation.ts

Open full source

```
// Task Delegation Pattern - Autogen TypeScript Example
// To run: npm install && npm run task-delegation

import axios from 'axios';
import * as readline from 'readline';
import * as dotenv from 'dotenv';
dotenv.config();

const MISTRAL_API_URL = 'https://api.mistral.ai/v1/chat/completions';
const MISTRAL_API_KEY = process.env.MISTRAL_API_KEY;

async function managerAgent(task: string): Promise<string> {
  // Step 1: Decompose the task
  const decomposePrompt = `You are a manager agent. Decompose the following task into
two subtasks and describe each one. Task: ${task}`;
  const decomposeResp = await axios.post(
    MISTRAL_API_URL,
    {
      model: 'mistral-tiny',
      messages: [{ role: 'user', content: decomposePrompt }],
    },
    {
      headers: {
        'Authorization': `Bearer ${MISTRAL_API_KEY}`,
```

```
        'Content-Type': 'application/json',
      },
    }
  );
  const subtasks =
decomposeResp.data.choices[0].message.content.split(/\n|\r/).filter(Boolean);
  const subtask1 = subtasks[0] || 'Subtask 1';
  const subtask2 = subtasks[1] || 'Subtask 2';

  // Step 2: Delegate to workers
  const worker1Prompt = `You are Worker Agent 1. Complete this subtask: ${subtask1}`;
  const worker2Prompt = `You are Worker Agent 2. Complete this subtask: ${subtask2}`;
  const [worker1Resp, worker2Resp] = await Promise.all([
    axios.post(MISTRAL_API_URL, {
      model: 'mistral-tiny',
      messages: [{ role: 'user', content: worker1Prompt }],
    }, {
      headers: {
        'Authorization': `Bearer ${MISTRAL_API_KEY}`,
        'Content-Type': 'application/json',
      },
    }),
    axios.post(MISTRAL_API_URL, {
      model: 'mistral-tiny',
      messages: [{ role: 'user', content: worker2Prompt }],
    }, {
      headers: {
        'Authorization': `Bearer ${MISTRAL_API_KEY}`,
        'Content-Type': 'application/json',
      },
    })
  ]);

  // Step 3: Aggregate results
  return `Results:\n- ${worker1Resp.data.choices[0].message.content}\n-
${worker2Resp.data.choices[0].message.content}`;
}

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
```

```
});

rl.question('Task: ', async (userInput: string) => {
  try {
    const result = await managerAgent(userInput);
    console.log(result);
  } catch (err) {
    console.error('Error:', err);
  }
  rl.close();
});
```

task-delegation-pattern/langgraph_python_example/task_delegation.py

Open full source

```
# Task Delegation Pattern - LangGraph Python Example

This example demonstrates the Task Delegation Pattern using LangGraph and Python. A manager agent decomposes a task and delegates subtasks to two worker agents, then aggregates the results. The LLM is Mistral.

## Requirements

- Python 3.8+
- `langgraph` library
- `python-dotenv` (for .env support)
- Mistral LLM API access

## Install dependencies

```bash
pip install langgraph python-dotenv requests
```

Example Code

```python
import os
from langgraph import Agent, Environment, LLM
from dotenv import load_dotenv
```
```

```
load_dotenv()

MISTRAL_API_KEY = os.getenv("MISTRAL_API_KEY")
MISTRAL_API_URL = "https://api.mistral.ai/v1/chat/completions"

class SimpleEnvironment(Environment):
    def get_observation(self):
        return input("Task: ")
    def send_action(self, action):
        print(action)

class ManagerAgent(Agent):
    def __init__(self, llm):
        self.llm = llm
    def act(self, task):
        prompt = f"You are a manager agent. Decompose the following task into two
subtasks and describe each one. Task: {task}"
        return self.llm.complete(prompt)

class WorkerAgent(Agent):
    def __init__(self, llm, name):
        self.llm = llm
        self.name = name
    def act(self, subtask):
        prompt = f"You are {self.name}. Complete this subtask: {subtask}"
        return self.llm.complete(prompt)

llm = LLM(
    provider="mistral",
    api_key=MISTRAL_API_KEY,
    api_url=MISTRAL_API_URL,
)

env = SimpleEnvironment()
manager = ManagerAgent(llm)
worker1 = WorkerAgent(llm, "Worker Agent 1")
worker2 = WorkerAgent(llm, "Worker Agent 2")

task = env.get_observation()
subtasks = manager.act(task).split("\n")
```

```
subtask1 = subtasks[0] if len(subtasks) > 0 else "Subtask 1"
subtask2 = subtasks[1] if len(subtasks) > 1 else "Subtask 2"
result1 = worker1.act(subtask1)
result2 = worker2.act(subtask2)
final = f"Results:\n- {result1}\n- {result2}"
env.send_action(final)
```

- Try a complex or multi-step task to see delegation in action.
- Make sure your `.env` file contains your Mistral API key.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `task-delegation-pattern/` folder from this repository.

Related Patterns

- Supervisor / Worker
- Debate and Consensus
- Parallel Agents
- Choosing the Right Pattern
- Resource-Aware Agent Design

Supervisor / Worker

Supervisor/Worker centralizes goal ownership, task state, routing, and quality gates while workers perform bounded specialist work.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Supervisor/Worker centralizes goal ownership, task state, routing, and quality gates while workers perform bounded specialist work.

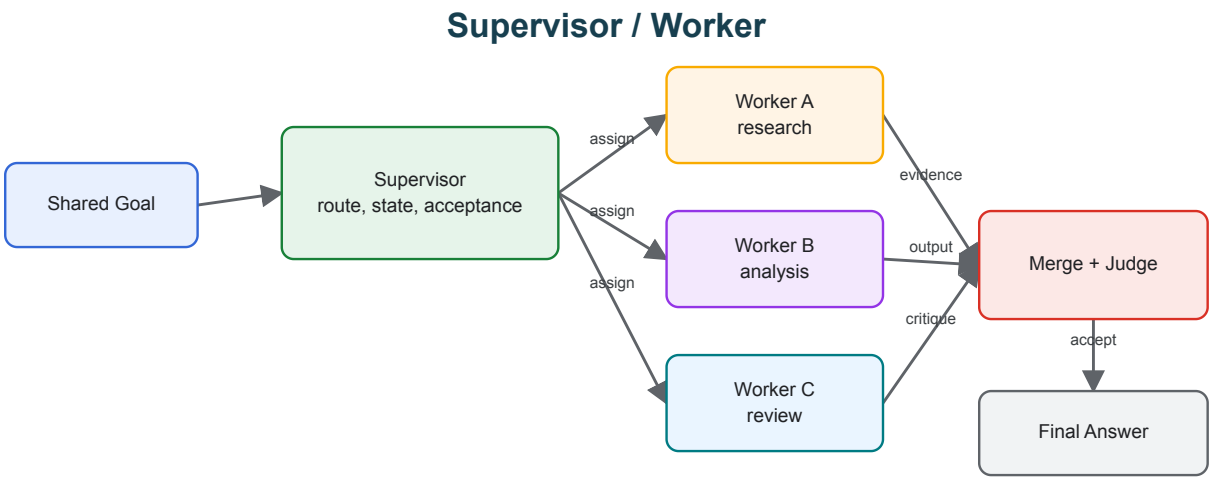
Use When

- Independent specialists are useful but centralized control is still required.
- The supervisor can route work and evaluate outputs.
- Workers have narrow roles and structured return contracts.

Avoid When

- The supervisor becomes a bottleneck with no added quality control.
- Workers can take uncontrolled side effects.
- No one owns final acceptance.

Architecture



System Shape

- **Pattern boundary:** a coordinator delegates bounded work to agents with narrow roles, then evaluates and merges their outputs.
- **State owner:** the coordinator owns the shared goal, decomposition, assignments, merge policy, and final acceptance.
- **Primary artifact:** `hierarchical-agent-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Supervisor/Worker centralizes goal ownership, task state, routing, and quality gates while workers perform bounded specialist work.
- **Runnable path:** start with `npm run hierarchical-agent` before adapting the pattern to a larger system.

Core Protocol

1. Define the shared goal, worker roles, expected outputs, and acceptance criteria.
2. Split work only where independent or specialist execution adds value.
3. Dispatch tasks with scoped context and permissions.
4. Collect outputs, errors, refusals, and evidence from each worker.
5. Merge results through an explicit judge, reducer, supervisor, or human review gate.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Compare multi-agent output against a single-agent baseline on the same tasks.
- Test worker disagreement, worker failure, duplicated work, and bad merge decisions.
- Measure quality lift, latency cost, token cost, merge accuracy, and accountability.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Give every worker a narrow contract and permission set.
- Make the merge policy explicit before workers run.
- Log per-worker inputs, outputs, and decision evidence.
- Keep one owner for final acceptance and escalation.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Run the Example

```
npm run hierarchical-agent
```

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

These excerpts show the implementation shape. The complete code is available in the download bundle and repository source.

`hierarchical-agent-pattern/autogen_typescript_example/hierarchical_agent.ts`

Open full source

```
import dotenv from 'dotenv';
dotenv.config();
import axios from 'axios';
import readline from 'readline';

const MISTRAL_API_KEY = process.env.MISTRAL_API_KEY;
const MISTRAL_API_URL = 'https://api.mistral.ai/v1/chat/completions';

if (!MISTRAL_API_KEY) {
  console.error('Please set MISTRAL_API_KEY in your .env file');
  process.exit(1);
}

async function askMistral(messages: any[]) {
  const response = await axios.post(
    MISTRAL_API_URL,
    {
      model: 'mistral-tiny',
      messages,
    },
    {
      headers: {
        'Authorization': `Bearer ${MISTRAL_API_KEY}`,
        'Content-Type': 'application/json',
      },
    }
  );
  return response.data.choices[0].message.content.trim();
}

async function main() {
  const rl = readline.createInterface({
    input: process.stdin,
    output: process.stdout,
  });

  rl.question('State the overall goal for the manager agent: ', async (goalInput) =>
{
    // Manager agent decomposes the goal into two sub-tasks
```

```
    let managerMessages = [
      { role: 'system', content: 'You are a manager agent. Decompose the user goal
into two sub-tasks and assign each to a worker agent.' },
      { role: 'user', content: `Goal: ${goalInput}` },
    ];
    let managerPlan = await askMistral(managerMessages);
    console.log('\nManager Agent Plan:\n', managerPlan);

    // Simulate two worker agents (for demo, just ask Mistral for each sub-task)
    const subTasks = managerPlan.match(/Sub-task [12]: (.*)/g) || [];
    let workerResults: string[] = [];
    for (let i = 0; i < subTasks.length; i++) {
      let workerMessages = [
        { role: 'system', content: `You are Worker Agent ${i+1}. Complete the
following sub-task as best as you can.` },
        { role: 'user', content: subTasks[i] },
      ];
      let workerResult = await askMistral(workerMessages);
      console.log(`\nWorker Agent ${i+1} Result:\n`, workerResult);
      workerResults.push(workerResult);
    }

    // Manager aggregates results
    let aggregationMessages = [
      { role: 'system', content: 'You are a manager agent. Aggregate the following
worker results into a final answer for the user.' },
      { role: 'user', content: workerResults.map((r, i) => `Worker ${i+1}:
${r}`).join('\n') },
    ];
    let finalResult = await askMistral(aggregationMessages);
    console.log('\nFinal Aggregated Result for User:\n', finalResult);

    rl.close();
  });
}

main();
```

hierarchical-agent-pattern/langgraph_python_example/hierarchical_agent.py

Open full source

```
import os
import requests
import re

def ask_mistral(messages):
    api_key = os.getenv('MISTRAL_API_KEY')
    if not api_key:
        raise ValueError('Please set MISTRAL_API_KEY in your .env file')
    url = 'https://api.mistral.ai/v1/chat/completions'
    headers = {
        'Authorization': f'Bearer {api_key}',
        'Content-Type': 'application/json',
    }
    data = {
        'model': 'mistral-tiny',
        'messages': messages,
    }
    response = requests.post(url, headers=headers, json=data)
    response.raise_for_status()
    return response.json()['choices'][0]['message']['content'].strip()

def main():
    goal_input = input('State the overall goal for the manager agent: ')
    # Manager agent decomposes the goal into two sub-tasks
    manager_messages = [
        {'role': 'system', 'content': 'You are a manager agent. Decompose the user
goal into two sub-tasks and assign each to a worker agent.'},
        {'role': 'user', 'content': f'Goal: {goal_input}'},
    ]
    manager_plan = ask_mistral(manager_messages)
    print('\nManager Agent Plan:\n', manager_plan)

    # Simulate two worker agents (for demo, just ask Mistral for each sub-task)
    sub_tasks = re.findall(r'Sub-task [12]: (.*)', manager_plan)
    worker_results = []
    for i, sub_task in enumerate(sub_tasks):
        worker_messages = [
            {'role': 'system', 'content': f'You are Worker Agent {i+1}. Complete the
following sub-task as best as you can.'},
            {'role': 'user', 'content': sub_task},
        ]
```

```
        worker_result = ask_mistral(worker_messages)
        print(f'\nWorker Agent {i+1} Result:\n', worker_result)
        worker_results.append(worker_result)

    # Manager aggregates results
    aggregation_messages = [
        {'role': 'system', 'content': 'You are a manager agent. Aggregate the
following worker results into a final answer for the user.'},
        {'role': 'user', 'content': '\n'.join([f'Worker {i+1}: {r}' for i, r in
enumerate(worker_results)])},
    ]
    final_result = ask_mistral(aggregation_messages)
    print('\nFinal Aggregated Result for User:\n', final_result)

if __name__ == '__main__':
    main()
```

Download

- Download source bundle
- Open source folder

The download bundle contains the current `hierarchical-agent-pattern/` folder from this repository.

Related Patterns

- Task Delegation
- Debate and Consensus
- Parallel Agents
- Choosing the Right Pattern
- Resource-Aware Agent Design

Debate and Consensus

Debate and consensus use multiple independent proposals, critiques, votes, or rankings before producing a final answer.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Debate and consensus use multiple independent proposals, critiques, votes, or rankings before producing a final answer.

Use When

- Diversity of reasoning improves quality.
- Aggregation rules are clear before agents start.
- The system can tolerate extra cost and latency.

Avoid When

- Agents are not independent and will repeat the same failure.
- Voting replaces evidence or tests.
- The final decision has no accountable owner.

System Shape

- **Pattern boundary:** a coordinator delegates bounded work to agents with narrow roles, then evaluates and merges their outputs.
- **State owner:** the coordinator owns the shared goal, decomposition, assignments, merge policy, and final acceptance.
- **Primary artifact:** `consensus-seeking-multi-agent-system-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Debate and consensus use multiple independent proposals, critiques, votes, or rankings before producing a final answer.

Core Protocol

1. Define the shared goal, worker roles, expected outputs, and acceptance criteria.
2. Split work only where independent or specialist execution adds value.
3. Dispatch tasks with scoped context and permissions.
4. Collect outputs, errors, refusals, and evidence from each worker.
5. Merge results through an explicit judge, reducer, supervisor, or human review gate.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Compare multi-agent output against a single-agent baseline on the same tasks.
- Test worker disagreement, worker failure, duplicated work, and bad merge decisions.
- Measure quality lift, latency cost, token cost, merge accuracy, and accountability.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Give every worker a narrow contract and permission set.
- Make the merge policy explicit before workers run.
- Log per-worker inputs, outputs, and decision evidence.
- Keep one owner for final acceptance and escalation.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

This pattern currently has no dedicated code excerpt. Use the source and download links below for the full pattern folder.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `consensus-seeking-multi-agent-system-pattern/` folder from this repository.

Related Patterns

- Task Delegation
- Supervisor / Worker
- Parallel Agents
- Choosing the Right Pattern
- Resource-Aware Agent Design

Parallel Agents

Parallel agents run independent work concurrently, then merge results through a fan-out/fan-in control point.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Parallel agents run independent work concurrently, then merge results through a fan-out/fan-in control point.

Use When

- Work can be split into independent searches, reviews, or candidate generations.
- Latency matters and parallelism is safe.
- The merge step can compare, rank, or synthesize results.

Avoid When

- Agents need shared mutable state during execution.
- The merge policy is vague.
- Parallel work increases cost without increasing quality.

System Shape

- **Pattern boundary:** a coordinator delegates bounded work to agents with narrow roles, then evaluates and merges their outputs.
- **State owner:** the coordinator owns the shared goal, decomposition, assignments, merge policy, and final acceptance.
- **Primary artifact:** `multi-agent-collaboration-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Parallel agents run independent work concurrently, then merge results through a fan-out/fan-in control point.
- **Runnable path:** start with `npm run multi-agent-collab` before adapting the pattern to a larger system.

Core Protocol

1. Define the shared goal, worker roles, expected outputs, and acceptance criteria.
2. Split work only where independent or specialist execution adds value.
3. Dispatch tasks with scoped context and permissions.
4. Collect outputs, errors, refusals, and evidence from each worker.
5. Merge results through an explicit judge, reducer, supervisor, or human review gate.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Compare multi-agent output against a single-agent baseline on the same tasks.
- Test worker disagreement, worker failure, duplicated work, and bad merge decisions.
- Measure quality lift, latency cost, token cost, merge accuracy, and accountability.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Give every worker a narrow contract and permission set.
- Make the merge policy explicit before workers run.
- Log per-worker inputs, outputs, and decision evidence.
- Keep one owner for final acceptance and escalation.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Run the Example

```
npm run multi-agent-collab
```

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

These excerpts show the implementation shape. The complete code is available in the download bundle and repository source.

multi-agent-collaboration-pattern/autogen_typescript_example/multi_agent_collab.ts

Open full source

```
// Multi-Agent Collaboration Pattern - Autogen TypeScript Example
// To run: npm install && npm run multi-agent-collab

import axios from 'axios';
import * as readline from 'readline';
import * as dotenv from 'dotenv';
dotenv.config();

const MISTRAL_API_URL = 'https://api.mistral.ai/v1/chat/completions';
const MISTRAL_API_KEY = process.env.MISTRAL_API_KEY;

async function agent(name: string, role: string, input: string): Promise<string> {
  const prompt = `You are ${name}, your role is: ${role}. Here is your input:
${input}`;
  const response = await axios.post(
    MISTRAL_API_URL,
    {
      model: 'mistral-tiny',
      messages: [{ role: 'user', content: prompt }],
    },
    {
      headers: {
        'Authorization': `Bearer ${MISTRAL_API_KEY}`,
        'Content-Type': 'application/json',
```

```
      },
    }
  );
  return response.data.choices[0].message.content;
}

async function multiAgentCollab(task: string) {
  // Agent 1: Idea Generator
  const idea = await agent('Alice', 'Idea Generator', task);
  console.log('Alice (Idea Generator):', idea);

  // Agent 2: Critic
  const critique = await agent('Bob', 'Critic', `Here is an idea: ${idea}\nPlease
critique or improve it.`);
  console.log('Bob (Critic):', critique);

  // Agent 1: Finalize
  const final = await agent('Alice', 'Idea Generator', `Here is the critique:
${critique}\nPlease finalize the solution.`);
  console.log('Alice (Finalized):', final);
}

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

rl.question('Task: ', async (userInput: string) => {
  try {
    await multiAgentCollab(userInput);
  } catch (err) {
    console.error('Error:', err);
  }
  rl.close();
});
```

multi-agent-collaboration-pattern/langgraph_python_example/multi_agent_collab.py

Open full source

````
# Multi-Agent Collaboration Pattern - LangGraph Python Example

This example demonstrates the Multi-Agent Collaboration Pattern using LangGraph and Python. Two agents (an Idea Generator and a Critic) collaborate to solve a task, exchanging messages and refining the solution. The LLM is Mistral.

## Requirements

- Python 3.8+
- `langgraph` library
- `python-dotenv` (for .env support)
- Mistral LLM API access

## Install dependencies

```bash
pip install langgraph python-dotenv requests
```

## Example Code

```python
import os
from langgraph import Agent, Environment, LLM
from dotenv import load_dotenv

load_dotenv()

MISTRAL_API_KEY = os.getenv("MISTRAL_API_KEY")
MISTRAL_API_URL = "https://api.mistral.ai/v1/chat/completions"

class SimpleEnvironment(Environment):
    def get_observation(self):
        return input("Task: ")
    def send_action(self, action):
        print(action)

class IdeaGenerator(Agent):
    def __init__(self, llm):
        self.llm = llm
    def act(self, observation):
````
````

```
        prompt = f"You are Alice, an Idea Generator. Here is your task:
{observation}"
        return self.llm.complete(prompt)

class Critic(Agent):
    def __init__(self, llm):
        self.llm = llm
    def act(self, idea):
        prompt = f"You are Bob, a Critic. Here is an idea: {idea}\nPlease critique or
improve it."
        return self.llm.complete(prompt)

llm = LLM(
    provider="mistral",
    api_key=MISTRAL_API_KEY,
    api_url=MISTRAL_API_URL,
)

env = SimpleEnvironment()
idea_agent = IdeaGenerator(llm)
critic_agent = Critic(llm)

task = env.get_observation()
idea = idea_agent.act(task)
print("Alice (Idea Generator):", idea)
critique = critic_agent.act(idea)
print("Bob (Critic):", critique)
final = idea_agent.act(f"Here is the critique: {critique}\nPlease finalize the
solution.")
print("Alice (Finalized):", final)
```

- Try a creative or open-ended task to see agent collaboration.
- Make sure your `.env` file contains your Mistral API key.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `multi-agent-collaboration-pattern/` folder from this repository.

Related Patterns

- Task Delegation
- Supervisor / Worker
- Debate and Consensus
- Choosing the Right Pattern
- Resource-Aware Agent Design

CrewAI Flows and Crews

CrewAI Flows own state and execution order. Crews group specialized agents that collaborate on delegated work inside the flow.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

The CrewAI Flows and Crews Pattern separates production workflow control from collaborative agent work. Flows own state and execution order; crews group specialized agents that perform delegated tasks inside the flow.

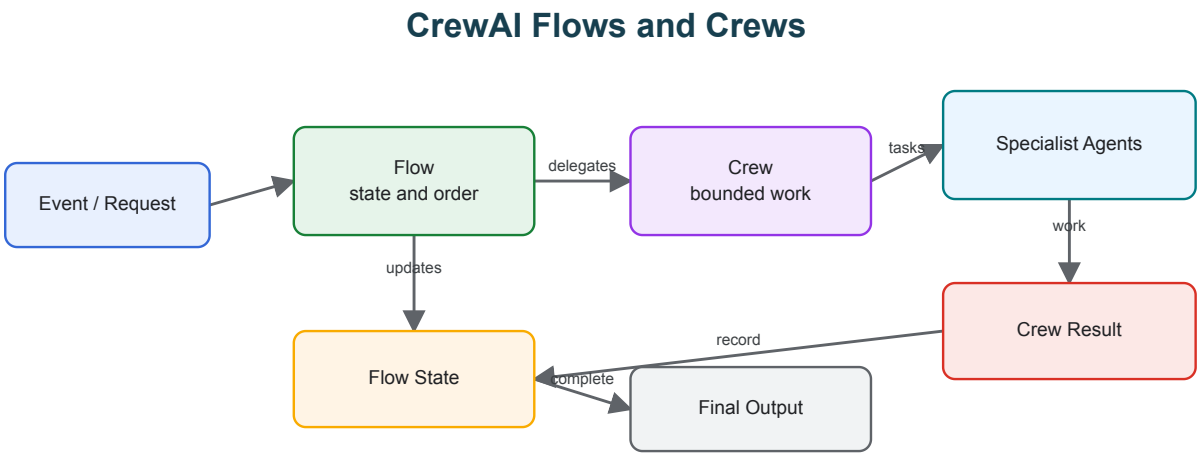
Use When

- You are building Python-first agent automations.
- The system needs explicit state and event-driven control flow.
- Multiple specialist agents need to collaborate on bounded tasks.

Avoid When

- A single deterministic workflow step is enough.
- Agents have unclear roles or overlapping responsibilities.
- You cannot define where flow state begins and crew-local context ends.

Architecture



System Shape

- **Pattern boundary:** a coordinator delegates bounded work to agents with narrow roles, then evaluates and merges their outputs.
- **State owner:** the coordinator owns the shared goal, decomposition, assignments, merge policy, and final acceptance.
- **Primary artifact:** `crewai-flows-and-crews-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** CrewAI Flows own state and execution order. Crews group specialized agents that collaborate on delegated work inside the flow.

Core Protocol

1. Define the shared goal, worker roles, expected outputs, and acceptance criteria.
2. Split work only where independent or specialist execution adds value.
3. Dispatch tasks with scoped context and permissions.
4. Collect outputs, errors, refusals, and evidence from each worker.
5. Merge results through an explicit judge, reducer, supervisor, or human review gate.

Implementation Notes

- Let flows manage state, branching, persistence, and execution order.
- Give each crew a bounded task with clear expected output.
- Give each agent a role that changes behavior, not just a different name.
- Test flow state transitions separately from crew output quality.

Failure Modes

- Crews used as a substitute for workflow design.
- Too many agents with vague roles.
- Flow state mutated implicitly through chat history.
- No evaluator for whether the crew result satisfies the flow step.

Evaluation Strategy

- Compare multi-agent output against a single-agent baseline on the same tasks.
- Test worker disagreement, worker failure, duplicated work, and bad merge decisions.
- Measure quality lift, latency cost, token cost, merge accuracy, and accountability.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Give every worker a narrow contract and permission set.
- Make the merge policy explicit before workers run.
- Log per-worker inputs, outputs, and decision evidence.
- Keep one owner for final acceptance and escalation.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

This pattern currently has no dedicated code excerpt. Use the source and download links below for the full pattern folder.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `crewai-flows-and-crews-pattern/` folder from this repository.

Related Patterns

- Task Delegation
- Consensus-Seeking Multi-Agent System
- Durable Workflow

Agentic System Architecture

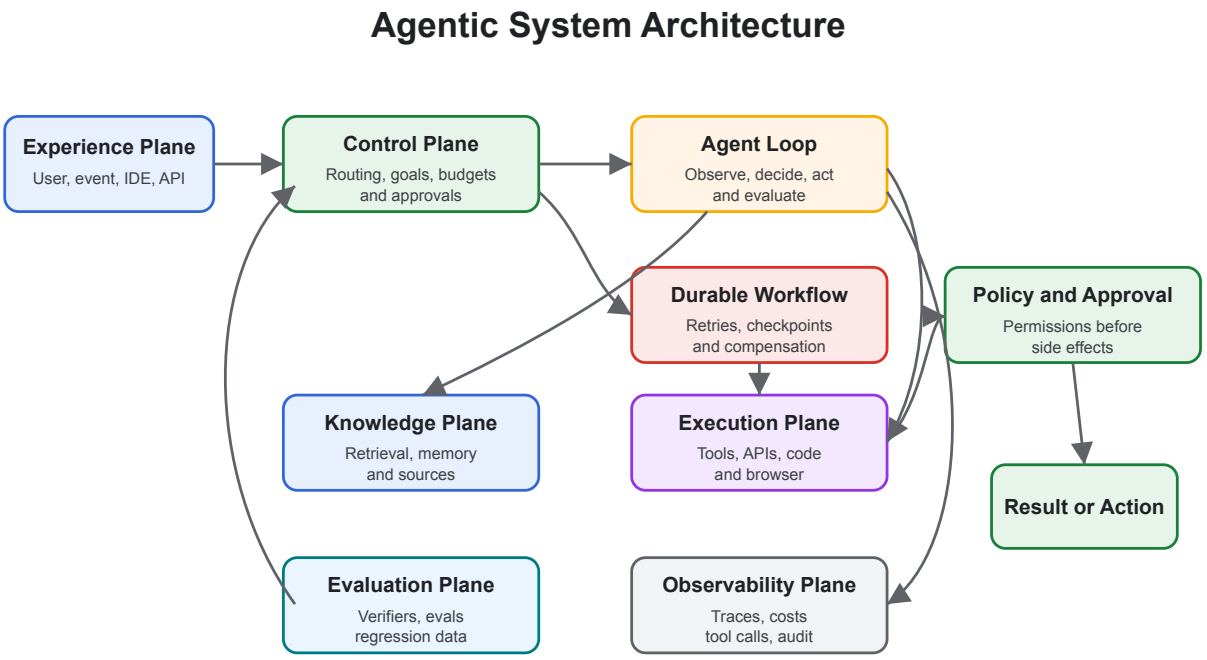
An agentic system is more than a model call wrapped in a loop. It is a set of control planes, execution planes, data planes, and safety planes that let software use model judgment without losing engineering control.

Use this chapter when a single pattern is not enough and you need to combine agents, tools, memory, policies, workflows, evals, and observability into one coherent system.

Core Idea

Separate the system into planes:

- **Experience plane:** chat, IDE, API, webhook, ticket, mobile, or scheduled entrypoint.
- **Control plane:** routing, planning, permissions, budgets, approvals, task state, and stop conditions.
- **Execution plane:** tools, code execution, browser actions, API calls, workflows, and external side effects.
- **Knowledge plane:** retrieval, memory, indexes, metadata, source freshness, and citations.
- **Evaluation plane:** offline evals, runtime checks, verifiers, red-team tests, and regression datasets.
- **Observability plane:** traces, costs, latency, tool calls, model inputs, decisions, and operator review.



Architecture Questions

- What owns the goal?
- What owns state?
- What may call tools?
- What requires approval?
- What happens when evidence is missing?
- What happens when the model is wrong?
- What can be replayed after a failure?
- What is deterministic, and what is model-mediated?

The system should have direct answers to those questions before it handles private data, money movement, production infrastructure, or customer-facing communication.

Boundary Design

The most important architectural choice is the boundary between model judgment and deterministic software. Keep model outputs as proposals until software validates them.

Good boundaries include:

- Typed tool schemas
- Policy checks before side effects
- Explicit state transitions

- Human approval for high-risk operations
- Retrieval filters and citations
- Budget and timeout limits
- Audit logs that connect prompt, decision, tool input, and result

Weak boundaries include:

- Broad shell or browser access without approval
- Prompt-only policy enforcement
- Unstructured memory writes
- Hidden retries
- Tool results that cannot be traced
- Agents that can rewrite their own operating rules without review

Composition Patterns

Common production systems combine several chapters from this book:

- **Agent Loop** for bounded observe-decide-act cycles.
- **Goals and State** for resumability and auditability.
- **MCP-first Tool Use** for discoverable tools.
- **Agentic RAG Systems** for evidence-grounded answers.
- **Durable Workflows** for long-running state, retries, and approvals.
- **Observability and Evals** for quality gates and regression control.
- **Policy Enforcement** for permission and compliance checks.

Failure Modes

- The model becomes the control plane and no deterministic component owns state or permissions.
- Every pattern is added at once, producing a system that is powerful but impossible to debug.
- Retrieval, memory, and tool output are mixed into one untrusted context blob.
- The system has no replay path after a bad action.
- Evals are added after production failures instead of before launch.

Design Rule

Architecture should make failure visible. If a run fails, an operator should be able to answer: what goal was active, what evidence was used, what tool calls ran, what policy checks passed, what changed, and why the system stopped.

Related Chapters

- Agent Loop
- Goals and State
- MCP-first Tool Use
- Agentic RAG Systems
- Durable Workflows
- Observability and Evals

Agentic RAG Systems

Agentic RAG uses agent behavior around retrieval. Instead of one retrieve-then-generate call, the system can plan queries, choose sources, call retrieval tools, inspect evidence, refine searches, verify citations, and decide whether to answer, ask for clarification, or escalate.

This chapter is architectural. The existing Semantic Recall and RAG chapter explains the retrieval pattern. This chapter explains how to build a full system around it.

Basic RAG vs Agentic RAG

Basic RAG is a pipeline:

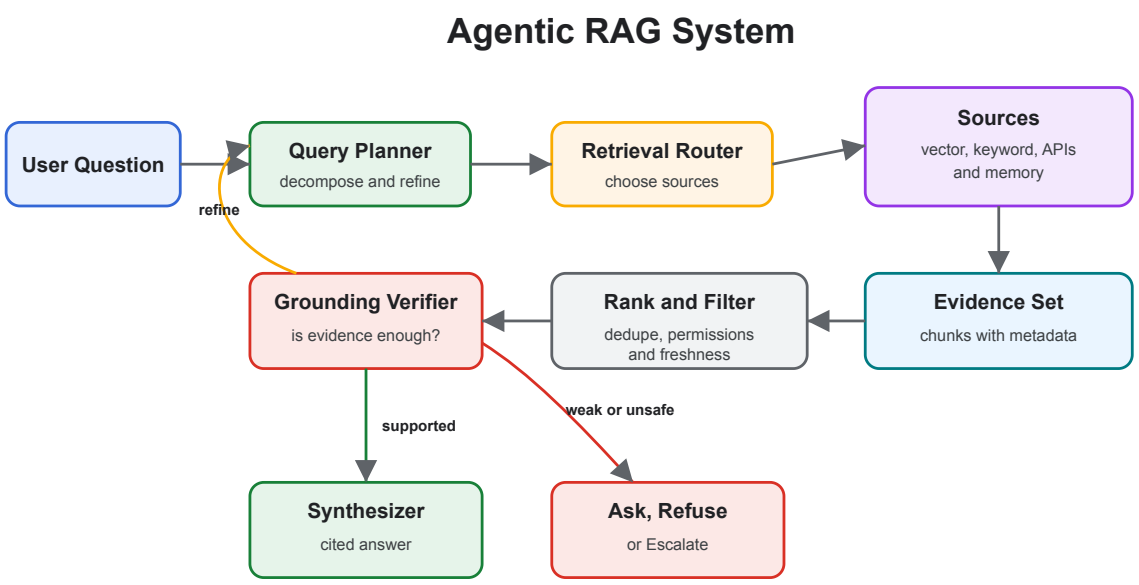
```
query -> retrieve -> inject context -> generate answer
```

Agentic RAG is a control loop:

```
goal -> plan retrieval -> search -> inspect evidence -> refine or verify -> answer or refuse
```

The shift matters because many real questions are not one search. They require decomposition, source selection, permissions, freshness checks, and answer verification.

Reference Flow



System Components

- **Query planner:** Decomposes complex requests into retrieval tasks.
- **Retrieval router:** Chooses indexes, databases, APIs, memories, or web sources.
- **Access filter:** Applies user permissions and source-level policy before retrieval.
- **Retriever:** Runs vector, keyword, graph, SQL, API, or hybrid search.
- **Ranker:** Removes duplicates, stale chunks, low-relevance evidence, and policy-ineligible content.
- **Verifier:** Checks whether the evidence actually supports the answer.
- **Synthesizer:** Produces the final answer with citations and uncertainty.
- **Telemetry:** Records query, sources, ranks, citations, costs, and verifier outcomes.

Use When

- The answer must be grounded in changing or private sources.
- The question may require multiple searches or source types.
- Users need citations, provenance, and freshness.
- Retrieval failures should lead to clarification, refusal, or escalation instead of hallucination.

Avoid When

- A deterministic database query would answer the question directly.
- The system cannot enforce source permissions.

- The corpus is too noisy to support reliable grounding.
- Citation quality is not evaluated.

Architecture Decisions

Define these decisions explicitly:

- Source inventory: which sources exist and who owns them.
- Freshness policy: how stale each source may be.
- Chunking policy: how documents are split and updated.
- Metadata policy: tenant, user, project, date, sensitivity, and source type.
- Retrieval strategy: vector, keyword, graph, SQL, API, or hybrid.
- Citation policy: what counts as a valid citation.
- Refusal policy: what happens when evidence is weak.
- Evaluation policy: what dataset proves retrieval quality.

Corrective RAG Loop

Corrective RAG adds a verifier before final synthesis. If the evidence is weak, the system changes the query or source rather than forcing an answer.

The diagram above shows this loop: weak evidence returns to planning, supported evidence moves to synthesis, and unsafe or missing evidence becomes a refusal, clarification, or escalation path.

Multi-Agent RAG

A multi-agent RAG system separates roles:

- Researcher: decomposes the information need.
- Retriever: searches specific sources.
- Verifier: checks evidence against claims.
- Synthesizer: writes the answer.
- Policy agent: checks source eligibility and sensitive-data boundaries.

Use separate agents only when the roles produce independent value. If all roles read the same evidence and repeat the same prompt, use one agent with structured steps.

Failure Modes

- Hallucinated citations: the answer cites a source that does not support the claim.
- Retrieval drift: the loop keeps searching adjacent topics instead of the user question.
- Over-fetching: too much context pushes out the important evidence.

- Permission leaks: retrieval returns documents the user should not see.
- Stale grounding: old documentation wins because it is easier to retrieve.
- Memory contamination: user preferences are treated as factual source material.

Production Controls

- Access-controlled indexes
- Metadata filters before retrieval
- Evidence-count and freshness thresholds
- Citation validation
- Retrieval eval datasets
- Source-level telemetry
- Human escalation for conflicting evidence
- Red-team tests for prompt injection in retrieved documents

Related Chapters

- Semantic Recall and RAG
- Context Engineering
- Knowledge-Bound Agents
- Evaluator-Optimizer
- Observability and Evals

Open Personal Agent Architectures

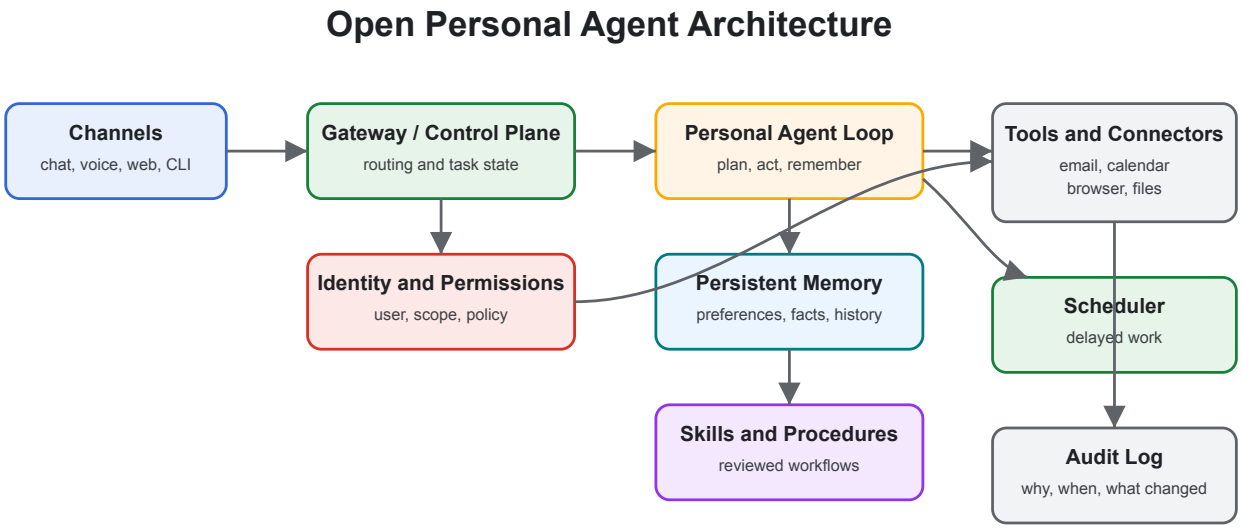
Open personal agents are self-hosted or user-controlled assistants that connect to chat apps, calendars, inboxes, browsers, files, memory, and automations. OpenClaw and Hermes Agent are useful examples because they emphasize different sides of the architecture: broad personal action versus long-term learning.

This chapter is not a product ranking. It uses these projects to explain the architecture of persistent personal agents.

Examples

- OpenClaw and its GitHub repository: a personal assistant that runs on user-controlled infrastructure and acts through channels such as chat apps, device surfaces, and connected tools.
- Hermes Agent and its GitHub repository: a persistent agent focused on memory, skill creation, and learning from repeated use.
- AutoGPT: a platform for creating and running continuous agents.
- OpenHands: an open-source platform for software-development agents.

Core Architecture



What Makes Them Different

A normal assistant answers in the current session. A personal agent keeps context and can act over time. That creates value and risk.

Useful capabilities:

- Persistent user preferences
- Project and relationship memory
- Scheduled work
- App integrations
- Reusable skills
- Multi-channel access
- Long-running tasks

New risks:

- Over-broad OAuth scopes
- Mistaken identity in email or chat
- Memory storing private or incorrect facts
- Automation without enough review
- Prompt injection through inboxes, calendars, pages, and documents
- Operational burden when self-hosted

OpenClaw-Style Pattern

OpenClaw-style systems optimize for reach: one assistant connected to many channels and tools.

Architecture emphasis:

- Gateway-first design
- Chat-native interaction
- Tool and app connectors
- User-owned deployment
- Broad task automation

Use this style when the main problem is giving a user one assistant across daily applications.

Hermes-Style Pattern

Hermes-style systems optimize for learning over time.

Architecture emphasis:

- Persistent memory
- Skill extraction from repeated workflows
- Self-improvement loops
- Long-running agent presence
- User model that deepens over sessions

Use this style when the main problem is making the assistant better at one user's recurring workflows.

Safety Architecture

Personal agents need stronger safety boundaries than chat-only assistants because they can touch private systems.

Minimum controls:

- Identity verification for requests from email, chat, and shared channels
- Tool scopes narrowed by task type
- Human approval before sending messages, spending money, changing access, or deleting data
- Separate memory types for preferences, facts, credentials, and task state
- Audit logs for every external action
- Prompt-injection filters for retrieved documents, emails, and webpages
- Secrets stored outside model-visible context

Failure Modes

- The agent trusts a spoofed sender and acts on private data.
- A chat instruction overrides the user's standing policy.
- The agent stores a temporary fact as durable memory.
- A connector exposes more data than the task needs.
- The user cannot inspect why an action happened.
- The system has no emergency stop or approval mode.

Design Rule

Treat the personal agent like an employee with keys. It needs identity, permissions, training, supervision, logs, and a way to revoke access.

Related Chapters

- Skills

- Memory-Augmented Agent
- Human Approval Gates
- Policy Enforcement
- Agentic System Architecture

Coding Agents

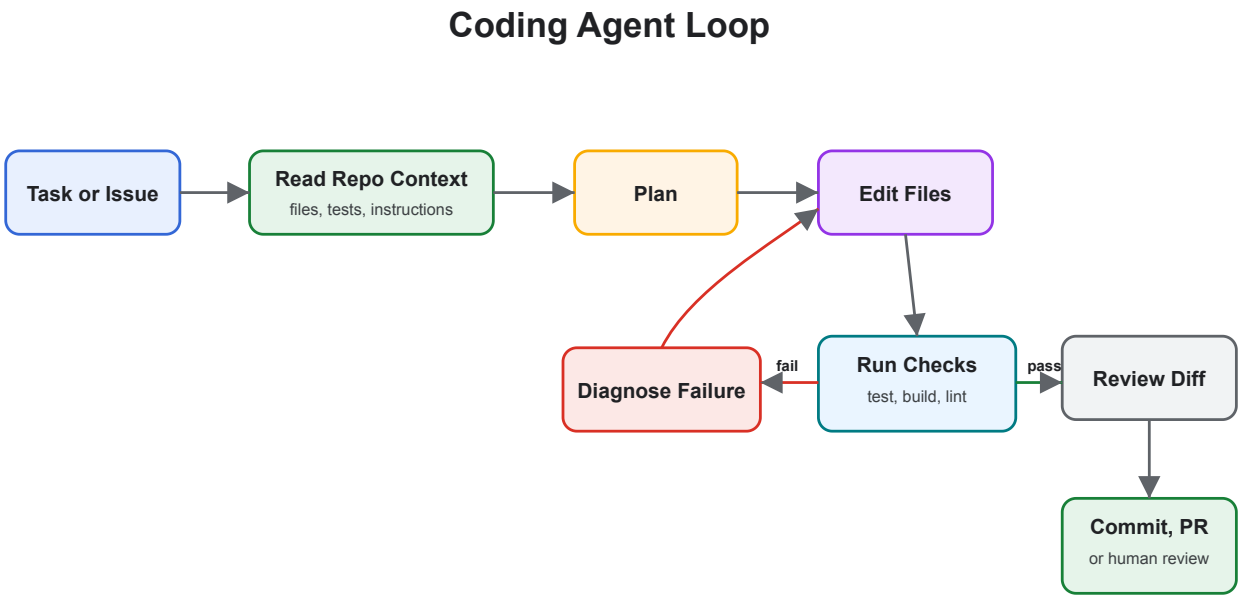
Coding agents operate inside software repositories. They read code, edit files, run commands, inspect failures, produce diffs, and often create commits or pull requests. Codex, Cursor Agent and Cloud Agent, Claude Code, OpenHands, and similar tools are examples of this architecture class.

The pattern is not “AI autocomplete.” It is a controlled development worker with repository context and execution privileges.

Examples

- Codex CLI and Codex IDE extension
- Cursor Agent, Plan Mode, and Cloud Agents
- Claude Code
- OpenHands and OpenHands GitHub

Core Loop



Surfaces

- **Local CLI:** runs near the repository and can use local tools.
- **IDE agent:** shares editor context, selected files, inline diffs, and local commands.

- **Cloud or background agent:** clones or mounts the repository in an isolated environment and returns a branch, diff, or PR.
- **CI/review agent:** reviews pull requests, comments on diffs, or proposes patches.
- **Multi-agent workspace:** runs several agents on separate branches or worktrees.

Architecture Concerns

Coding agents need unusually clear boundaries because they can change source code and run commands.

Design for:

- Repository instructions: coding standards, commands, architecture constraints, and review expectations.
- Workspace isolation: branch, worktree, container, or cloud environment per task.
- Approval policy: which commands and file edits need human approval.
- Test signal: fast checks first, then broader regression checks.
- Diff review: humans inspect changed behavior, not just final prose.
- Secret handling: no credentials in prompts, logs, or generated code.
- Dependency policy: explicit approval before adding packages or changing lockfiles.

Use When

- The task can be verified with tests, builds, type checks, screenshots, or review.
- The desired change can be described in concrete acceptance criteria.
- The agent can inspect enough repository context to follow local patterns.
- You can isolate work and review the resulting diff.

Avoid When

- The repository lacks tests or runnable checks and the change is high risk.
- The task is vague, political, or primarily product discovery.
- The agent needs broad production credentials.
- Multiple agents would edit the same files without coordination.

Operating Patterns

- Ask for a plan before large edits.
- Make the agent cite files and commands it used.
- Prefer small tasks with clear completion criteria.
- Use worktrees or branches for parallel agents.
- Require tests or type checks before commit.

- Review generated code like human code.
- Keep durable repo guidance in a project instruction file.

Failure Modes

- Plausible code that compiles but violates architecture.
- Broad refactors that mix behavior changes with formatting.
- Tests updated to match broken behavior.
- Hidden dependency changes.
- Shell commands that mutate local state unexpectedly.
- Agents fighting over the same files.
- Review fatigue when diffs are too large.

Design Rule

The coding agent should never be the only reviewer of its own code. It can propose, edit, test, and explain. A separate check, reviewer, or policy gate should decide whether the change lands.

Related Chapters

- Goals and State
- Skills
- Human Approval Gates
- Observability and Evals
- Architecture Decision Records for Agents

Computer-Use Agents

Computer-use agents operate software through a user interface when APIs, databases, or workflow tools are unavailable or insufficient. They read screens, choose UI actions, click, type, scroll, upload, download, and inspect results.

Use this pattern only when direct integration is not practical. A UI is the least stable interface an agent can operate.

Intent

Let an agent complete tasks in existing applications by controlling a browser, desktop, terminal, or remote environment under strong sandboxing and human oversight.

Computer-use agents are useful for legacy systems, one-off operational tasks, SaaS tools without APIs, cross-application workflows, and product testing.

Use When

- The system has no usable API.
- The API lacks required functionality.
- The workflow spans several user-facing applications.
- A human currently performs the task through a UI.
- You need to test a product the way a user experiences it.
- The task can tolerate slower execution and occasional recovery.

Avoid When

- A stable API or database integration exists.
- The workflow has high financial, legal, or safety impact without approval.
- The UI changes frequently and cannot be tested.
- Authentication, CAPTCHA, or 2FA blocks automation.
- The agent would need broad access to private screens or files.

If direct tool use is available, prefer MCP-first Tool Use.

Architecture

```
Goal
  -> Task state
```

```
-> Screen or DOM observation
-> UI action proposal
-> Policy and sandbox check
-> Action executor
-> Observation and trace
-> Stop, recover, or continue
```

The action executor should be deterministic. The model proposes an action; software validates and performs it.

Interface Representation

The agent needs a compact representation of the interface.

Common representations:

- screenshot with coordinates;
- accessibility tree;
- DOM snapshot;
- browser automation locator map;
- terminal buffer;
- application event log;
- image plus OCR;
- structured UI state from test instrumentation.

Use the richest structured representation available. Screenshots help when visual layout matters, but DOM or accessibility trees are easier to validate and replay.

Action Space

Keep the action space small and explicit.

Examples:

- click by stable selector;
- type text into a named field;
- select an option;
- upload a file from a sandbox path;
- press a limited key;
- navigate to an allowed URL;
- download to a sandbox directory;
- wait for a condition.

Avoid unrestricted “control the computer” actions unless the environment is disposable and isolated.

State and Recovery

Computer-use agents fail in messy ways:

- modals appear;
- pages load slowly;
- buttons move;
- sessions expire;
- downloads fail;
- validation errors appear;
- the UI changes after deployment.

Design recovery around checkpoints:

- current URL or application state;
- last successful action;
- visible error messages;
- files created or downloaded;
- external side effects;
- retry count;
- human approval state.

The agent should be able to stop with a useful report instead of blindly continuing.

Security Controls

Computer-use agents need strong containment:

- run in an isolated browser profile, container, VM, or remote desktop;
- restrict network destinations;
- isolate downloads and uploads;
- block access to local secrets;
- use scoped credentials;
- record UI actions;
- require approval for irreversible actions;
- clear sessions after runs;
- prevent copy/paste of hidden sensitive data into untrusted sites.

If the agent can see private data and browse untrusted content, treat the workflow as high risk.

Production Checklist

- Is there truly no better API or tool integration?
- Are actions restricted to a known set?
- Can every action be traced and replayed?
- Does the agent run in an isolated environment?
- Are credentials scoped to the task?
- Can the user approve high-risk actions?
- Does the run stop when the UI diverges?
- Are UI changes covered by regression tests?

Related Chapters

- Tool Use
- MCP-first Tool Use
- Agent Security and Sandboxing
- Circuit Breakers, Fallbacks, and Replay
- Coding Agents

Domain Agent Architectures

Domain agents apply agentic patterns inside a specialized field such as healthcare, finance, legal operations, education, science, insurance, or software engineering. The pattern is not "add an agent to a domain." The pattern is to encode the domain's evidence, constraints, risk, and review process into the architecture.

Use this chapter when an agent must operate under domain-specific standards rather than generic productivity expectations.

Intent

Build agents that respect the domain's knowledge sources, vocabulary, decision rights, compliance boundaries, and failure costs.

The architecture should make domain judgment auditable.

Domain Design Questions

Start with the domain, not the model.

- What decisions can the agent make?
- What decisions can it only recommend?
- What evidence is authoritative?
- What evidence is forbidden or insufficient?
- What requires professional review?
- What user data is sensitive?
- What are the domain-specific failure modes?
- What laws, policies, or standards apply?
- What record must be retained?
- What explanation must be shown to users or reviewers?

The answers shape tools, memory, evals, approval gates, and deployment.

Common Domain Shapes

Domain Shape	Agent Role	Architecture Notes
Healthcare or clinical support	Summarize, triage, draft, explain, prepare evidence.	Keep licensed humans in decision roles; cite sources; log recommendations.
Finance or insurance	Analyze documents, classify risk, prepare decisions.	Enforce policy, audit data lineage, require approval for financial actions.

Domain Shape	Agent Role	Architecture Notes
Legal operations	Search, summarize, compare, draft, extract obligations.	Preserve privilege boundaries; show citations; avoid unsupervised legal conclusions.
Education	Tutor, assess, adapt difficulty, produce feedback.	Track learning goals; avoid over-personalized unsupported claims.
Scientific research	Search literature, propose hypotheses, analyze data.	Separate hypothesis generation from validation; track provenance.
Software engineering	Search code, edit, test, review, prepare PRs.	Sandbox execution; preserve diffs; require CI and review gates.

Different domains can share patterns while requiring different risk controls.

Reference Architecture

```
Domain request
  -> identity and authorization
  -> domain router
  -> evidence retrieval
  -> domain tool execution
  -> policy and risk checks
  -> agent or workflow decision
  -> human review when required
  -> auditable output
```

The domain router should select sources, tools, and approval paths. It should not bypass governance.

Evidence Design

Domain agents need explicit evidence policy.

Define:

- authoritative source types;
- freshness requirements;
- citation format;
- source conflict handling;
- minimum evidence for an answer;
- unsupported claim behavior;
- source redaction rules;

- tenant and role boundaries.

If the agent cannot cite or explain the basis for a domain answer, it should lower confidence, ask for clarification, or escalate.

Review and Approval

Domain agents often produce recommendations rather than final decisions.

Use human review for:

- clinical, legal, financial, or compliance decisions;
- low-confidence classifications;
- conflicting evidence;
- exceptions to policy;
- irreversible actions;
- communications that create obligation or liability.

The review interface should show evidence, policy checks, extracted fields, uncertainty, and the proposed action.

Domain Evals

Generic evals are not enough.

Add evals for:

- domain terminology;
- document extraction;
- citation accuracy;
- missing evidence;
- policy exceptions;
- escalation thresholds;
- privacy constraints;
- adversarial or ambiguous inputs;
- reviewer override cases.

Use subject matter experts for labels where judgment matters.

Failure Modes

- The agent sounds authoritative while evidence is weak.
- Domain-specific policy lives only in prompts.
- The agent treats recommendations as decisions.

- Retrieval mixes tenants, jurisdictions, or policy versions.
- Evals measure language quality but not domain correctness.
- Human review lacks enough evidence to make a decision.

Related Chapters

- Agentic RAG Systems
- Policy Enforcement
- Human Approval Gates
- Evaluation-Driven Agent Development
- Knowledge-Bound Agents

Architecture Decision Records for Agents

Agentic systems change quickly. Architecture Decision Records keep model, memory, tool, policy, and workflow choices explicit enough that future maintainers can understand why the system behaves the way it does.

Use ADRs when a decision affects safety, cost, reliability, user trust, or the ability to debug production behavior.

What to Record

Record decisions about:

- Model selection and fallback models
- Tool permissions and approval rules
- Memory retention and deletion
- Retrieval sources and citation policy
- Workflow retries, compensation, and escalation
- Evaluation datasets and release gates
- Observability and logging boundaries
- Human review requirements
- Self-improvement and skill-update policies

ADR Template

```
# ADR-000: Title

## Status

Proposed | Accepted | Superseded

## Context

What problem are we solving? What constraints matter?

## Decision

What did we choose?

## Consequences
```

```
What improves? What gets harder? What risks remain?

## Verification

How will we know this decision is still working?
```

Agent-Specific Additions

Add these fields when relevant:

- **Autonomy level:** advisory, proposes edits, executes after approval, or executes autonomously.
- **Tool scope:** exact tools and operations allowed.
- **State owner:** chat history, workflow state, memory store, database, or external system.
- **Failure policy:** retry, re-plan, ask, refuse, rollback, or escalate.
- **Eval gate:** tests or datasets required before release.
- **Rollback path:** how to disable or reverse the decision.

Example Decisions

- Use a durable workflow for customer-impacting tasks instead of an in-memory loop.
- Require human approval before sending outbound email.
- Store episodic memory for project events but not personal secrets.
- Use hybrid keyword plus vector retrieval for support documentation.
- Run coding agents in disposable worktrees and require `npm test` before commit.
- Keep self-improvement as reviewed skill changes, not silent prompt mutation.

Failure Modes

- Decisions live only in prompts and disappear from engineering review.
- ADRs describe the happy path but not rollback or verification.
- Model upgrades happen without recording eval impact.
- Memory or tool-scope changes ship without privacy review.
- The ADR says “human approval” but not which actions require it.

Related Chapters

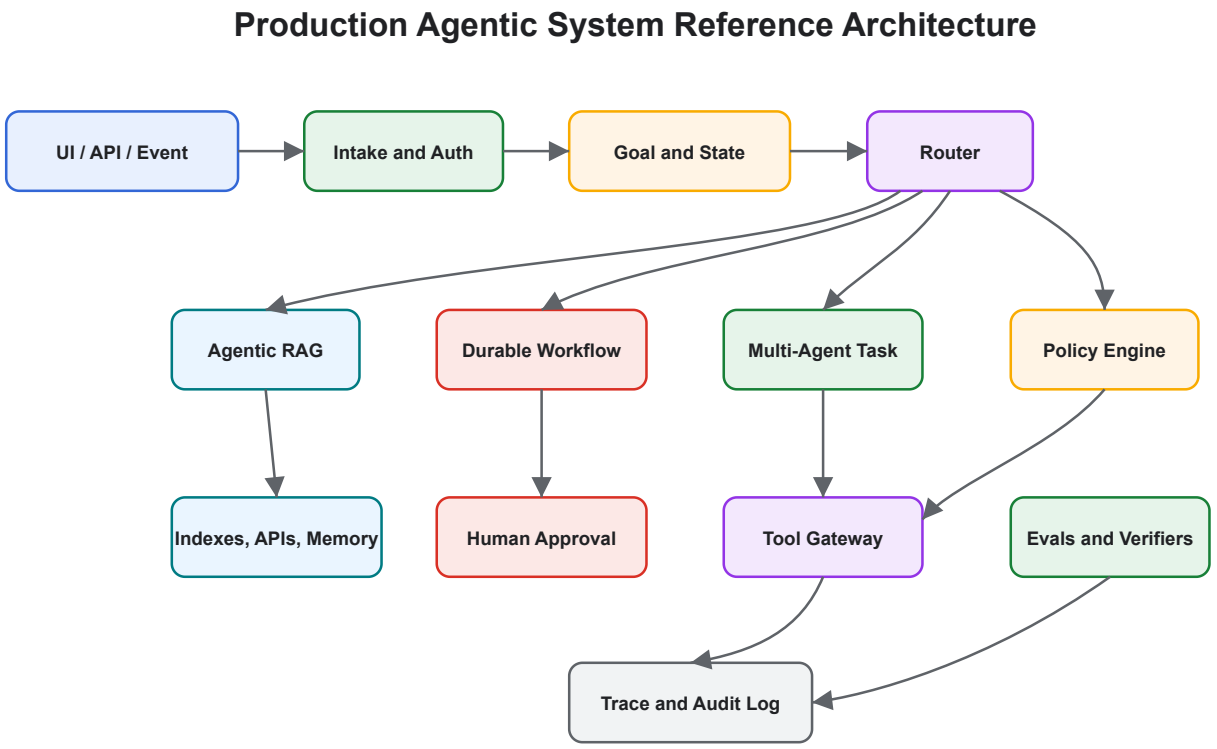
- Agentic System Architecture
- Agentic RAG Systems
- Policy Enforcement

- Observability and Evals

Reference Architecture

This reference architecture combines the core patterns in the book into a production-ready agentic system. It is intentionally conservative: deterministic software owns state, policy, and side effects; model calls propose decisions inside bounded loops.

Architecture



Request Flow

1. Authenticate the user or event source.
2. Create a goal record with constraints, user scope, and requested outcome.
3. Route the task to a direct answer, Agentic RAG, durable workflow, or multi-agent process.
4. Run policy checks before retrieval, tool use, or side effects.
5. Persist observations, decisions, tool calls, and outputs.
6. Verify evidence, output quality, and policy compliance.
7. Return an answer, request approval, refuse, or escalate.
8. Store traces for debugging and eval dataset improvement.

Control Points

- **Before retrieval:** enforce access control and source eligibility.
- **Before tool use:** validate schema, permission, budget, and approval needs.
- **Before final answer:** verify claims, citations, and policy.
- **Before memory write:** classify what kind of memory is being stored.
- **Before release:** run evals and regression checks.

Runtime Components

- Identity and tenant boundary
- Goal and state store
- Prompt and instruction registry
- Tool gateway
- Retrieval router
- Memory service
- Policy engine
- Approval service
- Workflow engine
- Evals service
- Trace and audit store

Minimum Production Checklist

- Explicit owner for goal and state
- Tool schemas and validation
- Human approval for high-risk actions
- Access-controlled retrieval
- Citation validation for grounded answers
- Evals for core tasks
- Traces for every run
- Budget, timeout, and cancellation controls
- Incident review path
- Rollback or disable switch

Scaling Path

Start small:

1. Single agent with tool validation.
2. Add goals and state.
3. Add retrieval and citation checks.
4. Add durable workflow for long-running tasks.
5. Add human approval gates.

6. Add eval datasets and observability.
7. Add multi-agent decomposition only when one agent becomes a bottleneck.

Related Chapters

- Agentic System Architecture
- Agentic RAG Systems
- MCP-first Tool Use
- Durable Workflows
- Policy Enforcement
- Observability and Evals

Durable Workflows

Durable workflows make agentic systems resumable and auditable by owning retries, checkpoints, approvals, compensation, and long-running state.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

The Durable Workflow Pattern wraps agent steps in a resumable execution model. The workflow owns ordering, retries, persisted state, approvals, and compensation; agents perform bounded work inside workflow steps.

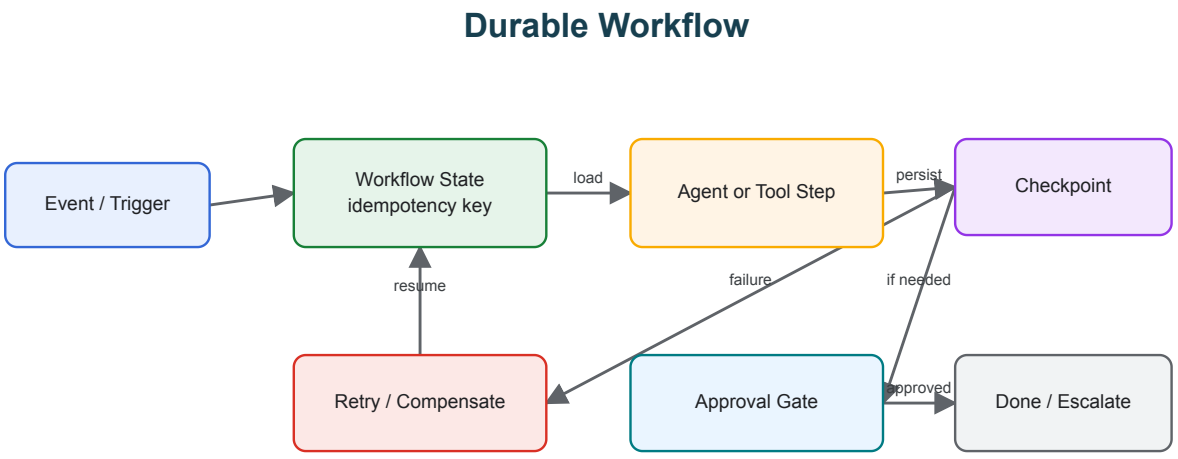
Use When

- Work spans minutes, hours, or external approvals.
- Tool calls may fail and must be retried safely.
- You need auditability, resumability, and operational control.

Avoid When

- The task is a short interactive chat response.
- State does not need to survive process restarts.
- The workflow engine would add more complexity than the task deserves.

Architecture



System Shape

- **Pattern boundary:** a production service or framework hosts the agent behind durable workflow, policy, observability, and deployment boundaries.
- **State owner:** the runtime owns durable state, retries, traces, triggers, deployment configuration, and operational controls.
- **Primary artifact:** `durable-workflow-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Durable workflows make agentic systems resumable and auditable by owning retries, checkpoints, approvals, compensation, and long-running state.

Core Protocol

1. Receive a user request, event, schedule, or workflow step with an idempotency key.
2. Load durable state, policy context, memory, and runtime configuration.
3. Execute one bounded step through the agent, tool, or workflow engine.
4. Checkpoint result, trace data, cost, and error state.
5. Retry, compensate, continue, or escalate according to operational policy.

Implementation Notes

- Persist state after every externally visible action.
- Make steps idempotent or attach idempotency keys to tool calls.
- Separate orchestration state from model conversation state.
- Model calls should be retryable only when repeating them cannot duplicate side effects.

Failure Modes

- Retrying side-effectful steps without idempotency.
- Losing human approval state after deployment or restart.
- Workflows that hide agent uncertainty behind a successful task status.
- No compensation path for partially completed external actions.

Evaluation Strategy

- Replay production-like traces through regression evals before deployment.
- Test retries, duplicate events, partial outages, policy denial, and human approval waits.
- Measure reliability, recovery time, cost, latency, user impact, and eval regression rate.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Use durable checkpoints for long-running or externally visible work.
- Add structured traces, metrics, cost tracking, and replay data.
- Define deployment rollback and feature-flag strategy.
- Document operational ownership, alerts, and escalation paths.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

This pattern currently has no dedicated code excerpt. Use the source and download links below for the full pattern folder.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `durable-workflow-pattern/` folder from this repository.

Related Patterns

- Goals and State
- Self-Healing Workflow
- Human-in-the-Loop Approval

Observability and Evals

Observability records what happened. Evals decide whether behavior is good enough.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

The Observability and Evals Pattern makes agent behavior inspectable and testable. It captures traces, tool calls, prompts, model outputs, costs, latencies, and evaluation results so teams can debug and improve systems over time.

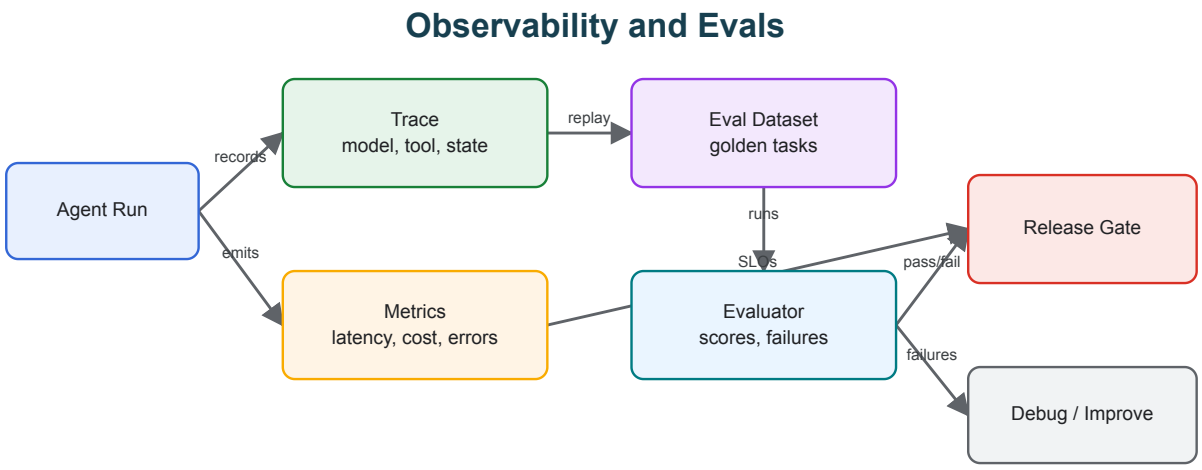
Use When

- Agent decisions affect users, money, data, or external systems.
- You need regression tests for prompts, tools, routing, or workflows.
- Failures are hard to reproduce from final answers alone.

Avoid When

- You cannot store traces safely because of privacy or regulatory constraints.
- The prototype is throwaway and has no operational users.
- You only log final answers and call that observability.

Architecture



System Shape

- **Pattern boundary:** a production service or framework hosts the agent behind durable workflow, policy, observability, and deployment boundaries.
- **State owner:** the runtime owns durable state, retries, traces, triggers, deployment configuration, and operational controls.
- **Primary artifact:** `observability-and-evals-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Observability records what happened. Evals decide whether behavior is good enough.

Core Protocol

1. Receive a user request, event, schedule, or workflow step with an idempotency key.
2. Load durable state, policy context, memory, and runtime configuration.
3. Execute one bounded step through the agent, tool, or workflow engine.
4. Checkpoint result, trace data, cost, and error state.
5. Retry, compensate, continue, or escalate according to operational policy.

Implementation Notes

- Trace at the level of run, loop iteration, model call, tool call, workflow step, and evaluator result.
- Store enough input/output detail to reproduce failures, with redaction for sensitive data.
- Maintain golden datasets for routing, structured outputs, tool plans, and final answers.
- Treat eval failures as release blockers for production agents.

Failure Modes

- Logs that omit the prompt, tool input, or model configuration.
- Evals that only check happy paths.
- Metrics without trace IDs, making incidents hard to investigate.
- Storing sensitive data without retention or redaction rules.

Evaluation Strategy

- Replay production-like traces through regression evals before deployment.
- Test retries, duplicate events, partial outages, policy denial, and human approval waits.
- Measure reliability, recovery time, cost, latency, user impact, and eval regression rate.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Use durable checkpoints for long-running or externally visible work.
- Add structured traces, metrics, cost tracking, and replay data.
- Define deployment rollback and feature-flag strategy.
- Document operational ownership, alerts, and escalation paths.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

This pattern currently has no dedicated code excerpt. Use the source and download links below for the full pattern folder.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `observability-and-evals-pattern/` folder from this repository.

Related Patterns

- Evaluator-Optimizer
- Mastra Runtime
- Compliance/Policy Enforcer

Policy Enforcement

Policy enforcement constrains what the agent may say or do through permissions, data-access rules, business rules, safety rules, and escalation.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Policy enforcement constrains what the agent may say or do through permissions, data-access rules, business rules, safety rules, and escalation.

Use When

- Actions must be checked before execution.
- The agent handles regulated, private, or business-critical data.
- Policy decisions must be auditable.

Avoid When

- Policy is only written as prompt text with no runtime enforcement.
- The system cannot identify the actor, resource, action, and context.
- Exceptions are silent or unreviewed.

System Shape

- **Pattern boundary:** a production service or framework hosts the agent behind durable workflow, policy, observability, and deployment boundaries.
- **State owner:** the runtime owns durable state, retries, traces, triggers, deployment configuration, and operational controls.
- **Primary artifact:** `compliance-policy-enforcer-agent/` contains the runnable reference implementation and examples.
- **Operational promise:** Policy enforcement constrains what the agent may say or do through permissions, data-access rules, business rules, safety rules, and escalation.

Core Protocol

1. Receive a user request, event, schedule, or workflow step with an idempotency key.
2. Load durable state, policy context, memory, and runtime configuration.
3. Execute one bounded step through the agent, tool, or workflow engine.
4. Checkpoint result, trace data, cost, and error state.
5. Retry, compensate, continue, or escalate according to operational policy.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Replay production-like traces through regression evals before deployment.
- Test retries, duplicate events, partial outages, policy denial, and human approval waits.
- Measure reliability, recovery time, cost, latency, user impact, and eval regression rate.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Use durable checkpoints for long-running or externally visible work.
- Add structured traces, metrics, cost tracking, and replay data.
- Define deployment rollback and feature-flag strategy.
- Document operational ownership, alerts, and escalation paths.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

This pattern currently has no dedicated code excerpt. Use the source and download links below for the full pattern folder.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `compliance-policy-enforcer-agent/` folder from this repository.

Related Patterns

- Durable Workflows
- Observability and Evals
- Event-Triggered Agents
- Choosing the Right Pattern
- Resource-Aware Agent Design

Event-Triggered Agents

Event-triggered agents run in response to webhooks, queues, schedules, or domain events.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

Event-triggered agents run in response to webhooks, queues, schedules, or domain events.

Use When

- A domain event should start bounded agent work.
- The task can be idempotent and observable.
- No human may be watching the interaction live.

Avoid When

- The event payload lacks enough context to act safely.
- Duplicate delivery could cause duplicate side effects.
- Failures cannot be retried or dead-lettered.

System Shape

- **Pattern boundary:** a production service or framework hosts the agent behind durable workflow, policy, observability, and deployment boundaries.
- **State owner:** the runtime owns durable state, retries, traces, triggers, deployment configuration, and operational controls.
- **Primary artifact:** `event-triggered-agent-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Event-triggered agents run in response to webhooks, queues, schedules, or domain events.

Core Protocol

1. Receive a user request, event, schedule, or workflow step with an idempotency key.
2. Load durable state, policy context, memory, and runtime configuration.

3. Execute one bounded step through the agent, tool, or workflow engine.
4. Checkpoint result, trace data, cost, and error state.
5. Retry, compensate, continue, or escalate according to operational policy.

Implementation Notes

- Keep the pattern boundary explicit: inputs, state, side effects, and outputs should be visible.
- Validate model-produced decisions before they affect tools, users, or durable state.
- Emit enough trace data to debug failures after the run.

Failure Modes

- The pattern is applied where a simpler deterministic workflow would be better.
- State, tool calls, or model decisions are not observable enough to debug.
- The system lacks clear stop, retry, or escalation behavior.

Evaluation Strategy

- Replay production-like traces through regression evals before deployment.
- Test retries, duplicate events, partial outages, policy denial, and human approval waits.
- Measure reliability, recovery time, cost, latency, user impact, and eval regression rate.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Use durable checkpoints for long-running or externally visible work.
- Add structured traces, metrics, cost tracking, and replay data.
- Define deployment rollback and feature-flag strategy.
- Document operational ownership, alerts, and escalation paths.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

This pattern currently has no dedicated code excerpt. Use the source and download links below for the full pattern folder.

Download

- Download source bundle
- Open source folder

The download bundle contains the current `event-triggered-agent-pattern/` folder from this repository.

Related Patterns

- Durable Workflows
- Observability and Evals
- Policy Enforcement
- Choosing the Right Pattern
- Resource-Aware Agent Design

Mastra Runtime

Mastra is a TypeScript runtime pattern for applications that need agents, workflows, tools, memory, evals, and observability in one framework.

> Source and downloads
>
> - Repository source
> - Download code bundle

Intent

The Mastra Runtime Pattern uses Mastra as a TypeScript runtime for production agent applications. Mastra gives agents, workflows, tools, memory, evals, and observability a shared application structure.

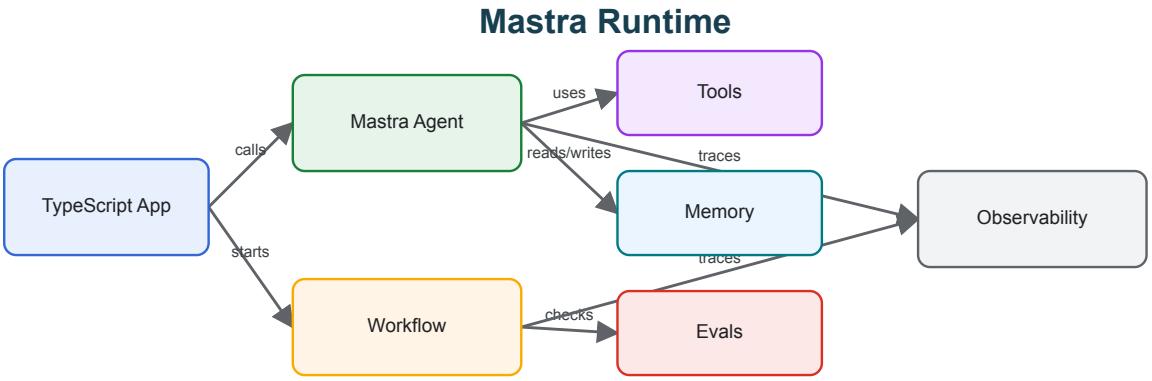
Use When

- You are building a TypeScript or Node-based agent product.
- You need agents and deterministic workflows in the same runtime.
- You want memory, tools, evals, and tracing to be first-class concerns.

Avoid When

- You only need a small script or single model call.
- Your team is committed to a Python-first agent stack.
- You cannot accept framework conventions around project structure and deployment.

Architecture



System Shape

- **Pattern boundary:** a production service or framework hosts the agent behind durable workflow, policy, observability, and deployment boundaries.
- **State owner:** the runtime owns durable state, retries, traces, triggers, deployment configuration, and operational controls.
- **Primary artifact:** `mastra-runtime-pattern/` contains the runnable reference implementation and examples.
- **Operational promise:** Mastra is a TypeScript runtime pattern for applications that need agents, workflows, tools, memory, evals, and observability in one framework.

Core Protocol

1. Receive a user request, event, schedule, or workflow step with an idempotency key.
2. Load durable state, policy context, memory, and runtime configuration.
3. Execute one bounded step through the agent, tool, or workflow engine.
4. Checkpoint result, trace data, cost, and error state.
5. Retry, compensate, continue, or escalate according to operational policy.

Implementation Notes

- Use agents for open-ended decisions where the next step is not known upfront.
- Use workflows for predetermined control flow, state transitions, retries, and production orchestration.
- Keep tools typed and independently testable.
- Capture traces and evals from the beginning rather than adding them after failures.

Failure Modes

- Treating the framework as the architecture instead of modeling goals, state, and failure modes.
- Putting deterministic workflow logic inside prompts.
- Creating tools with vague descriptions and unvalidated inputs.
- Shipping without eval datasets or trace review.

Evaluation Strategy

- Replay production-like traces through regression evals before deployment.
- Test retries, duplicate events, partial outages, policy denial, and human approval waits.
- Measure reliability, recovery time, cost, latency, user impact, and eval regression rate.
- Include cases that prove each “Use When” condition is true for this pattern.
- Include negative cases from “Avoid When” so the system chooses a simpler or safer pattern when appropriate.

Production Checklist

- Use durable checkpoints for long-running or externally visible work.
- Add structured traces, metrics, cost tracking, and replay data.
- Define deployment rollback and feature-flag strategy.
- Document operational ownership, alerts, and escalation paths.
- Define human escalation for ambiguous, high-risk, or policy-blocked work.
- Keep the source bundle, generated chapter, tests, and deployment artifact in the same release.

Code Walkthrough

Read the excerpt as the smallest executable expression of the pattern. The surrounding chapter explains the design constraints; the code shows where those constraints become concrete interfaces, state, validation, or control flow.

Source Code

This pattern currently has no dedicated code excerpt. Use the source and download links below for the full pattern folder.

Download

- Download source bundle

- Open source folder

The download bundle contains the current `mastra-runtime-pattern/` folder from this repository.

Related Patterns

- Durable Workflow
- Observability and Evals
- Agent Loop

Deprecated / Historical Patterns

Deprecated patterns are preserved in the repository but removed from the active learning path.

They are either speculative, too broad, duplicated by stronger chapters, or currently too thin to stand as active patterns.

- Agent Marketplace: speculative compared with A2A, routing, and orchestration.
- Agent Swarm: folded into Parallel Agents, search, and optimization.
- Hybrid Agent: too broad to teach as a standalone pattern.
- Meta-Cognitive Agent: folded into Reflection, Evaluator-Optimizer, and Self-Improvement.
- Recursive Agent: folded into Planning, Goals and State, and Task Delegation.
- Distributed Agent: replaced by A2A, durable workflows, and protocol-first composition.
- API Integration Copilot: archived as an applied placeholder.
- Data Pipeline Orchestrator Agent: archived as an applied placeholder.
- Multi-Modal Tool-Using Agent: archived until it has a developed multimodal implementation.

Source archive: `deprecated`